

《计算机组成原理实验》

(第三次)

厦门大学信息学院

2026年4月

目录

一、存储器组件验证实验

- 1、ROM存储器组件电路
- 2、RAM存储器组件电路（同步模式）
- 3、RAM存储器组件电路（异步模式）
- 4、RAM存储器组件电路（分离模式）

二、MIPS RAM 存储器验证实验

- 1、MIPS RAM 存储器电路
- 2、简单的MIPS RAM 存储器测试电路
- 3、复杂的MIPS RAM 存储器测试电路

三、存储器扩展设计实验

- 1、存储器扩展电路（设计实验）
- 2、存储器扩展测试电路

四、MIPS 寄存器文件验证实验

- 1、MIPS 寄存器文件电路
- 2、简单的MIPS 寄存器文件测试电路
- 3、复杂的MIPS 寄存器文件测试电路
- 4、MIPS 寄存器文件电路（R0=0）
- 5、简单的MIPS 寄存器文件测试电路（R0=0）
- 6、复杂的MIPS 寄存器文件测试电路（R0=0）

五、寄存器文件设计实验

- 1、寄存器文件电路（4个寄存器）（设计实验）
- 2、寄存器文件电路（4个寄存器，R0=0）（设计实验）

六、cache 验证实验

- 1、直接相联映射方式 cache 电路
- 2、直接相联映射方式 cache 测试电路
- 3、全相联映射方式 cache 电路
- 4、全相联映射方式 cache 测试电路
- 5、2路组相联映射方式 cache 电路
- 6、2路组相联映射方式 cache 测试电路
- 7、4路组相联映射方式 cache 电路
- 8、4路组相联映射方式 cache 测试电路

七、cache 设计实验（挑战性实验，从以下4组中任选1组）

第1组：直接相联映射方式 cache 电路（4个cache块）
全相联映射方式 cache 电路（16个cache块）

第2组：直接相联映射方式 cache 电路（16个cache块）
全相联映射方式 cache 电路（4个cache块）

第3组：2路组相联映射方式 cache 电路（16个cache块）
直接相联映射方式 cache 电路（16个cache块）

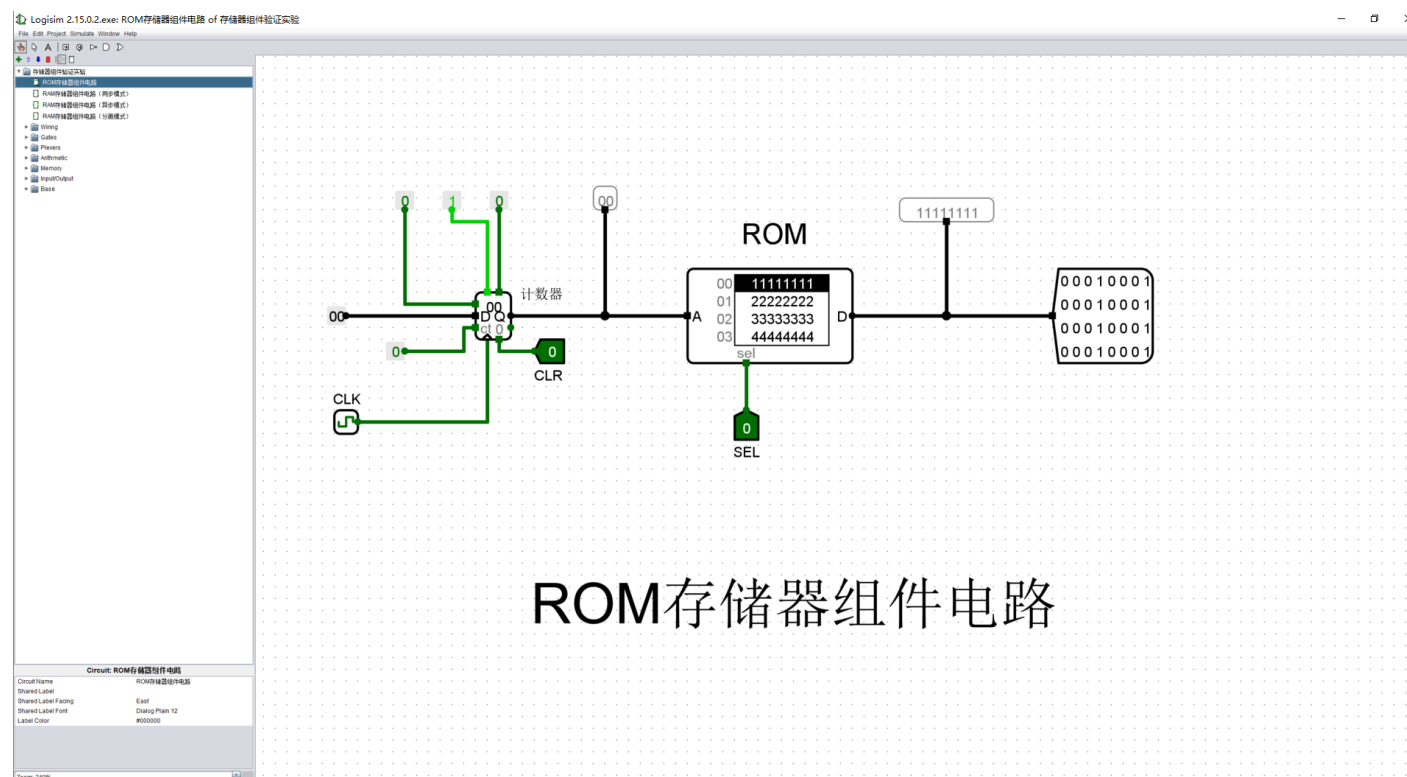
第4组：全相联映射方式 cache 电路（16个cache块）
8路组相联映射方式 cache 电路（16个cache块）

参考设计文件：存储器组件验证实验.circ

一、存储器组件验证实验

- 1、ROM存储器组件电路
- 2、RAM存储器组件电路（同步模式）
- 3、RAM存储器组件电路（异步模式）
- 4、RAM存储器组件电路（分离模式）

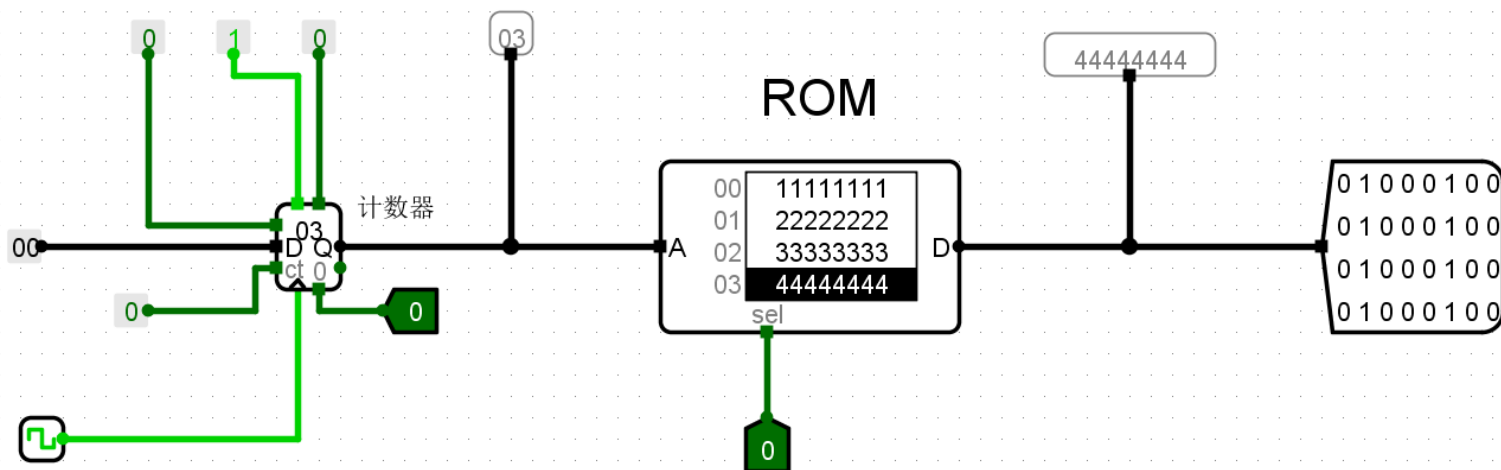
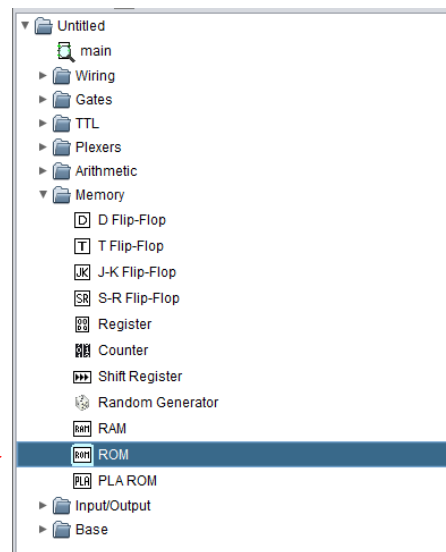
- 运行“**Logisim**”，在Logisim中打开“**存储器组件验证实验.circ**”。
- 也可以直接双击“**存储器组件验证实验.circ**”，打开该设计文件。

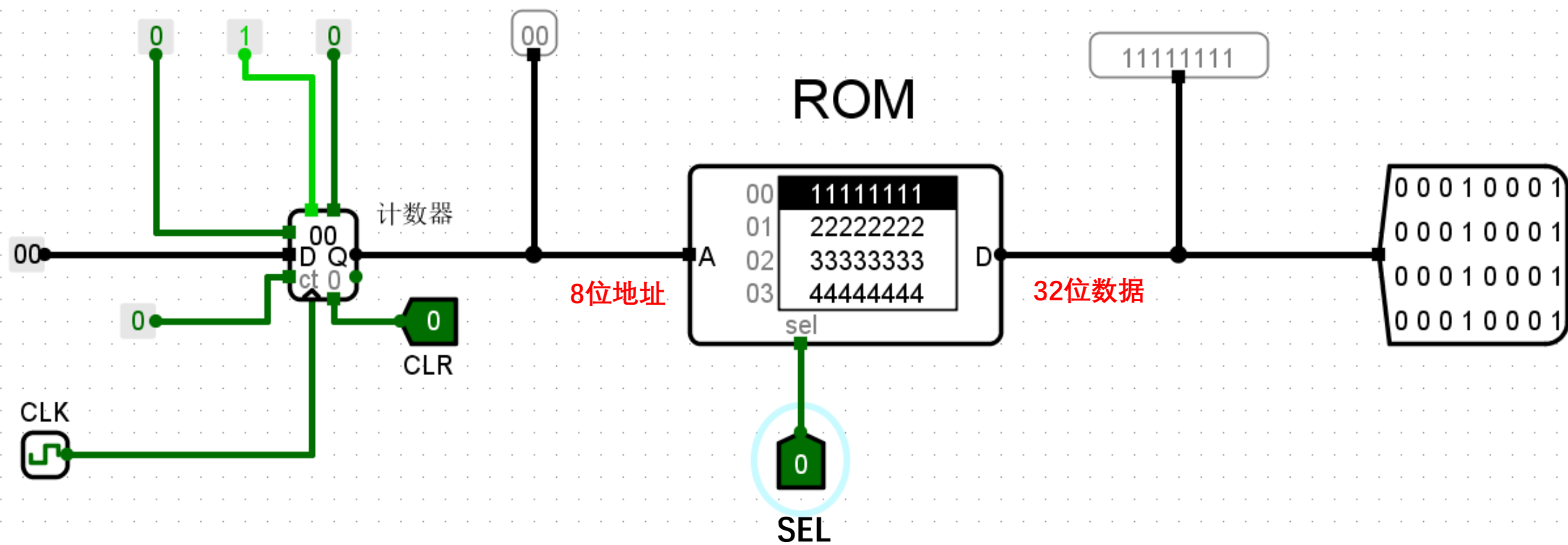


• 1、ROM存储器组件电路

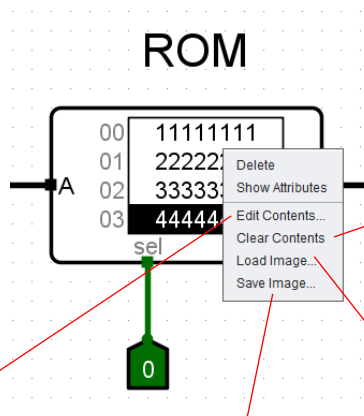
- Logisim中有**ROM存储器组件**，其输入、输出及控制端如下：
 - 输入：地址A（例如设置为8位）。
 - 输出：数据D（例如设置为32位）。
 - 控制端（输入）：片选信号sel；**sel=0**，ROM工作；**sel=1**，ROM不工作。
- 可以使用**8位计数器**为ROM提供地址，具体电路如下：

Logisim中的
ROM存储器组件

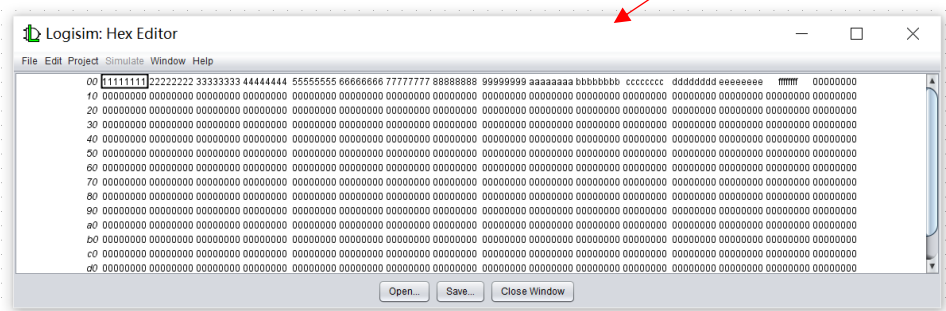




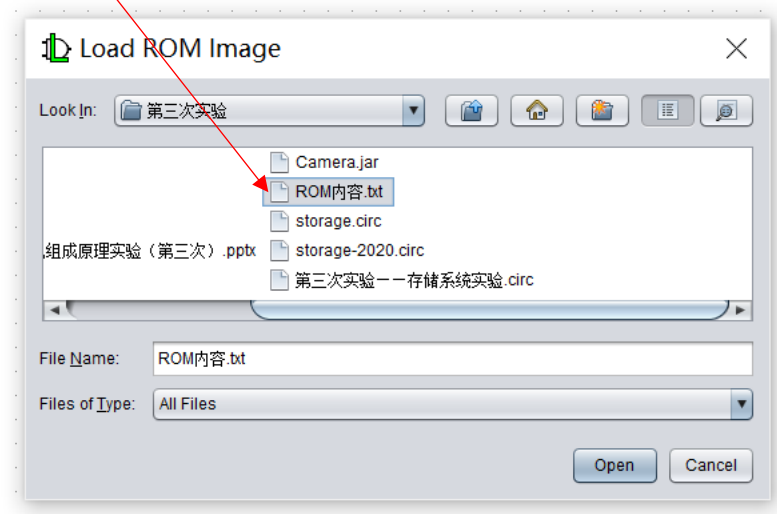
ROM存储器组件电路



清除ROM中的内容



查看、修改ROM中的内容



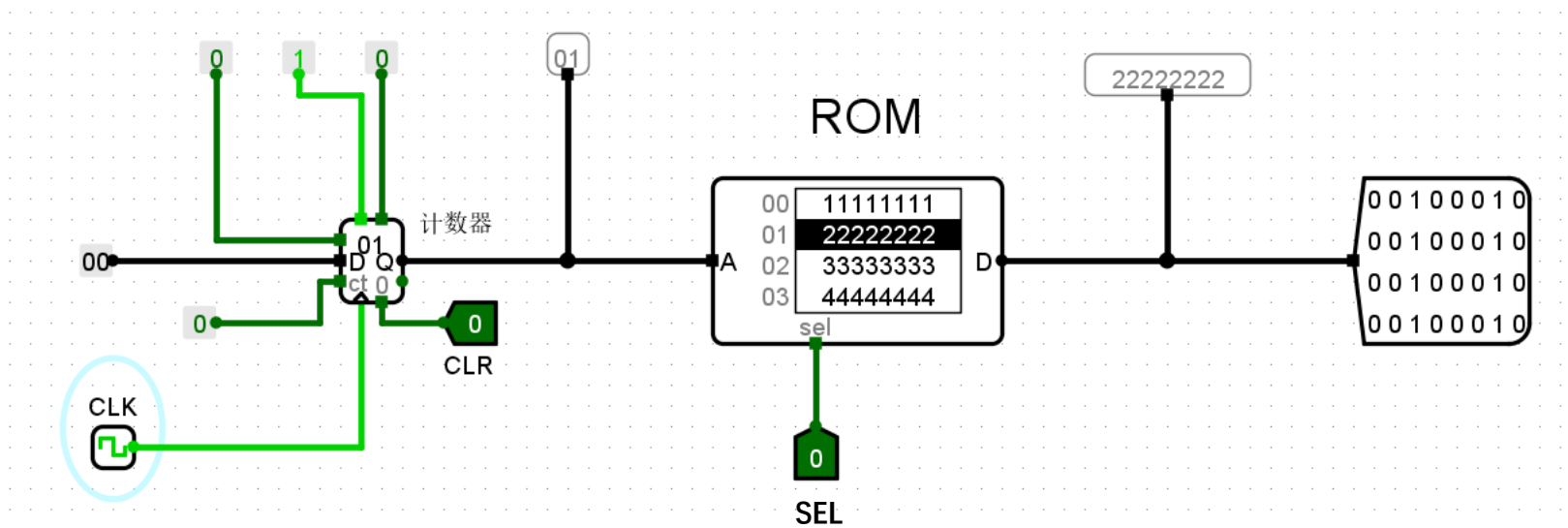
保存ROM中的内容到文件

从文件中装入ROM中的内容

• 实验步骤:

- ① 向ROM中装入内容，具体内容在“ROM内容.txt”文件中；
- ② 置CLR=0、SEL=0；然后点击CLK，则改变地址A的值，观察对应的输出数据D；
- ③ 置CLR=1，让地址A变为0；再置CLR=0，然后点击CLK，则重新开始；
- ④ 置SEL=1，则ROM存储器不工作。

ROM内容.txt - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
v2.0 raw
11111111
22222222
33333333
44444444
55555555
66666666
77777777
88888888
99999999
aaaaaaaa
bbbbbbbb
cccccccc
dddddddd
eeeeeeee
ffffff



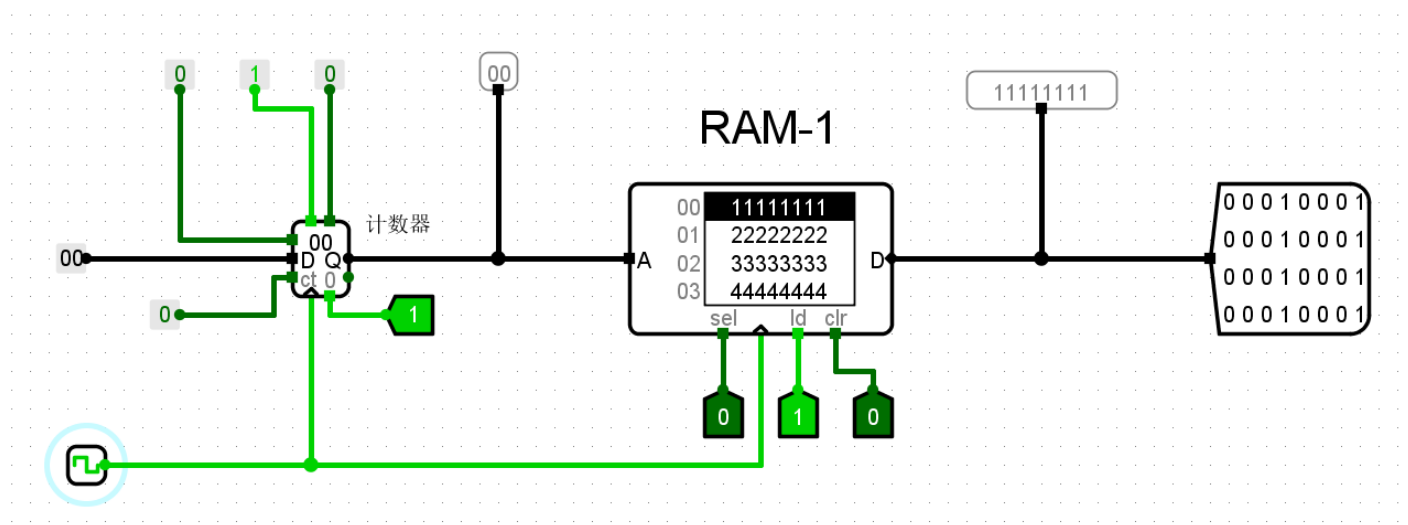
• 2、RAM存储器组件电路（同步模式：具有同步时钟clk）

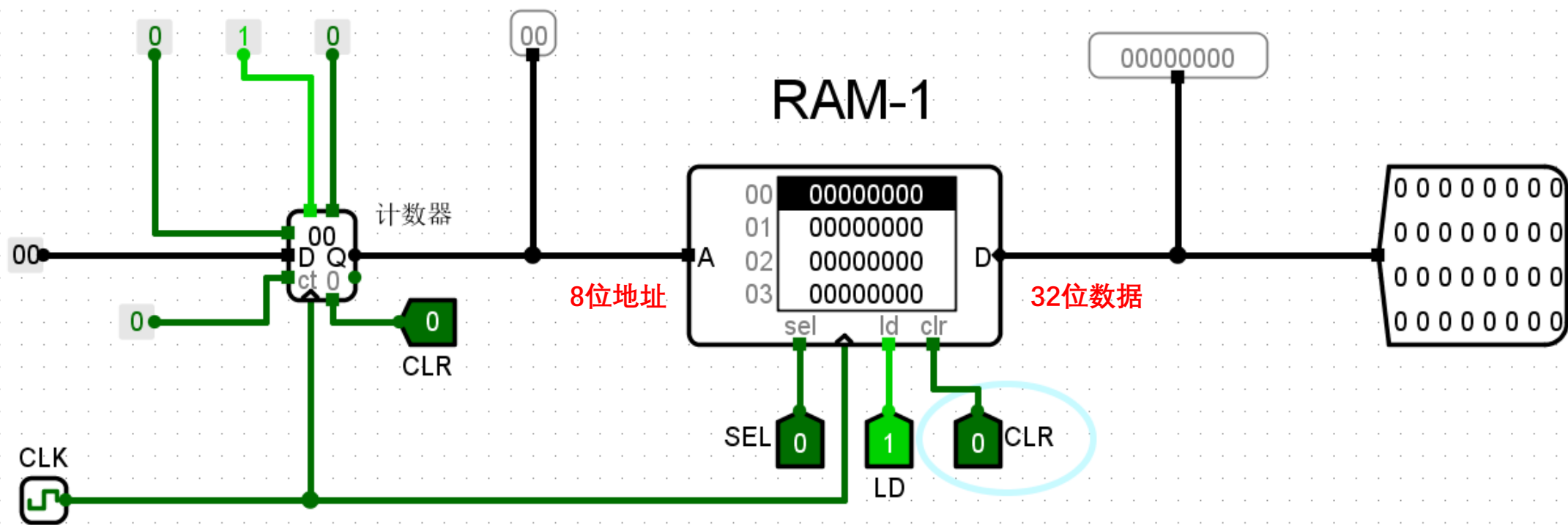
- Logisim中有**RAM存储器组件**，其有3种工作模式。
- 第1种工作模式为：**同步模式**，其输入、输出及控制端如下：
 - 输入：地址A（例如设置为8位）。
 - 输出：数据D（例如设置为32位）。
 - 控制端（输入）：
 - sel: 片选信号；**sel=0**，RAM工作；**sel=1**，RAM不工作；
 - ld: 输出允许信号；**ld=1**，RAM工作；**ld=0**，RAM不工作；
 - clr: 清0信号；**clr=1**，RAM的内容清0；**clr=0**，RAM内容不清0；
 - clk: 时钟。

RAM	
Address Bit Width	8
Data Bit Width	32
Label	
Label Font	Dialog Plain 12
Label Color	#000000
Data Interface	One synchronous load/store port
Sel Active On:	Low Level

同步模式

- 可以使用**8位计数器**为RAM提供地址，具体电路如下；同步模式的RAM只有读出操作，不能进行写入操作；具体实验步骤与ROM存储器类似。





RAM存储器组件电路（同步模式）

• 3、RAM存储器组件电路（异步模式：没有同步时钟clk）

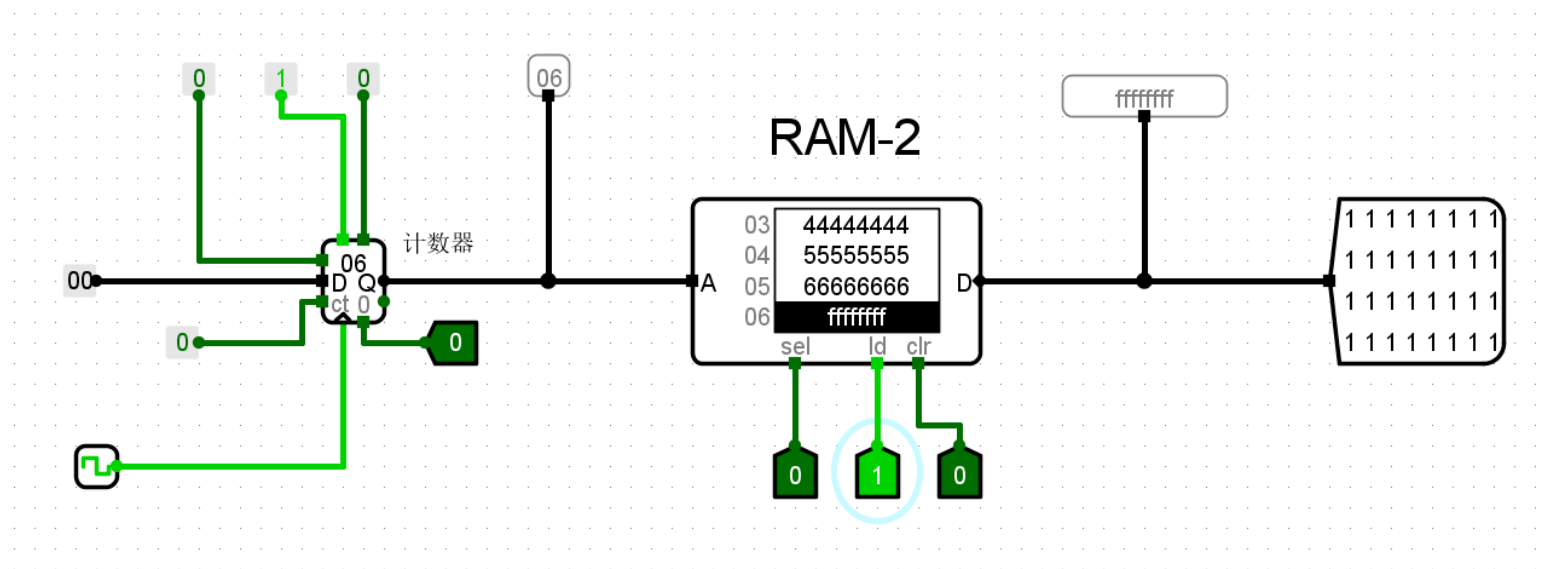
- 第2种工作模式为：**异步模式**，其输入、输出及控制端如下：

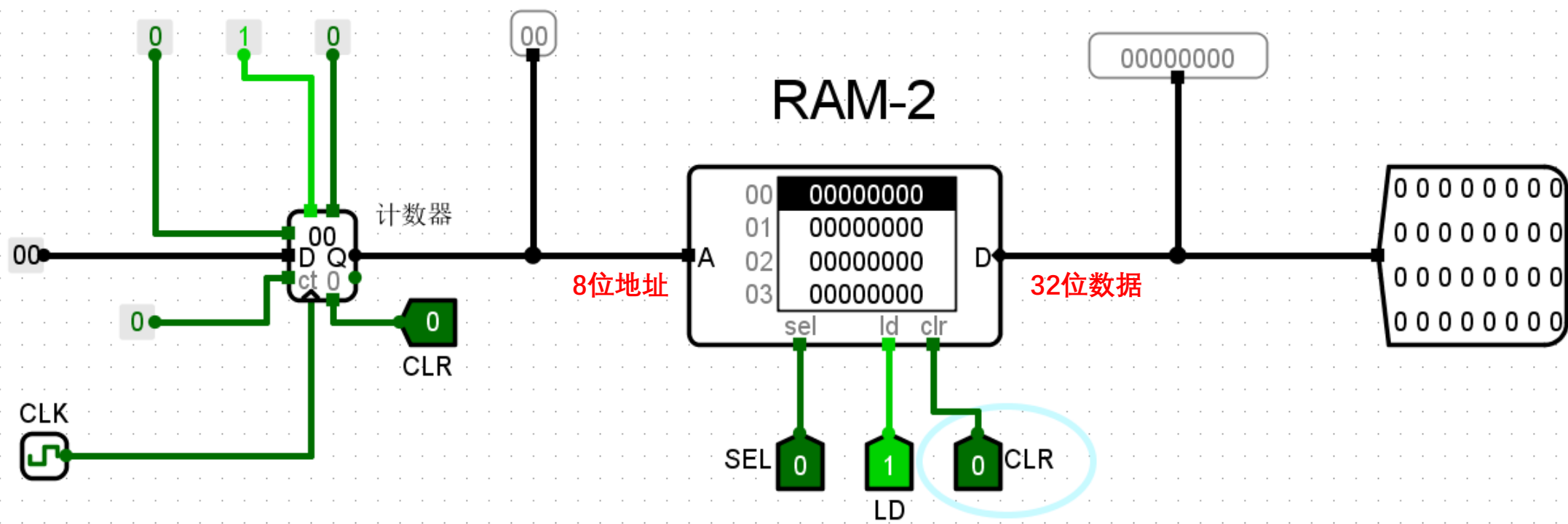
- 输入：地址A（例如设置为8位）。
- 输出：数据D（例如设置为32位）。
- 控制端（输入）：
 - sel：片选信号；**sel=0**，RAM工作；sel=1，RAM不工作；
 - ld：输出允许信号；**ld=1**，RAM工作；ld=0，RAM不工作；
 - clr：清0信号；clr=1，RAM的内容清0；**clr=0**，RAM内容不清0；
 - 没有时钟信号。**

RAM	
Address Bit Width	8
Data Bit Width	32
Label	
Label Font	Dialog Plain 12
Label Color	#000000
Data Interface	One asynchronous load/store port
Sel Active On:	Low Level

异步模式

- 可以使用**8位计数器**为RAM提供地址，具体电路如下；异步模式的RAM只有读出操作，不能进行写入操作；具体实验步骤与ROM存储器类似。





RAM存储器组件电路（异步模式）

• 4、RAM存储器组件电路（分离模式：可读可写，有同步时钟clk）

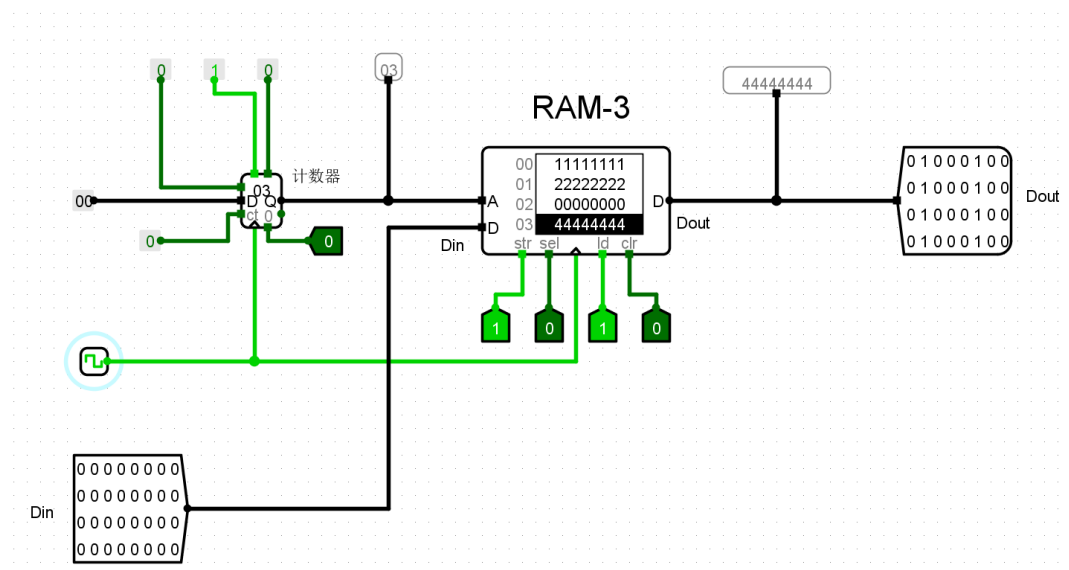
• 第3种工作模式为：分离模式，其输入、输出及控制端如下：

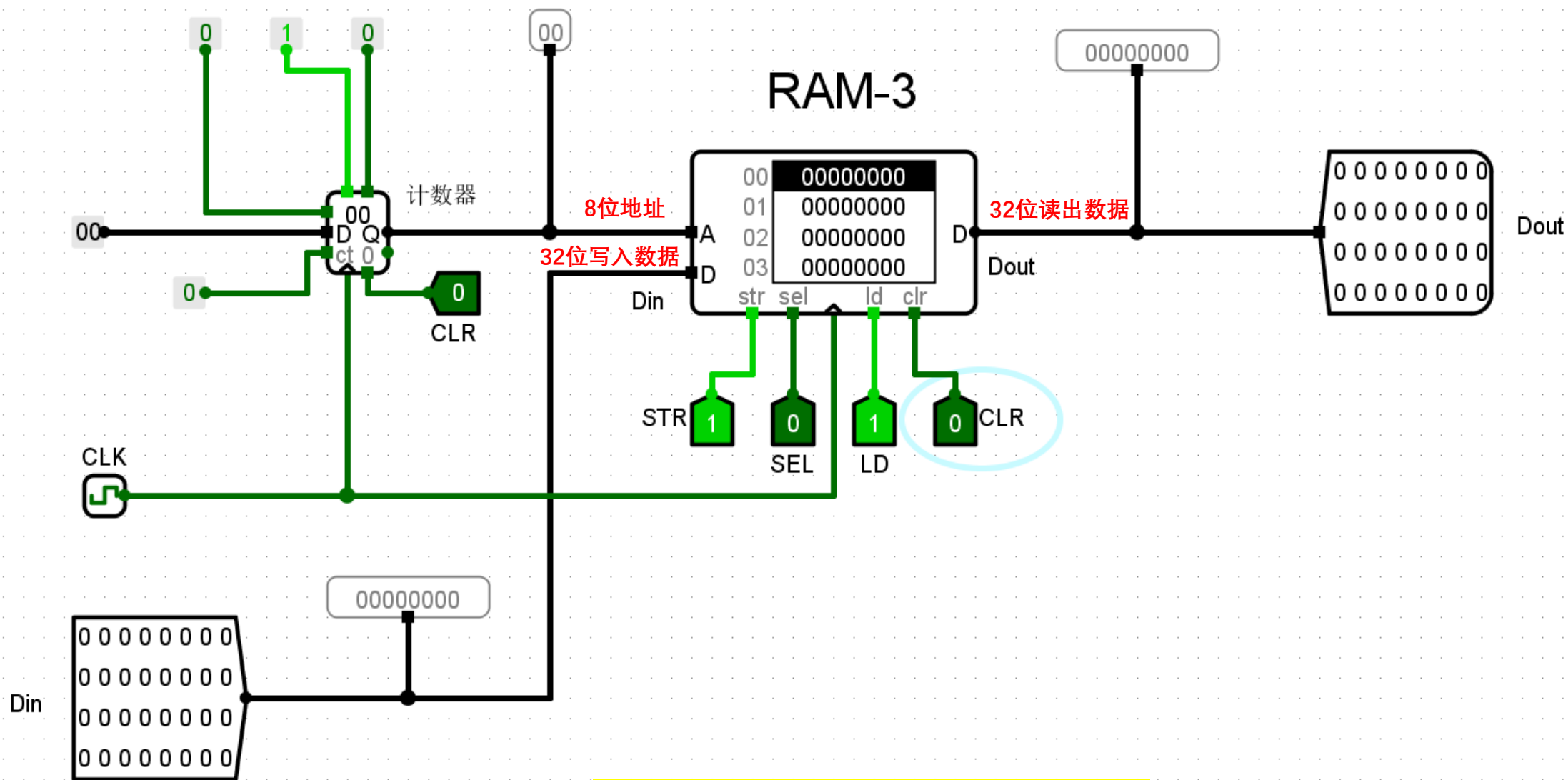
- 输入：地址A（例如设置为8位），数据Din（例如设置为32位）。
- 输出：数据Dout（例如设置为32位）。
- 控制端（输入）：
 - sel：片选信号；sel=0，RAM工作；sel=1，RAM不工作；
 - ld：输出允许信号；ld=1，RAM工作；ld=0，RAM不工作；
 - clr：清0信号；clr=1，RAM的内容清0；clr=0，RAM内容不清0；
 - str：输入允许信号；str=1，将Din的内容装入地址A的RAM单元；str=0，不装入；
 - clk：时钟。

RAM	
Address Bit Width	8
Data Bit Width	32
Label	
Label Font	Dialog Plain 12
Label Color	#000000
Data Interface	Separate load and store ports
Sel Active On:	Low Level

分离模式

- 可以使用8位计数器为RAM提供地址，具体电路如下；分离模式的RAM可以进行读写操作；str=1时，为写入操作；str=0时，为读出操作；具体实验步骤与ROM存储器类似。





RAM存储器组件电路（分离模式）

- 请同学们对上述4种存储器进行读写操作：

① ROM存储器；

② RAM存储器（同步模式）；

③ RAM存储器（异步模式）；

④ RAM存储器（分离模式）。

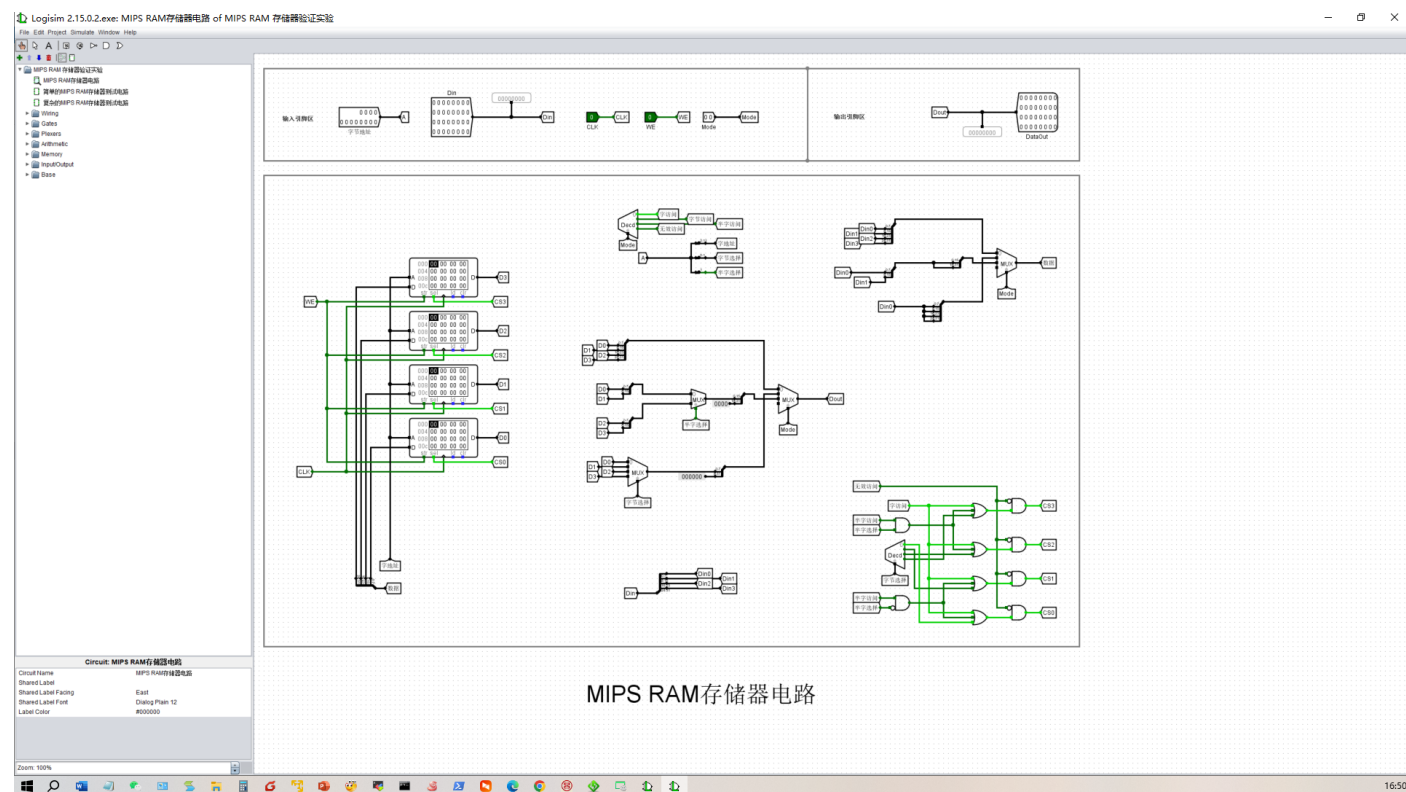
- 包括：将文本文件中的内容装入到存储器中，将存储器中的内容保存到文本文件中，对ROM、RAM（同步模式）、RAM（异步模式）进行读操作，对RAM（分离模式）进行读写操作。

参考设计文件：MIPS RAM 存储器验证实验.circ

二、MIPS RAM 存储器验证实验

- 1、MIPS RAM 存储器电路
- 2、简单的MIPS RAM 存储器测试电路
- 3、复杂的MIPS RAM 存储器测试电路

- 运行“**Logisim**”，在Logisim中打开“**MIPS RAM 存储器验证实验.circ**”。
- 也可以直接双击“**MIPS RAM 存储器验证实验.circ**”，打开该设计文件。



MIPS: Microprocessor without Interlocked Pipeline Stages, 没有互锁流水线的微处理器

MIPS RAM 存储器: 该存储器用于采用MIPS指令系统的CPU

• 1、MIPS RAM 存储器电路

- 主存通常是**按字节进行编址**；当存储器的数据宽度为32位时，主存可以按字节访问、按半字访问、按字访问：
 - 按**字节访问**时，主存的地址为：0、1、2、3、4、5、6、7、8、……；得到的数据为8位（字节）。
 - 按**半字访问**时，主存的地址为：0、2、4、6、8、……；得到的数据为16位（半字）。
 - 按**字访问**时，主存的地址为：0、4、8、……；得到的数据为32位（字）。

0	XX	XX	XX	XX
4	XX	XX	XX	XX
8	XX	XX	XX	XX
12	XX	XX	XX	XX
16	XX	XX	XX	XX
20	XX	XX	XX	XX

- Logisim的存储器数据宽度是固定的，因此需要增加有关辅助电路，才能构成一个**MIPS RAM存储器**（该存储器将用于采用MIPS指令系统的CPU），使其能满足按字节、半字、字进行访问；本实验的MIPS RAM存储器的容量为**4KB**。

- MIPS RAM存储器电路的输入、输出和控制信号：

- 输入：**

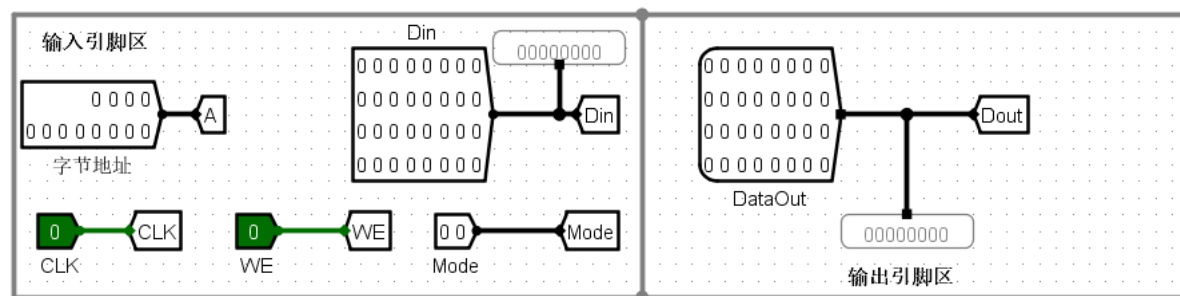
- A（字节地址）：主存的地址，12位（ $2^{12}=4K$ ），按字节进行编址；
- Din：写入数据，32位；如果是写入半字，则放在低16位；如果是写入字节，则放在最低的8位。

- 输出：**

- Dout：读出数据，32位；如果是读出半字，则位于低16位；如果是读出字节，则位于最低的8位。

- 控制信号（输入）：**

- CLK：时钟信号；
- WE：读写控制信号；WE=1，写入；WE=0，读出；
- Mode：访问模式，2位；Mode=01，按字节访问；Mode=10，按半字访问；Mode=00，按字访问；Mode=11，无效访问（存储器不工作）。



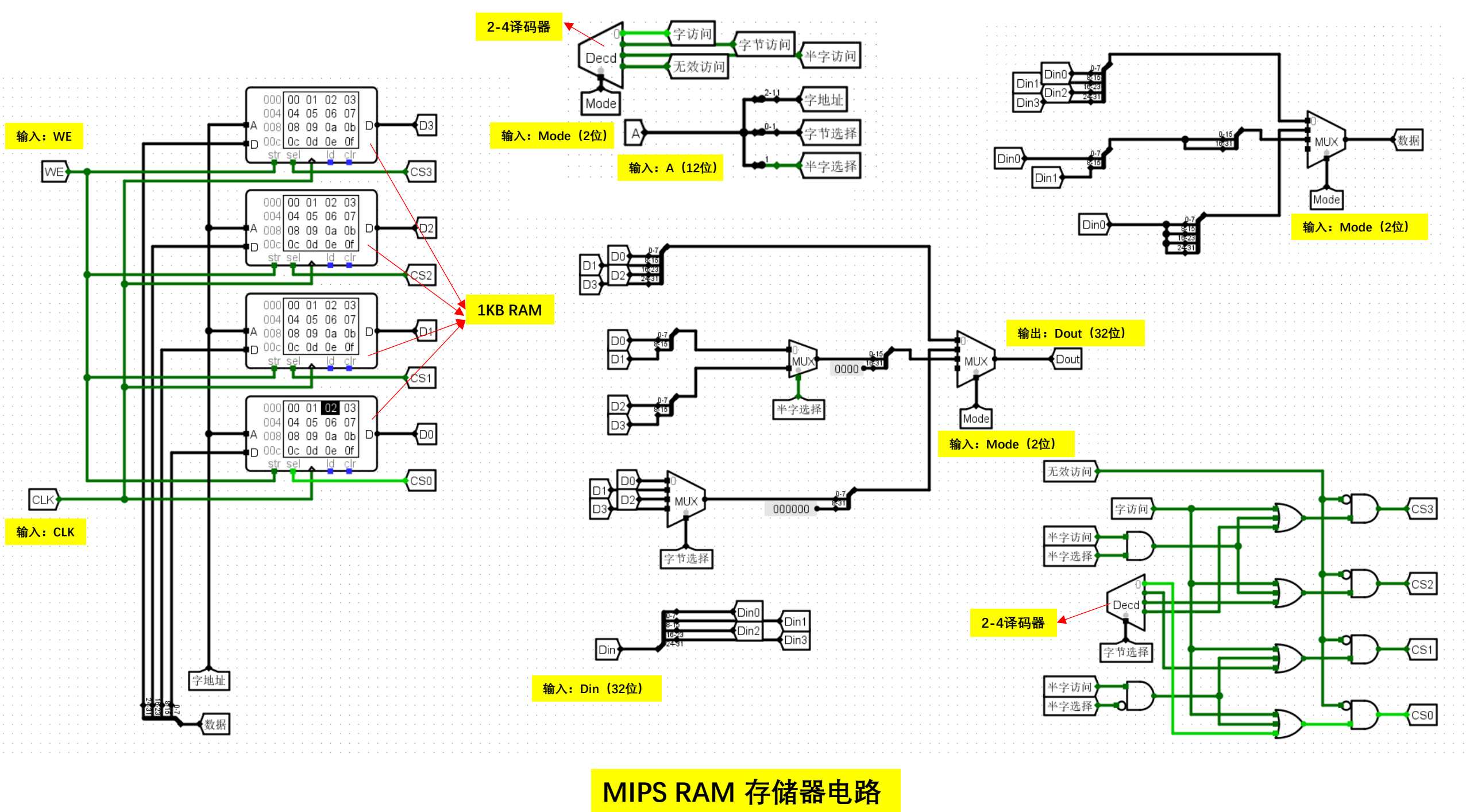
Mode=01，按字节访问

Mode=10，按半字访问

Mode=00，按字访问

Mode=11，无效访问（存储器不工作）

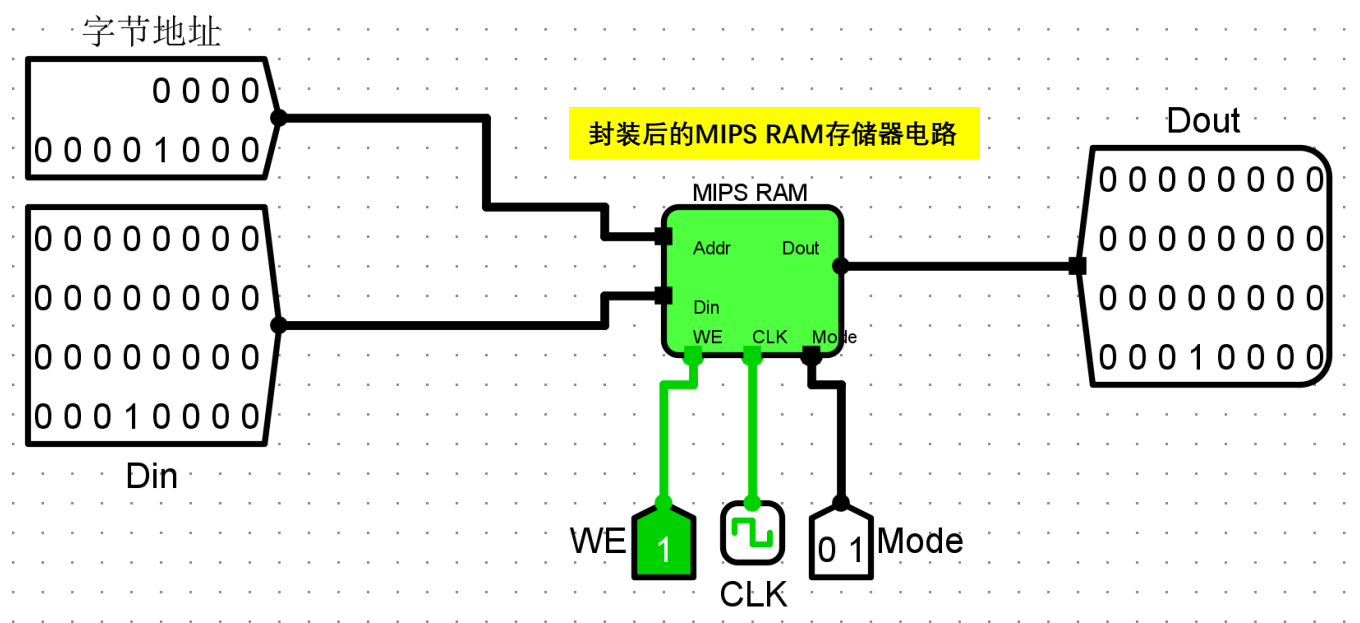
MIPS RAM存储器电路的输入、输出和控制信号



MIPS RAM 存储器电路

• 2、简单的 MIPS RAM 存储器测试电路

- 将前面的MIPS RAM 存储器电路封装后, 再添加一些输入、输出及控制信号, 就可以构成一个简单的 MIPS RAM 存储器测试电路:



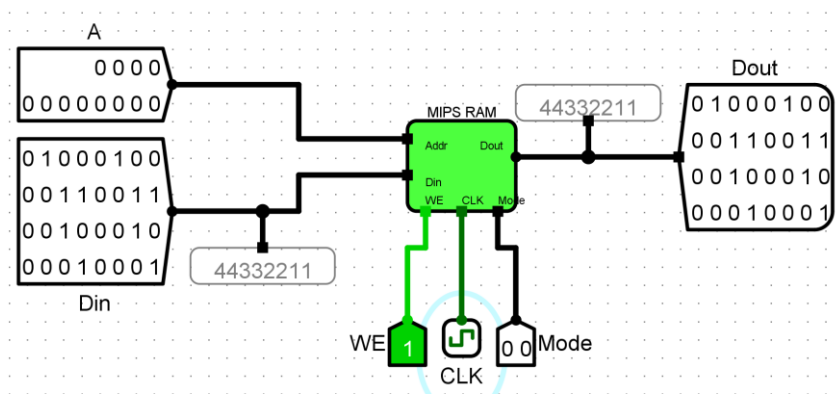
简单的MIPS RAM 存储器测试电路

- 测试步骤:

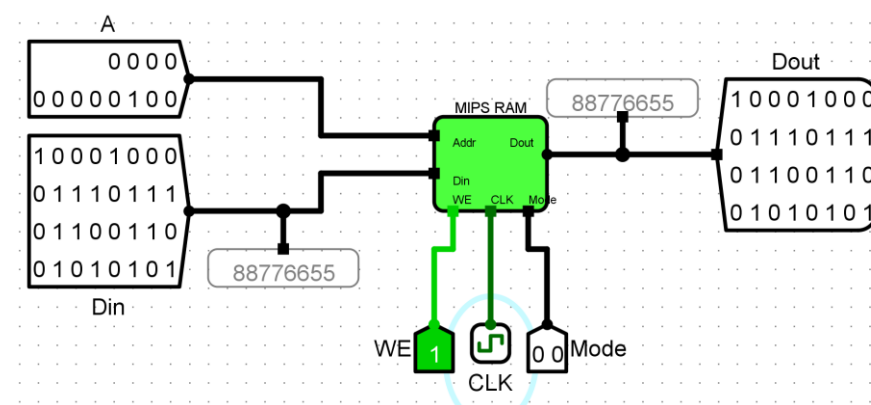
- 第1步: 将存储单元0~7, 分别写入11~88 (均为十六进制, 下同)。

- 置Mode=00, 按字访问; 置WE=1, 存储器写
- 置A=0、Din=44332211, 按CLK按钮2次, 则写入到0~3单元
- 置A=4、Din=88776655, 按CLK按钮2次, 则写入到4~7单元

	3	2	1	0
0	44	33	22	11
4	88	77	66	55
8	xx	xx	xx	xx
12	xx	xx	xx	xx
16	xx	xx	xx	xx
20	xx	xx	xx	xx



A=0, Din=44332211



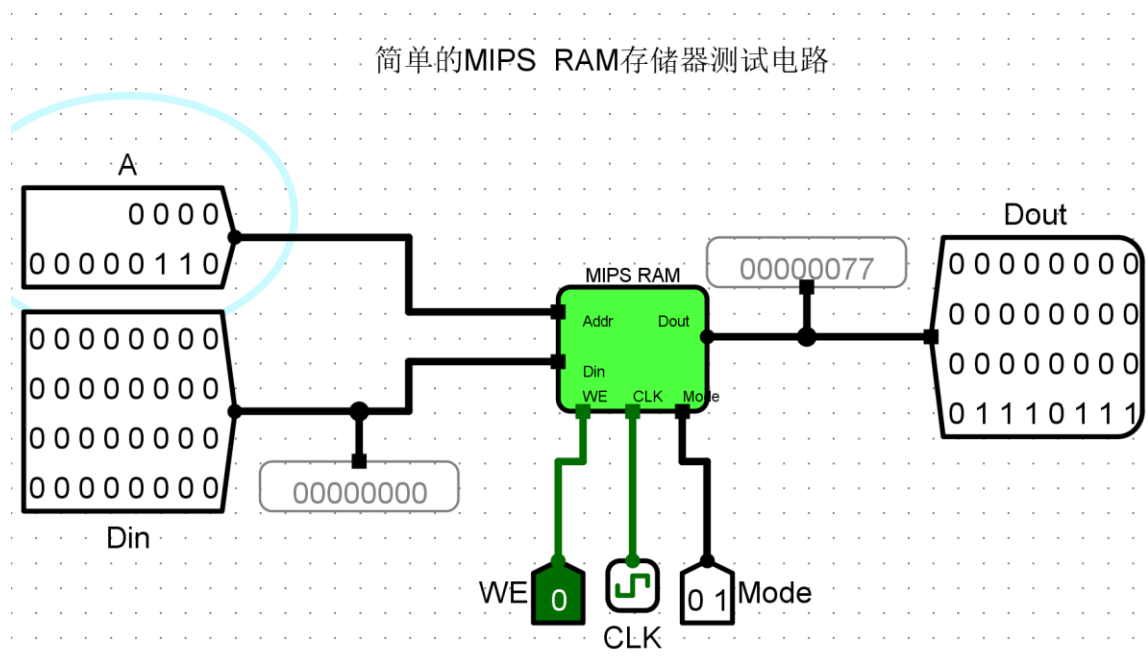
A=4, Din=88776655

- 第2步：按字节访问方式读。

- 置Mode=01，按**字节**访问；置WE=0，存储器**读**

- 置A=0，则Dout=00000011=11
- 置A=1，则Dout=00000022=22
- 置A=2，则Dout=00000033=33
- 置A=3，则Dout=00000044=44
- 置A=4，则Dout=00000055=55
- 置A=5，则Dout=00000066=66
- 置A=6，则Dout=00000077=77
- 置A=7，则Dout=00000088=88

0	44	33	22	11
4	88	77	66	55
8	xx	xx	xx	xx
12	xx	xx	xx	xx
16	xx	xx	xx	xx
20	xx	xx	xx	xx

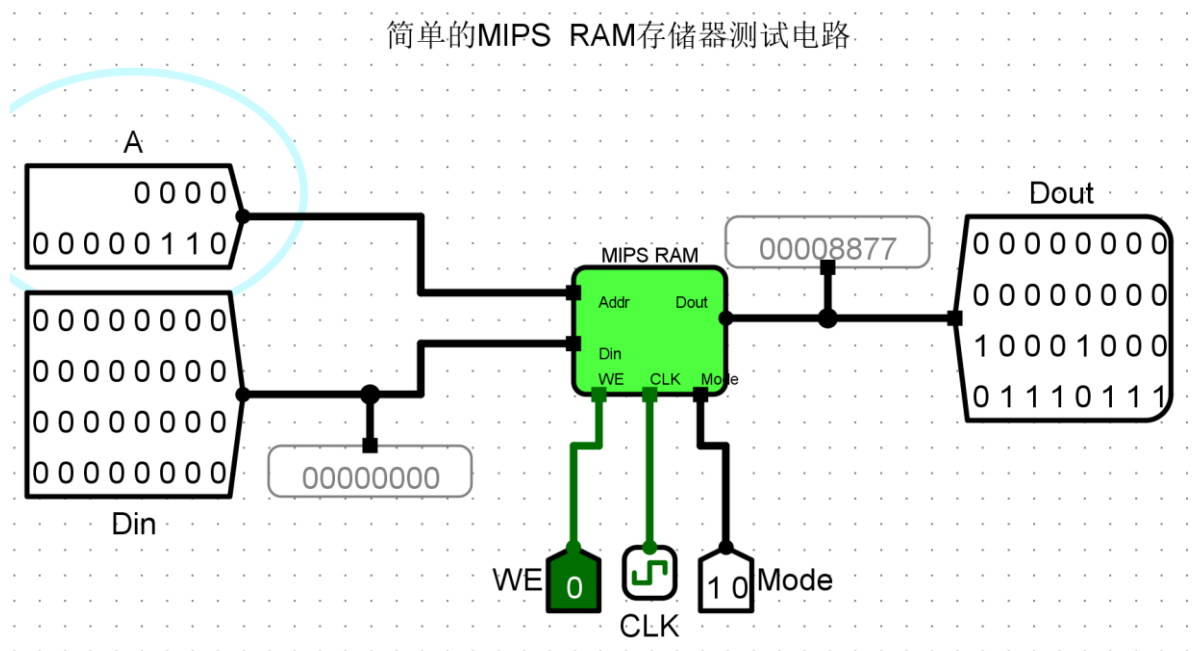


A=6, Dout=77

- 第3步：按半字访问方式读。

- 置Mode=10，按半字访问；置WE=0，存储器读
- 置A=0，则Dout=00002211=2211
- 置A=2，则Dout=00004433=4433
- 置A=4，则Dout=00006655=6655
- 置A=6，则Dout=00008877=8877

0	44	33	22	11
4	88	77	66	55
8	XX	XX	XX	XX
12	XX	XX	XX	XX
16	XX	XX	XX	XX
20	XX	XX	XX	XX

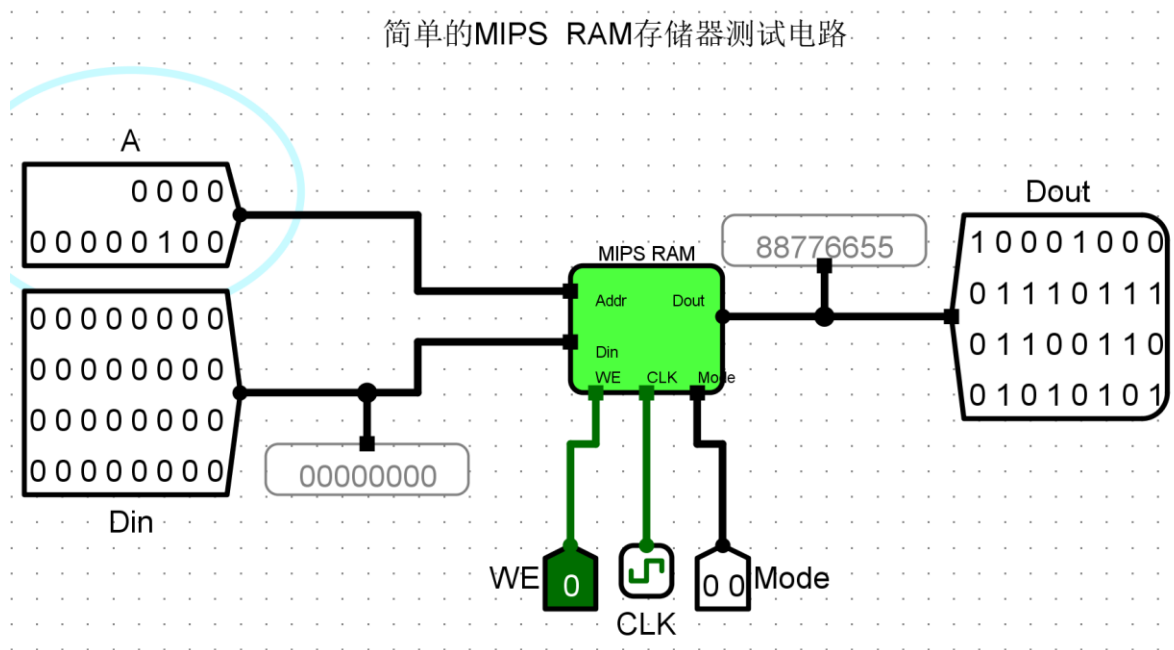


A=6, Dout=8877

- 第4步：按字访问方式读。

- 置Mode=00，按字访问；置WE=0，存储器读
- 置A=0，则Dout=44332211
- 置A=4，则Dout=88776655

0	44	33	22	11
4	88	77	66	55
8	xx	xx	xx	xx
12	xx	xx	xx	xx
16	xx	xx	xx	xx
20	xx	xx	xx	xx



A=4, Dout=88776655

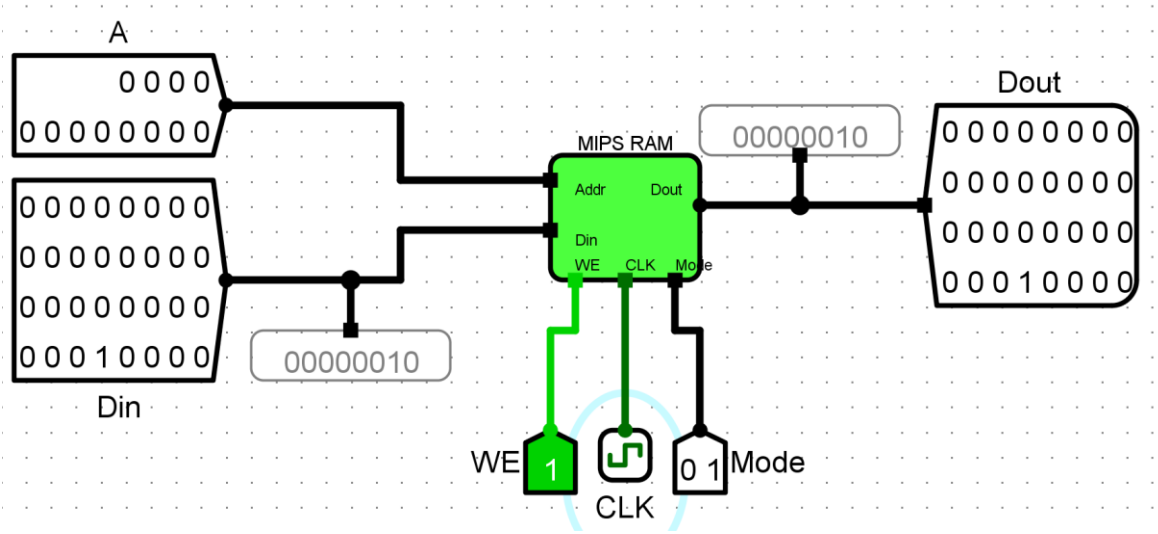
第5步：按字节访问方式写。

置Mode=01，按字节访问；置WE=1，存储器写

- 置A=0、Din=10；按CLK键2次，则将10写入0号单元
- 置A=1、Din=20；按CLK键2次，则将20写入1号单元
- 置A=2、Din=30；按CLK键2次，则将30写入2号单元
- 置A=3、Din=40；按CLK键2次，则将40写入3号单元
- 置A=4、Din=50；按CLK键2次，则将50写入4号单元
- 置A=5、Din=60；按CLK键2次，则将60写入5号单元
- 置A=6、Din=70；按CLK键2次，则将70写入6号单元
- 置A=7、Din=80；按CLK键2次，则将80写入7号单元

0	44	33	22	11
4	88	77	66	55
8	xx	xx	xx	xx
12	xx	xx	xx	xx
16	xx	xx	xx	xx
20	xx	xx	xx	xx

0	40	30	20	10
4	80	70	60	50
8	xx	xx	xx	xx
12	xx	xx	xx	xx
16	xx	xx	xx	xx
20	xx	xx	xx	xx



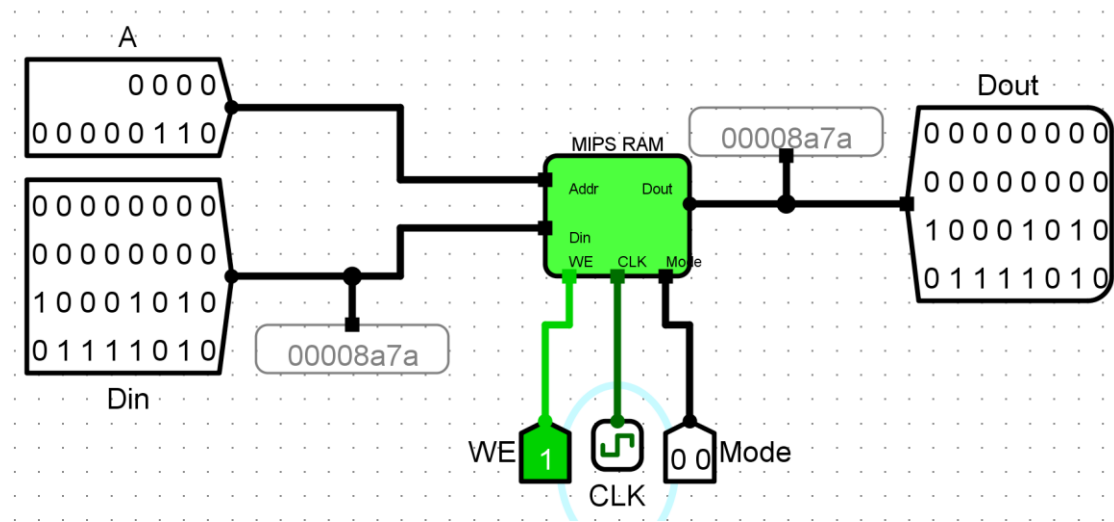
A=0, Din=10

- 第6步：按半字访问方式写。

- 置Mode=10，按半字访问；置WE=1，存储器写
- 置A=0、Din=2a1a；按CLK键2次，则将2a1a写入0-1号单元
- 置A=2、Din=4a3a；按CLK键2次，则将4a3a写入2-3号单元
- 置A=4、Din=6a5a；按CLK键2次，则将6a5a写入4-5号单元
- 置A=6、Din=8a7a；按CLK键2次，则将8a7a写入6-7号单元

0	40	30	20	10
4	80	70	60	50
8	xx	xx	xx	xx
12	xx	xx	xx	xx
16	xx	xx	xx	xx
20	xx	xx	xx	xx

0	4a	3a	2a	1a
4	8a	7a	6a	5a
8	xx	xx	xx	xx
12	xx	xx	xx	xx
16	xx	xx	xx	xx
20	xx	xx	xx	xx



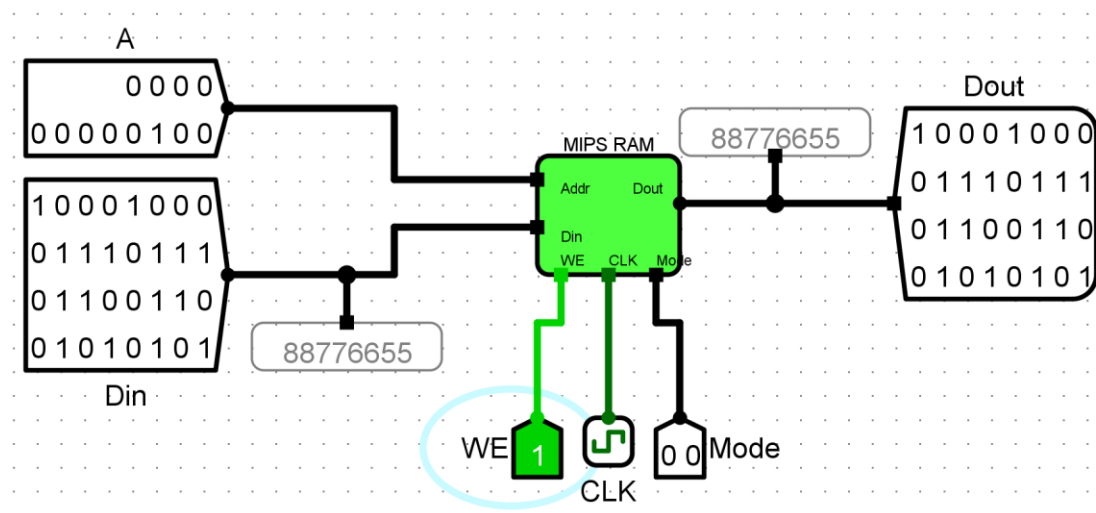
A=6, Din=8a7a

- 第7步：按字访问方式写。

- 置Mode=00，按字访问；置WE=1，存储器写
- 置A=0、Din=44332211；按CLK键2次，则将44332211写入0-3号单元
- 置A=4、Din=88776655；按CLK键2次，则将88776655写入4-7号单元

0	4a	3a	2a	1a
4	8a	7a	6a	5a
8	xx	xx	xx	xx
12	xx	xx	xx	xx
16	xx	xx	xx	xx
20	xx	xx	xx	xx

0	44	33	22	11
4	88	77	66	55
8	xx	xx	xx	xx
12	xx	xx	xx	xx
16	xx	xx	xx	xx
20	xx	xx	xx	xx

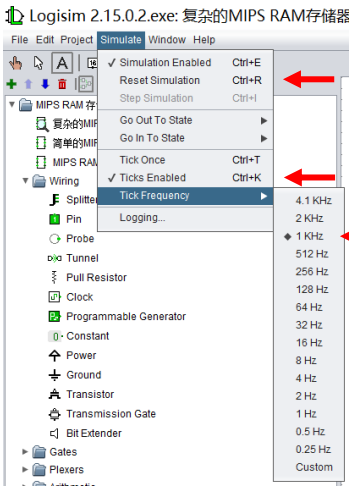


A=4, Din=88776655

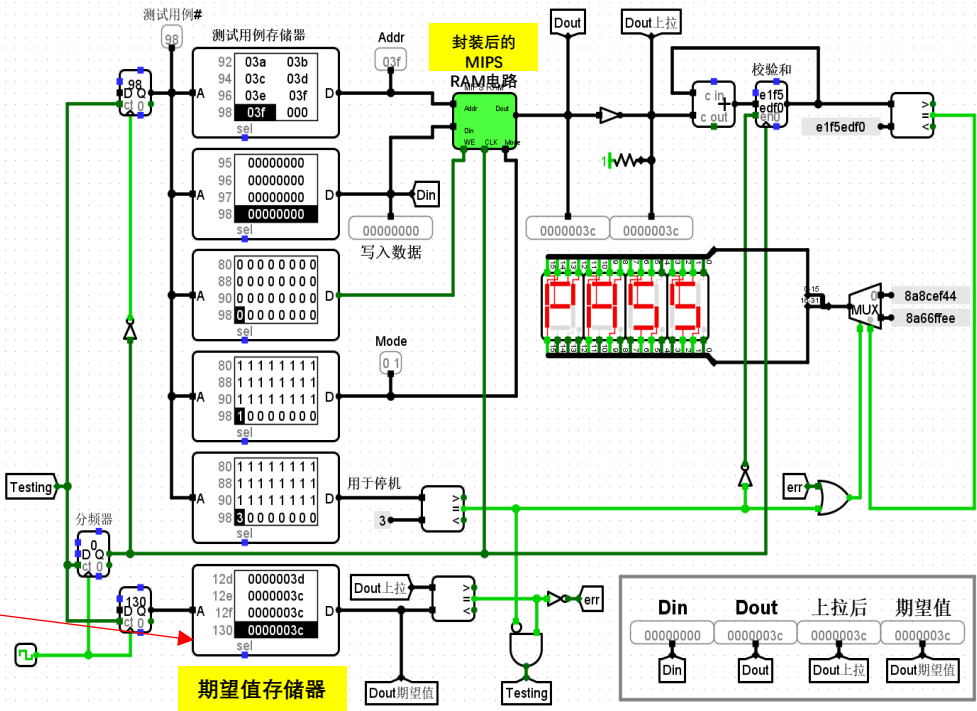
3、复杂的 MIPS RAM 存储器测试电路

测试步骤：

- ① 设置时钟频率=1KHz。
- ② 按Ctrl+R，使电路复位。
- ③ 按Ctrl+K，启动时钟。
- ④ 如果显示“PASS”，表示测试通过。
- ⑤ 如果显示“FAIL”，表示测试失败。
- ⑥ 再按Ctrl+K，则停止时钟。
- ⑦ 修改“期望值”存储器的内容。
- ⑧ 再按Ctrl+K，启动时钟，运行结束后，将显示“FAIL”。

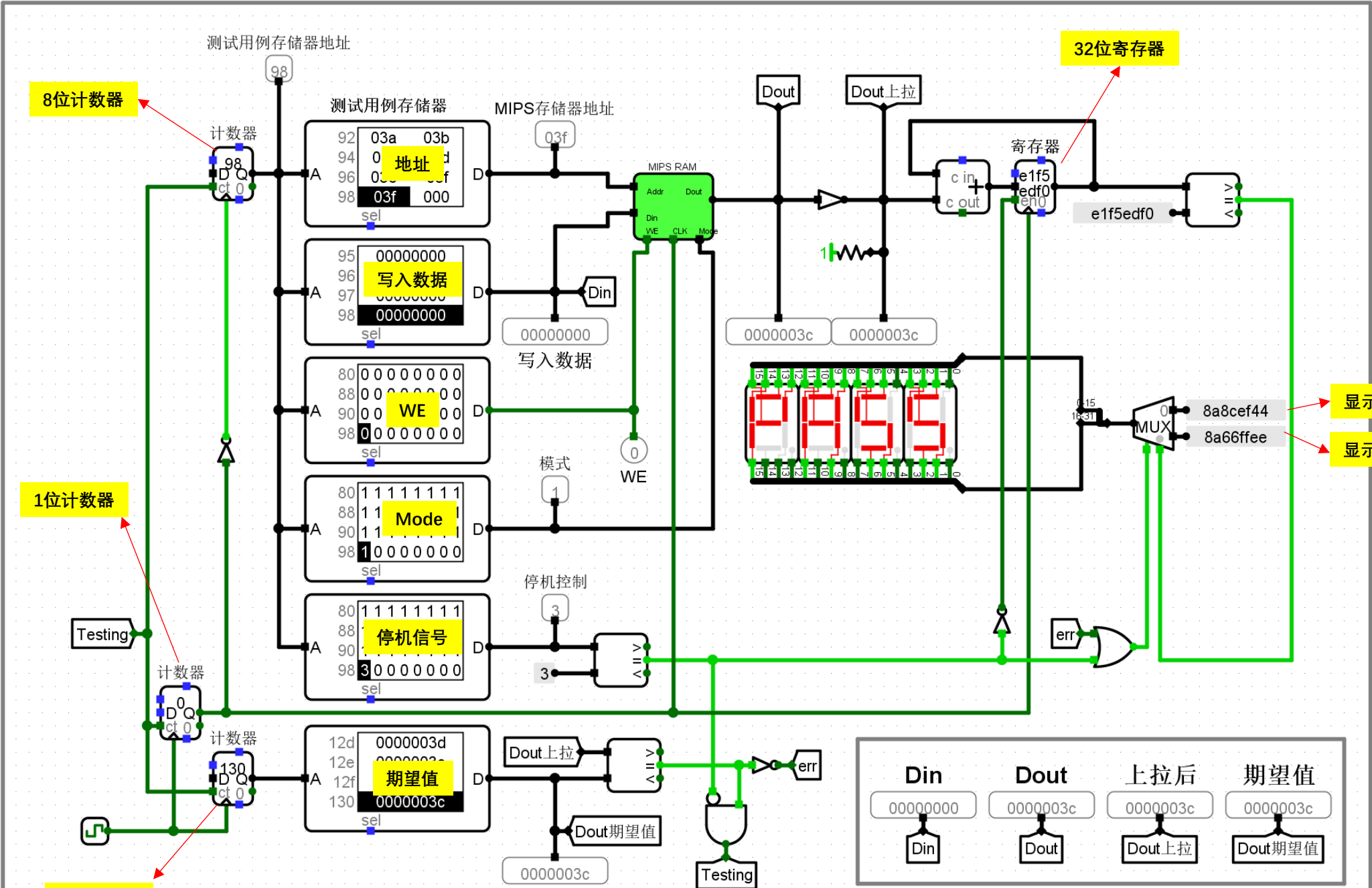


电路功能：测试MIPS RAM存储器，ctrl+k启动时钟后，会自动对完成的MIPS RAM存储器进行批量读写测试，并将最终写入的数据逐一读出进行校验和计算
电路状态稳定后测试正确显示PASS，错误显示FAIL，并且测试系统会停止在出错的时刻，记录测试用例编号，方便故障重现
写入测试时，会将写入前RAM数据和写入后的RAM数据分别和期望值进行比较，如果不等表示数据错误，读出时数据应该和期望一致，否则表示出错。



出错时，系统自动停机，观察上面三个值看看哪里不一样
重新复位，自动运行时钟到出错用例附近单步，观察问题
在错误用例前一拍观察Din数据，再观察写入数据前后读出的数据

复杂的MIPS RAM存储器测试电路



复杂的MIPS RAM存储器测试电路

MIPS RAM存储器测试用例——Mode.txt
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
v2.0 raw
16*0 8*2 16*1 16*0 32*2 65*1

共153个模式：

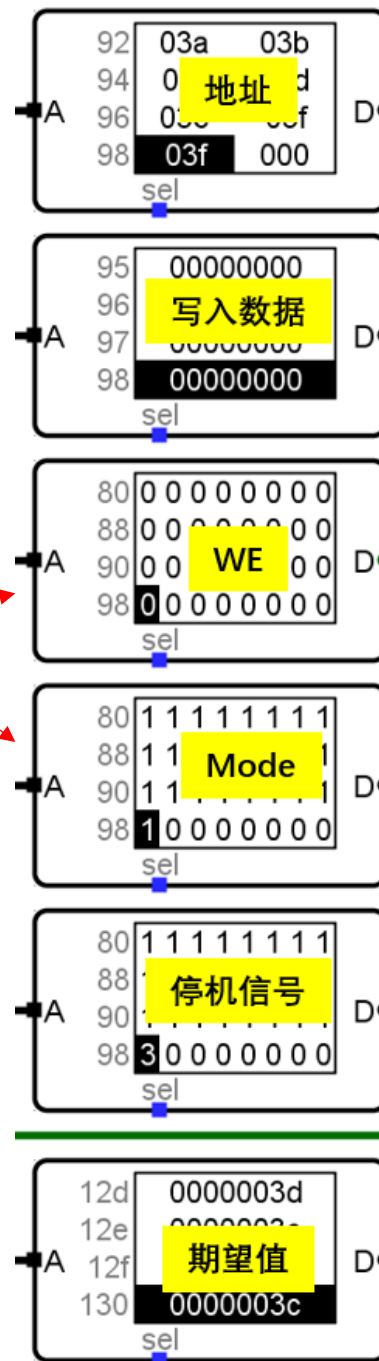
- 1、前40次（16+8+16）为写入操作
- 2、后113次（16+32+65）为读出、停机操作
- 3、0表示按字访问，1表示按字节访问，2表示按半字访问
- 4、Mode=01，按字节访问；Mode=10，按半字访问；Mode=00，按字访问

MIPS RAM存储器测试用例——WE.txt -
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
v2.0 raw
40*1

前40次为写入操作

6个存储器中的内容

测试用例存储器



上述文本文件在“MIPS RAM 存储器测试用例.zip”中

6个存储器中的内容

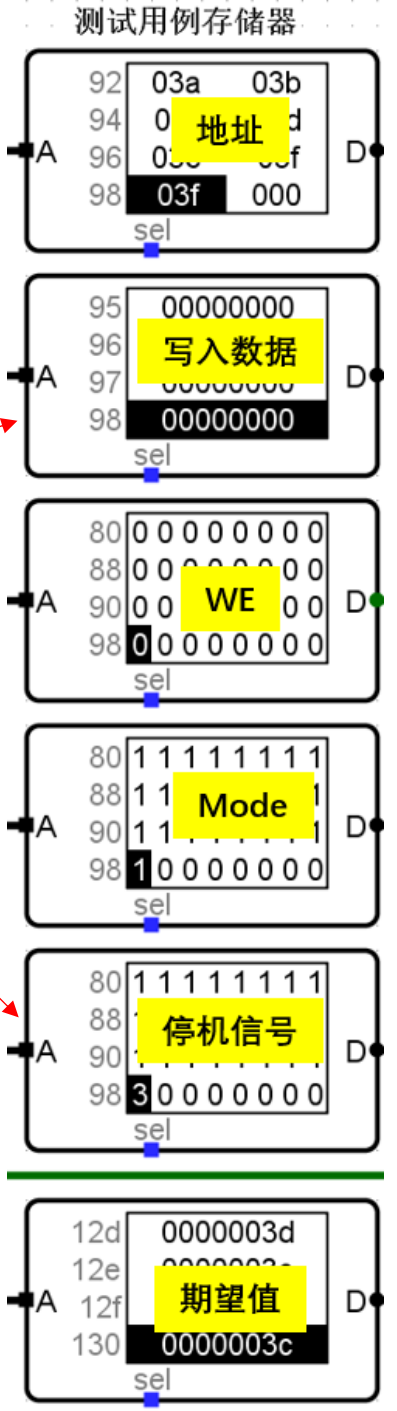
MIPS RAM存储器测试用例——停机信号.txt
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
v2.0 raw
16*0 8*2 16*1 16*0 32*2 64*1 3

- 共153次操作：
- 1、前40次（16+8+16）为写入操作（非3）
 - 2、后112次（16+32+64）为读出操作（非3）
 - 3、最后1次为停机（3）

MIPS RAM存储器测试用例——写入数据.txt - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
v2.0 raw
203 607 a0b e0f 4*0
20212223 24252627 28292a2b 2c2d2e2f 30310000 34350000 38390000 3c3d0000
1 405 809 c0d 3233 3637 3a3b 3e3f
10 14 18 1c 11 15 19 1d
12 16 1a 1e 13 17 1b 1f

前40次写入的数据

上述文本文件在“MIPS RAM 存储器测试用例.zip”中



v2.0 raw

```

0 4 8 c 10 14 18 1c
20 24 28 2c 30 34 38 3c
2 6 a e 31 35 39 3d
13 17 1b 1f 12 16 1a 1e
11 15 19 1d 10 14 18 1c

```

```

0 4 8 c 10 14 18 1c
20 24 28 2c 30 34 38 3c
0 2 4 6 8 a c e
10 12 14 16 18 1a 1c 1e
20 22 24 26 28 2a 2c 2e
30 32 34 36 38 3a 3c 3e
0 1 2 3 4 5 6 7
8 9 a b c d e f
10 11 12 13 14 15 16 17
18 19 1a 1b 1c 1d 1e 1f
20 21 22 23 24 25 26 27
28 29 2a 2b 2c 2d 2e 2f
30 31 32 33 34 35 36 37
38 39 3a 3b 3c 3d 3e 3f

```

3f

6个存储器中的内容

共153个地址：

1、前40次为写入操作地址：

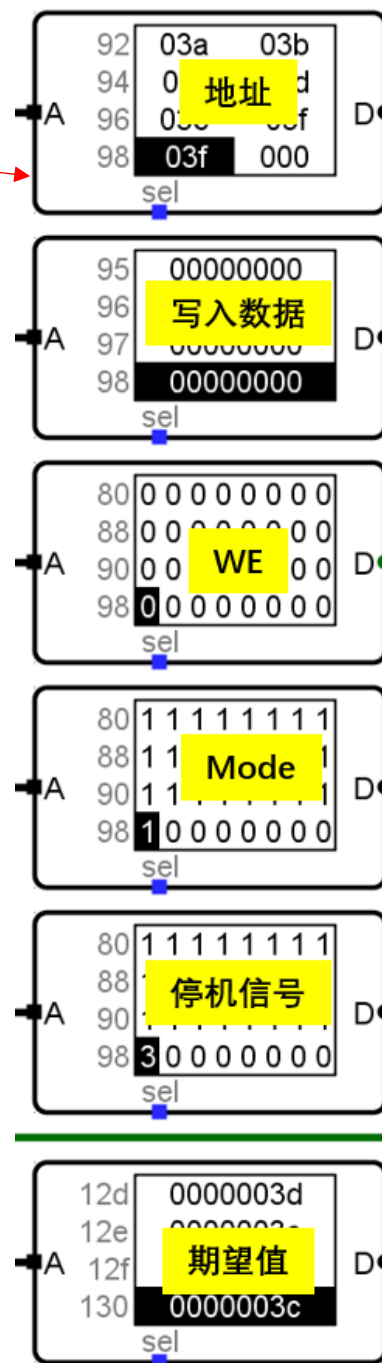
16次按字访问
8次按半字访问
16次按字节访问

2、后112次为读出操作地址：

16次按字访问
32次按半字访问
64次按字节访问

3、最后1次为停机 (3f)

测试用例存储器



上述文本文件在“MIPS RAM 存储器测试用例.zip”中

6个存储器中的内容

v2.0 raw

0 203 0 607 0 a0b 0 e0f

9*0 20212223 0 24252627 0 28292a2b 0 2c2d2e2f

0 30310000 0 34350000 0 38390000 0 3c3d0000

0 1 0 405 0 809 0 c0d

0 3233 0 3637 0 3a3b 0 3e3f

0 10 0 14 0 18 0 1c

0 11 0 15 0 19 0 1d

0 12 0 16 0 1a 0 1e

0 13 0 17 0 1b 0 1f

10203 10203 4050607 4050607 8090a0b 8090a0b c0d0e0f c0d0e0f

10111213 10111213 14151617 14151617 18191a1b 18191a1b 1c1d1e1f 1c1d1e1f

20212223 20212223 24252627 24252627 28292a2b 28292a2b 2c2d2e2f 2c2d2e2f

30313233 30313233 34353637 34353637 38393a3b 38393a3b 3c3d3e3f 3c3d3e3f

203 203 1 1 607 607 405 405

a0b a0b 809 809 e0f e0f c0d c0d

1213 1213 1011 1011 1617 1617 1415 1415

1a1b 1a1b 1819 1819 1e1f 1e1f 1c1d 1c1d

2223 2223 2021 2021 2627 2627 2425 2425

2a2b 2a2b 2829 2829 2e2f 2e2f 2c2d 2c2d

3233 3233 3031 3031 3637 3637 3435 3435

3a3b 3a3b 3839 3839 3e3f 3e3f 3c3d 3c3d

3 3 2 2 1 1 0 0

7 7 6 6 5 5 4 4

b b a a 9 9 8 8

f f e e d d c c

13 13 12 12 11 11 10 10

17 17 16 16 15 15 14 14

1b 1b 1a 1a 19 19 18 18

1f 1f 1e 1e 1d 1d 1c 1c

23 23 22 22 21 21 20 20

27 27 26 26 25 25 24 24

2b 2b 2a 2a 29 29 28 28

2f 2f 2e 2e 2d 2d 2c 2c

33 33 32 32 31 31 30 30

37 37 36 36 35 35 34 34

3b 3b 3a 3a 39 39 38 38

3f 3f 3e 3e 3d 3d 3c 3c

3c

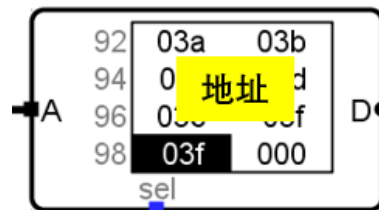
共305个数据：

1、前80个数据，对应40个写入的数据

2、后224个数据，对应112个读出的数据；
这224个数据之和，除以2，即为：
1e1f5edf0，去掉最高位，即为：
e1f5edf0

3、最后1个为停机（3c）

测试用例存储器



• 什么时候会显示“FAIL”?

- (1) MIPS RAM电路有问题。
- (2) 修改写入数据存储器的内容。
- (3) 修改期望值存储器的内容。

MIPS RAM存储器测试用例——写入数据.txt - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw

203 607 a0b e0f 4*0

20212223 24252627 28292a2b 2c2d2e2f 30310000 34350000 38390000 3c3d0000

1 405 809 c0d 3233 3637 3a3b 3e3f

10 14 18 1c 11 15 19 1d

12 16 1a 1e 13 17 1b 1f

“1”改为“2”

MIPS RAM存储器测试用例——修改后的写入数据.txt - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw

203 607 a0b e0f 4*0

20212223 24252627 28292a2b 2c2d2e2f 30310000 34350000 38390000 3c3d0000

2 405 809 c0d 3233 3637 3a3b 3e3f

10 14 18 1c 11 15 19 1d

12 16 1a 1e 13 17 1b 1f

MIPS RAM存储器测试用例——期望值.txt - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw

0 203 0 607 0 a0b 0 e0f

9*0 20212223 0 24252627 0 28292a2b 0 2c2d2e2f

0 30310000 0 34350000 0 38390000 0 3c3d0000

0 1 0 405 0 809 0 c0d

0 3233 0 3637 0 3a3b 0 3e3f

0 10 0 14 0 18 0 1c

0 11 0 15 0 19 0 1d

0 12 0 16 0 1a 0 1e

0 13 0 17 0 1b 0 1f

10203 10203 4050607 4050607 8090a0b 8090a0b c0d0e0f c0d0e0f

10111213 10111213 14151617 14151617 18191a1b 18191a1b 1c1d1e1f 1c1d1e1f

20212223 20212223 24252627 24252627 28292a2b 28292a2b 2c2d2e2f 2c2d2e2f

30313233 30313233 34353637 34353637 38393a3b 38393a3b 3c3d3e3f 3c3d3e3f

203 203 1 1 607 607 405 405

a0b a0b 809 809 e0f e0f c0d c0d

1213 1213 1011 1011 1617 1617 1415 1415

1a1b 1a1b 1819 1819 1e1f 1e1f 1c1d 1c1d

2223 2223 2021 2021 2627 2627 2425 2425

2a2b 2a2b 2829 2829 2e2f 2e2f 2c2d 2c2d

3233 3233 3031 3031 3637 3637 3435 3435

3a3b 3a3b 3839 3839 3e3f 3e3f 3c3d 3c3d

3 3 2 2 1 1 0 0

7 7 6 6 5 5 4 4

b b a a 9 9 8 8

f f e e d d c c

13 13 12 12 11 11 10 10

17 17 16 16 15 15 14 14

1b 1b 1a 1a 19 19 18 18

1f 1f 1e 1e 1d 1d 1c 1c

23 23 22 22 21 21 20 20

27 27 26 26 25 25 24 24

2b 2b 2a 2a 29 29 28 28

2f 2f 2e 2e 2d 2d 2c 2c

33 33 32 32 31 31 30 30

37 37 36 36 35 35 34 34

3b 3b 3a 3a 39 39 38 38

3f 3f 3e 3e 3d 3d 3c 3c

“3”改为“4”

MIPS RAM存储器测试用例——修改后的期望值.txt - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw

0 203 0 607 0 a0b 0 e0f

9*0 20212223 0 24252627 0 28292a2b 0 2c2d2e2f

0 30310000 0 34350000 0 38390000 0 3c3d0000

0 1 0 405 0 809 0 c0d

0 3233 0 3637 0 3a3b 0 3e3f

0 10 0 14 0 18 0 1c

0 11 0 15 0 19 0 1d

0 12 0 16 0 1a 0 1e

0 13 0 17 0 1b 0 1f

10203 10203 4050607 4050607 8090a0b 8090a0b c0d0e0f c0d0e0f

10111213 10111213 14151617 14151617 18191a1b 18191a1b 1c1d1e1f 1c1d1e1f

20212223 20212223 24252627 24252627 28292a2b 28292a2b 2c2d2e2f 2c2d2e2f

30313233 30313233 34353637 34353637 38393a3b 38393a3b 3c3d3e3f 3c3d3e3f

203 203 1 1 607 607 405 405

a0b a0b 809 809 e0f e0f c0d c0d

1213 1213 1011 1011 1617 1617 1415 1415

1a1b 1a1b 1819 1819 1e1f 1e1f 1c1d 1c1d

2223 2223 2021 2021 2627 2627 2425 2425

2a2b 2a2b 2829 2829 2e2f 2e2f 2c2d 2c2d

3233 3233 3031 3031 3637 3637 3435 3435

3a3b 3a3b 3839 3839 3e3f 3e3f 3c3d 3c3d

4 3 2 2 1 1 0 0

7 7 6 6 5 5 4 4

b b a a 9 9 8 8

f f e e d d c c

13 13 12 12 11 11 10 10

17 17 16 16 15 15 14 14

1b 1b 1a 1a 19 19 18 18

1f 1f 1e 1e 1d 1d 1c 1c

23 23 22 22 21 21 20 20

27 27 26 26 25 25 24 24

2b 2b 2a 2a 29 29 28 28

2f 2f 2e 2e 2d 2d 2c 2c

33 33 32 32 31 31 30 30

37 37 36 36 35 35 34 34

3b 3b 3a 3a 39 39 38 38

3f 3f 3e 3e 3d 3d 3c 3c

3c

上述文本文件在“MIPS RAM 存储器测试用例.zip”中

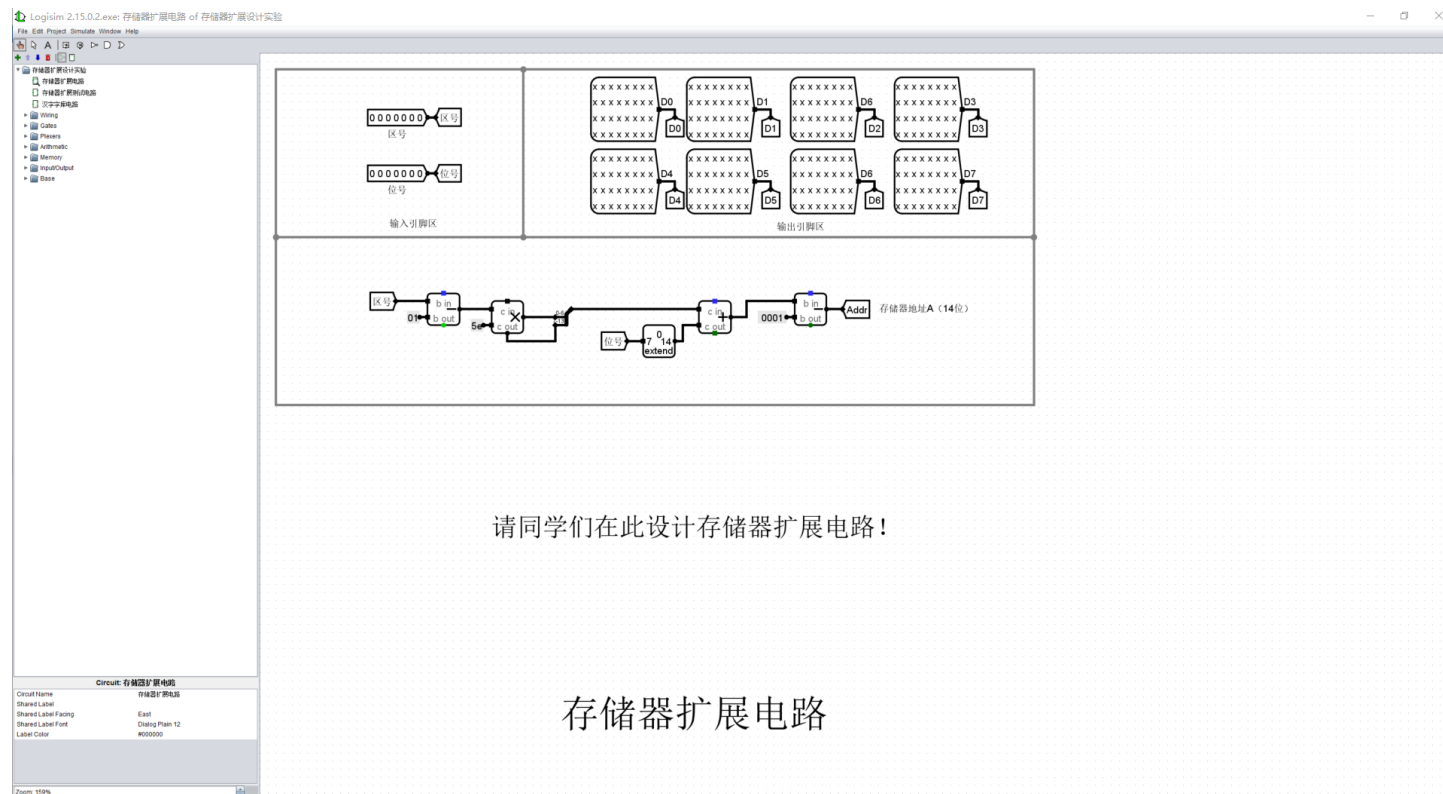
- 请同学们分析“**MIPS RAM 存储器电路**”的原理，指出该电路是如何实现：
 - 存储器读：
 - 按字节访问
 - 按半字访问
 - 按字访问
 - 存储器写：
 - 按字节访问
 - 按半字访问
 - 按字访问
- 有兴趣的同学，请分析“**复杂的MIPS RAM存储器测试电路**”的原理！

参考设计文件：存储器扩展设计实验.circ

三、存储器扩展设计实验

- 1、存储器扩展电路（设计实验）
- 2、存储器扩展测试电路

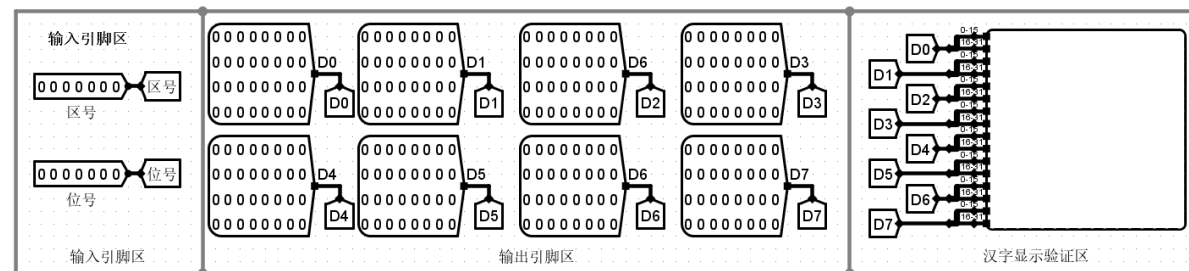
- 运行“**Logisim**”，在Logisim中打开“**存储器扩展设计实验.circ**”。
- 也可以直接双击“**存储器扩展设计实验.circ**”，打开该设计文件。

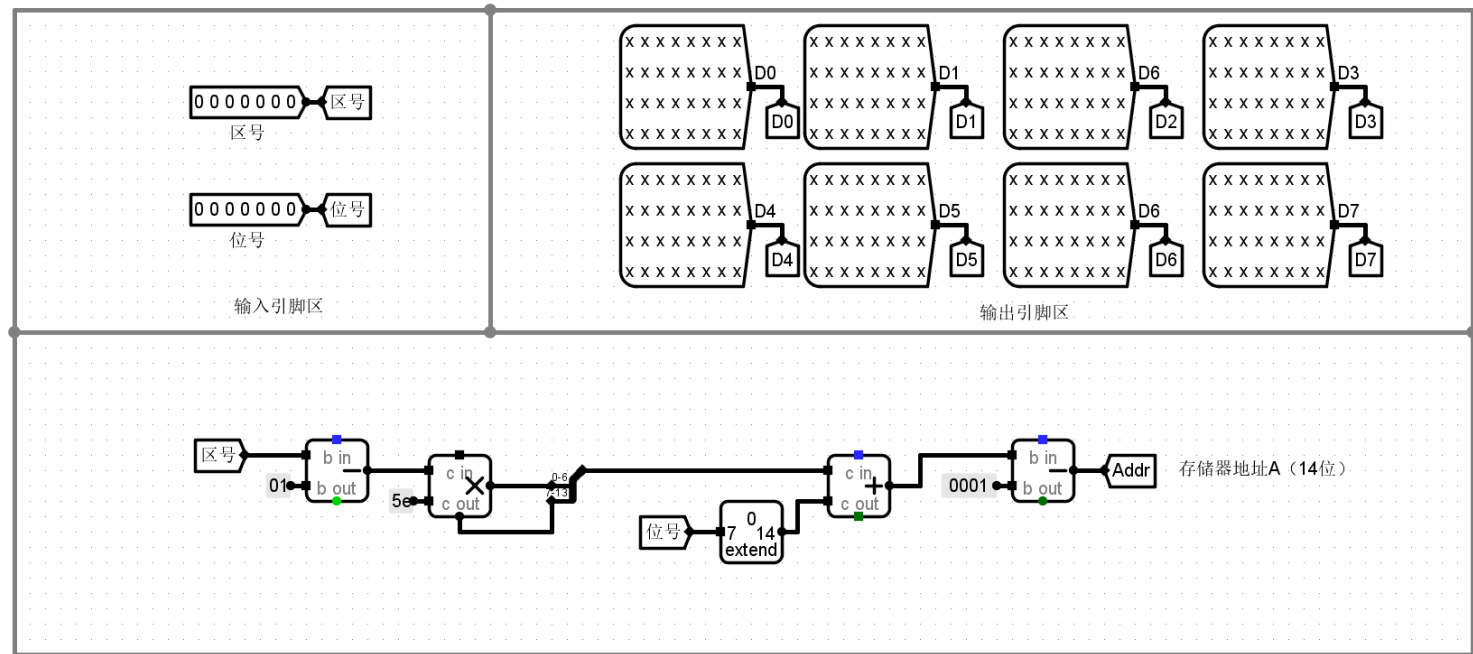


• 1、存储器扩展电路（设计实验）

- 汉字的区位码为**14位**（区号7位、位号7位）；采用16x16点阵显示汉字，即显示一个汉字需要**256位**（ $16 \times 16 = 256$ ）二进制数；因此可以用一个地址长度为14位、数据宽度为256位的ROM存储器存放汉字的点阵码，即ROM存储器的容量为 **$2^{14} \times 256$ 位**=**16Kx256位**。
- 但是Logisim中存储器组件的数据宽度最大值为32位，因此需要进行存储器的扩展。
- 现要求使用4片**4Kx32位**ROM、7片**16Kx32位**ROM，扩展成容量为**16Kx256位**的ROM存储器，请同学们设计该存储器扩展电路。
- 存储器扩展电路的输入和输出如下：
 - **输入**：汉字的区号和位号，经过计算后得到存储器的地址A（14位）。
 - **输出**：汉字的16x16点阵码，即存储器的数据D（256位）。

存储器扩展电路的输入和输出





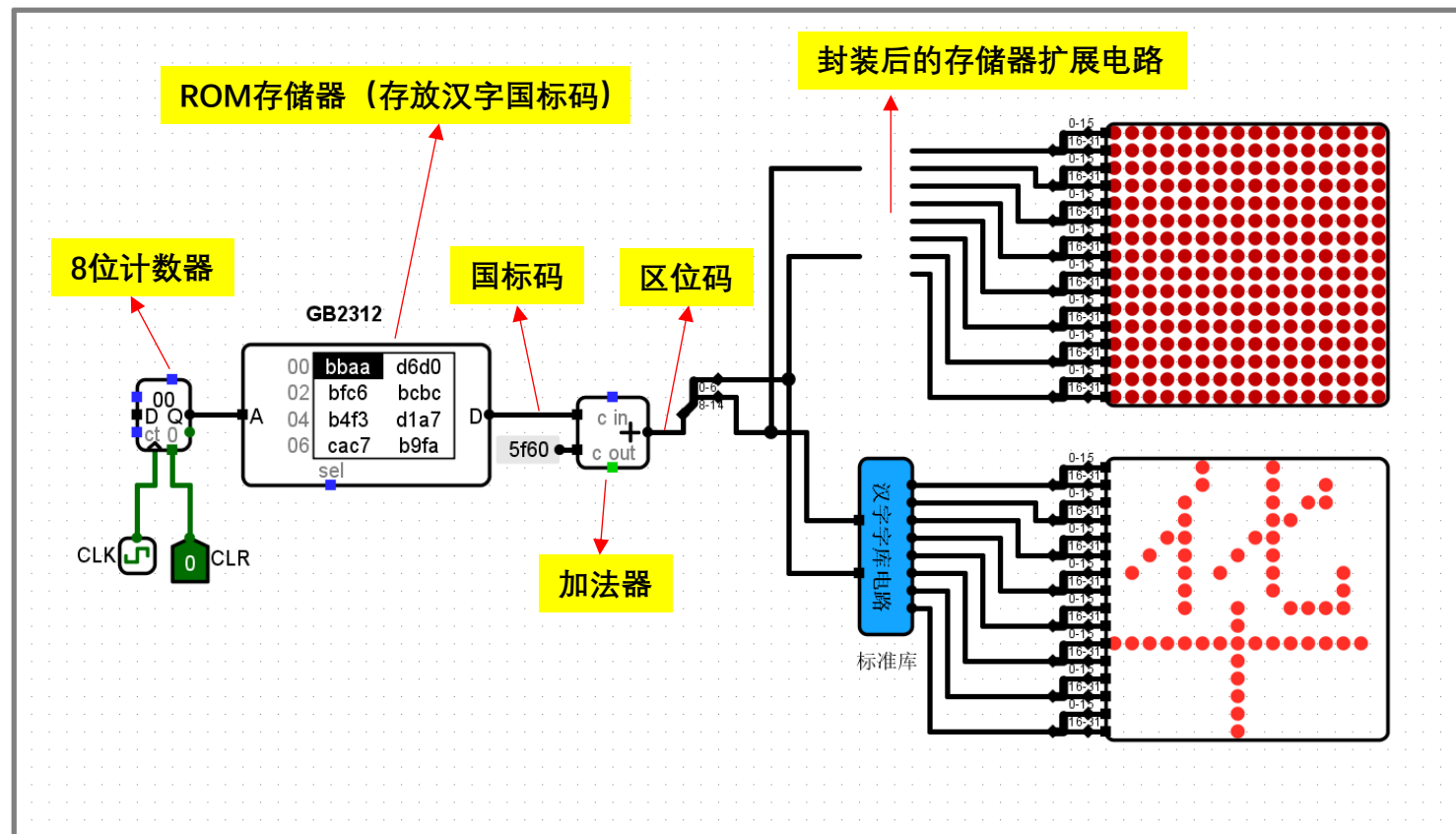
请同学们在此设计存储器扩展电路！

存储器扩展电路

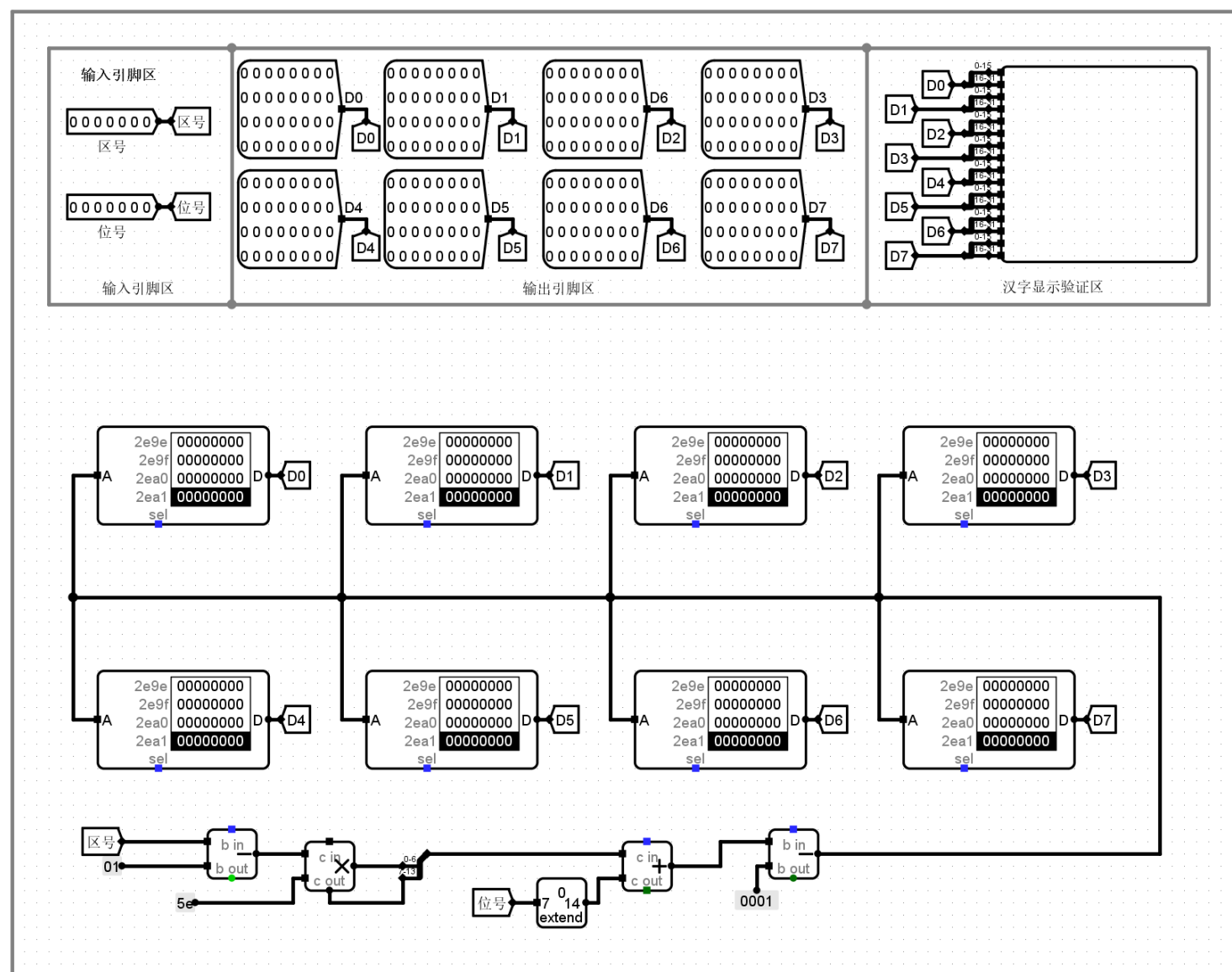
- 存储器扩展电路**设计思路**：

- 现有4片4Kx32位ROM存储器、7片16Kx32位ROM存储器；
- 要求扩展成容量为16Kx256位的ROM存储器；
- 因此需要进行**字、位扩展**；
- 先将4片4Kx32位ROM存储器，通过**字扩展**方式（需要1个**2-4译码器**），扩展成16Kx32位的ROM；
- 再将7片16Kx32位ROM存储器，以及上述字扩展后的16Kx32位的ROM，通过**位扩展**方式，得到16Kx256位的ROM存储器；
- 存储器的地址A（14位）为区位码经过相关的电路计算后的值（该电路已经有）；
- 存储器的输出（128位）分为8个32位：D0、**D1**、D2、D3、D4、D5、D6、D7，其中**D1**为经过字扩展后的16Kx32位的ROM的输出。

- 2、存储器扩展测试电路



存储器扩展测试电路

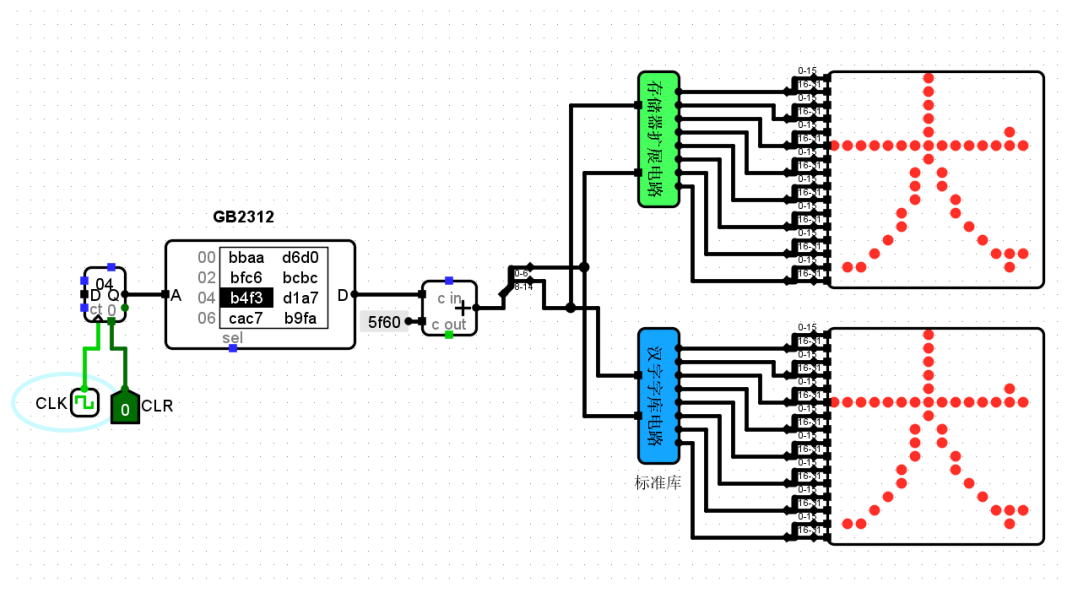


汉字字库电路

- **测试步骤:**

- ① 将设计好的存储器扩展电路进行封装，然后放置到存储器扩展测试电路中。
- ② 将“D0存储器的内容.txt”、“D2存储器的内容.txt”、“D3存储器的内容.txt”、“D4存储器的内容.txt”、“D5存储器的内容.txt”、“D6存储器的内容.txt”、“D7存储器的内容.txt”中的内容分别装入到设计好的存储器扩展电路的7个16Kx32位ROM存储器中。
- ③ 将“D1-0存储器的内容.txt”、“D1-1存储器的内容.txt”、“D1-2存储器的内容.txt”、“D1-3存储器的内容.txt”中的内容分别装入到设计好的存储器扩展电路的4个4Kx32位ROM存储器中。
- ④ 点击存储器扩展测试电路的“CLR”，使其为“1”；再点击，使其为“0”。
- ⑤ 点击存储器扩展测试电路的“CLK”按钮2次，此时右边的LED点阵将显示汉字。
- ⑥ 如果上下两个LED点阵显示的汉字是相同的，则表示存储器扩展电路是正确的；否则，要么是存储器扩展电路设计不正确，要么是第②、③步的汉字点阵码没有装入正确。

上述文本文件在“存储器扩展电路存储器中的内容.zip”中

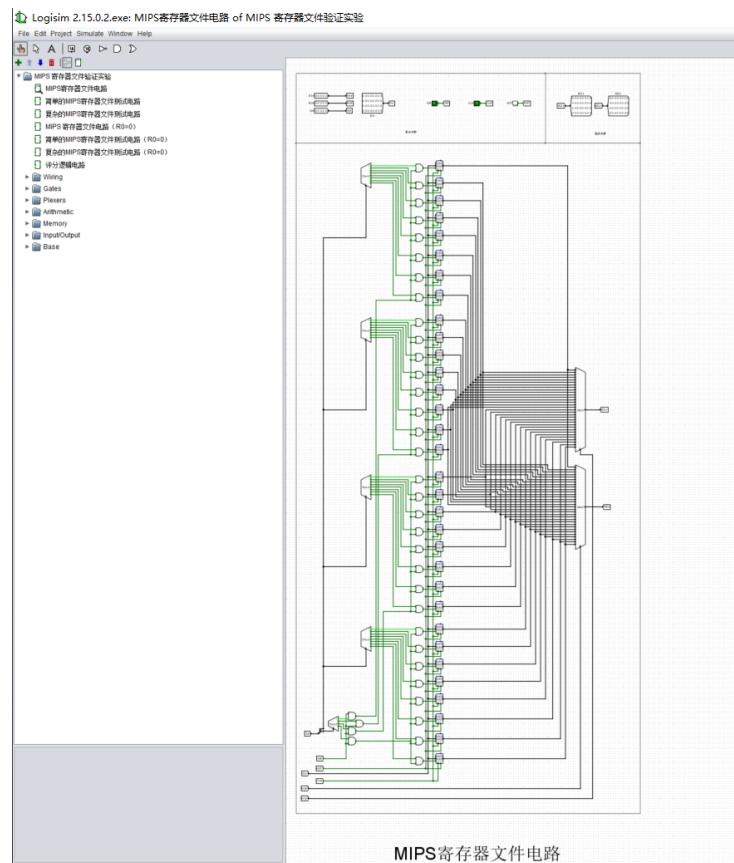


参考设计文件：MIPS寄存器文件验证实验.circ

四、MIPS 寄存器文件验证实验

- 1、MIPS 寄存器文件电路
- 2、简单的MIPS 寄存器文件测试电路
- 3、复杂的MIPS 寄存器文件测试电路
- 4、MIPS 寄存器文件电路 (R0=0)
- 5、简单的MIPS 寄存器文件测试电路 (R0=0)
- 6、复杂的MIPS 寄存器文件测试电路 (R0=0)

- 运行“**Logisim**”，在Logisim中打开“**MIPS寄存器文件验证实验.circ**”。
- 也可以直接双击“**MIPS寄存器文件验证实验.circ**”，打开该设计文件。

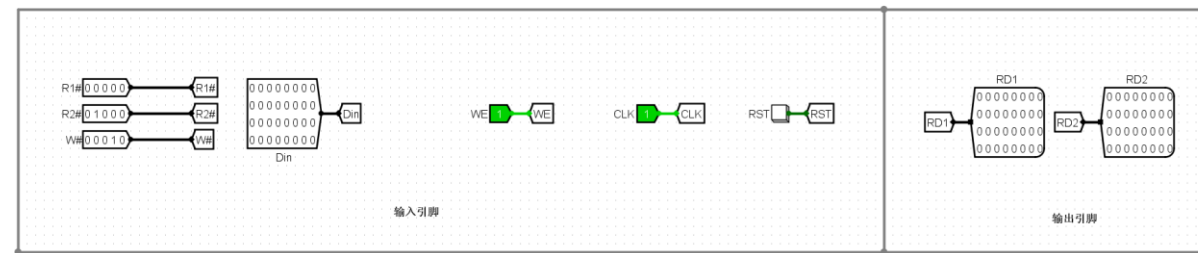


MIPS: Microprocessor without Interlocked Pipeline Stages, 没有互锁流水线的微处理器

MIPS 寄存器文件: 该寄存器文件用于采用MIPS指令系统的CPU

• 1、MIPS 寄存器文件电路

- **MIPS 寄存器文件** (也称寄存器组、寄存器堆; 用于采用MIPS指令系统的CPU) 电路由**32个32位的寄存器**构成, 寄存器编号为R0~R31; 该寄存器文件有**2个读出口、1个写入口**。
- MIPS 寄存器文件电路的输入、输出和控制信号为:
 - **输入:**
 - R1#: 读出口1的寄存器编号 (5位)
 - R2#: 读出口2的寄存器编号 (5位)
 - W#: 写入口Din的寄存器编号 (5位)
 - Din: 写入的数据 (32位)
 - **输出:**
 - RD1: 读出口1的输出 (32位)
 - RD2: 读出口2的输出 (32位)
 - **控制信号 (输入):**
 - WE: 读写控制信号; WE=1, 写入; WE=0, 读出
 - CLK: 时钟信号
 - RST: 复位信号 (该信号不用于封装)



MIPS 寄存器文件设计电路的输入、输出和控制信号

MIPS 寄存器文件电路

3-8译码器

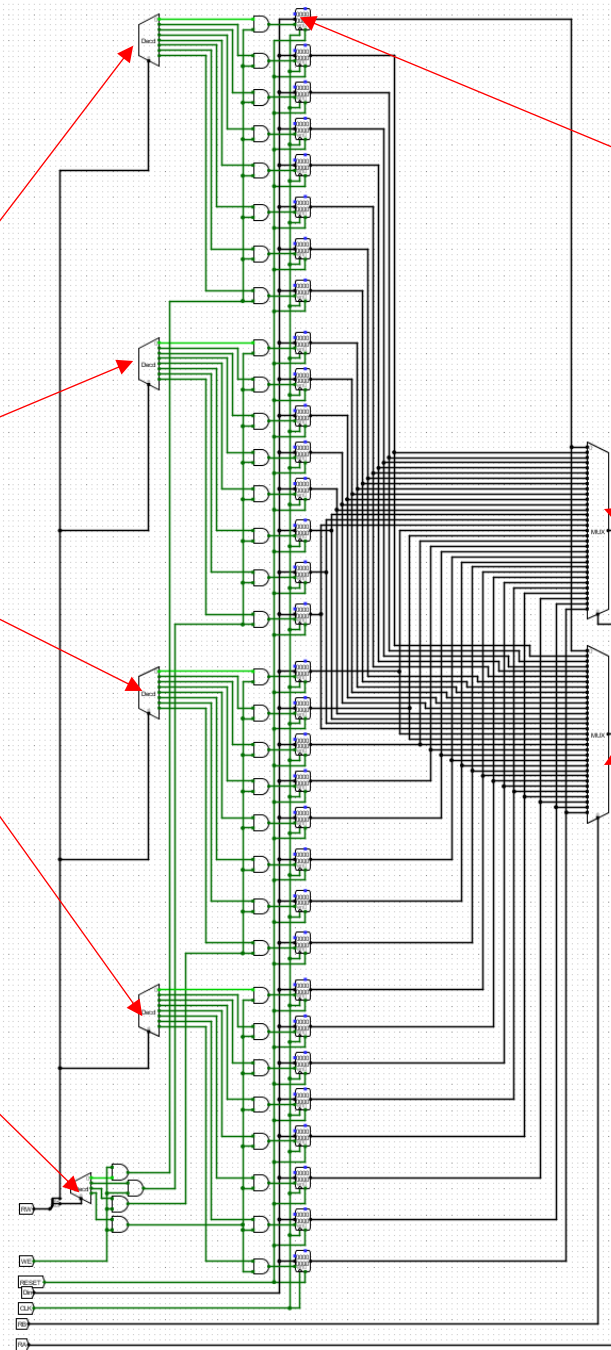
2-4译码器

MIPS 寄存器文件电路

32位寄存器

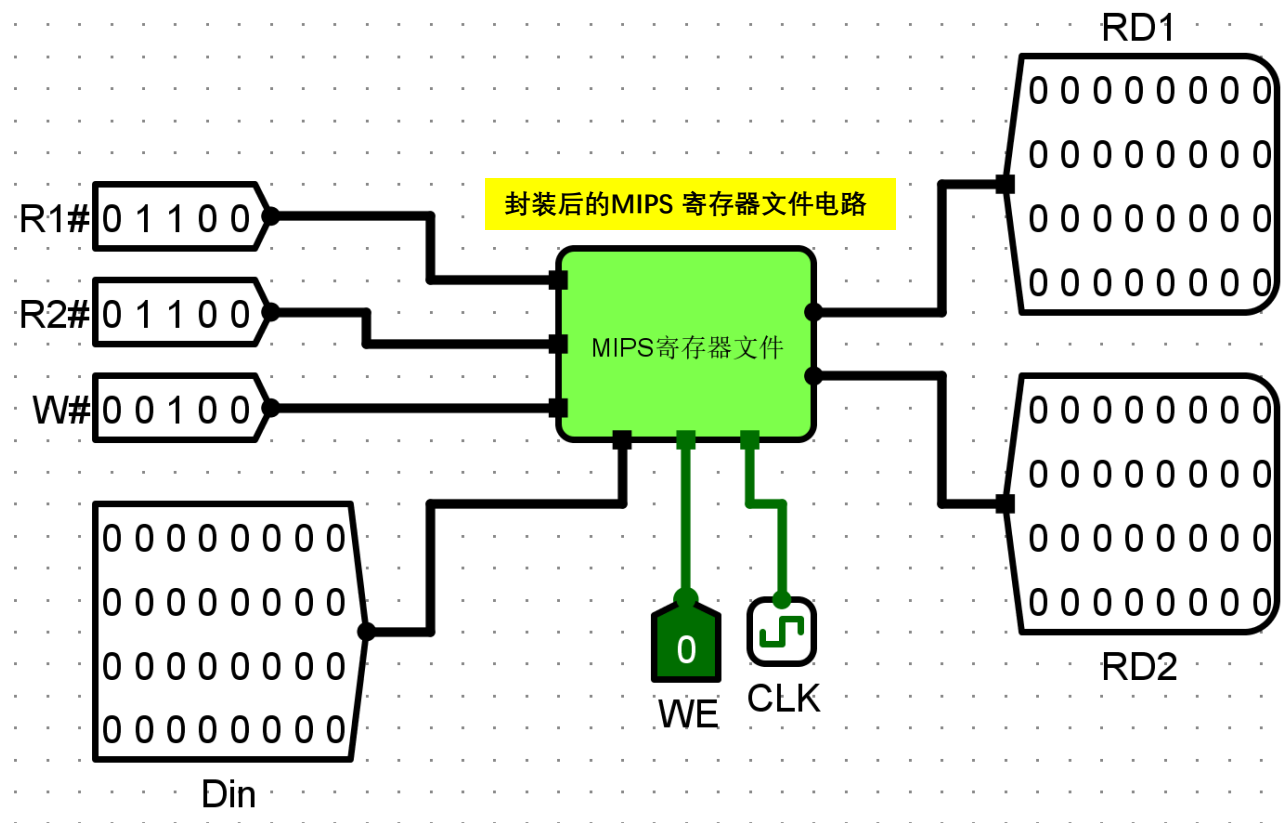
共32个

32选1多路选择器



• 2、简单的 MIPS 寄存器文件测试电路

- 将前面的MIPS 寄存器文件电路封装后，再添加一些输入、输出及控制信号，就可以构成一个简单的 MIPS 寄存器文件测试电路：

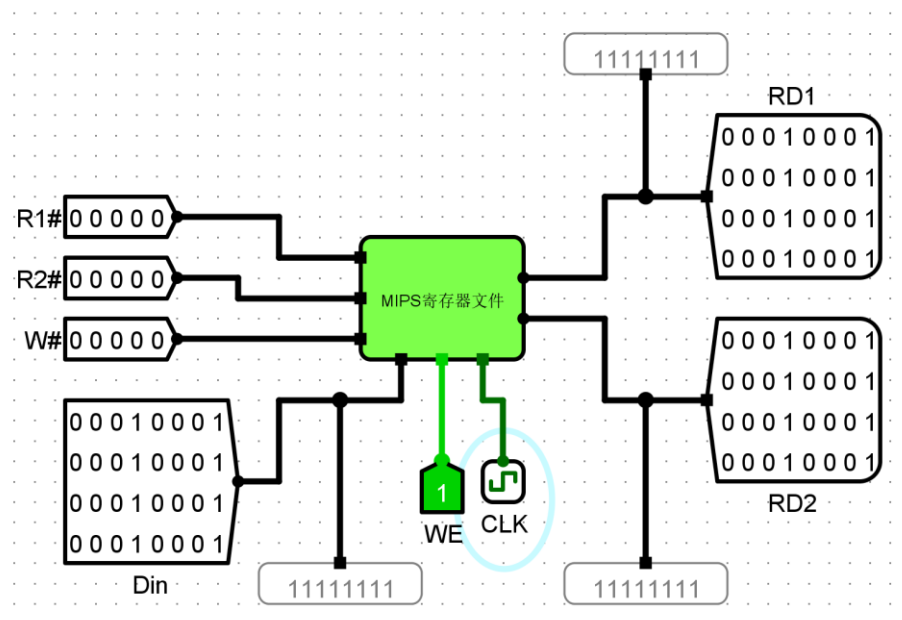


简单的 MIPS 寄存器文件测试电路

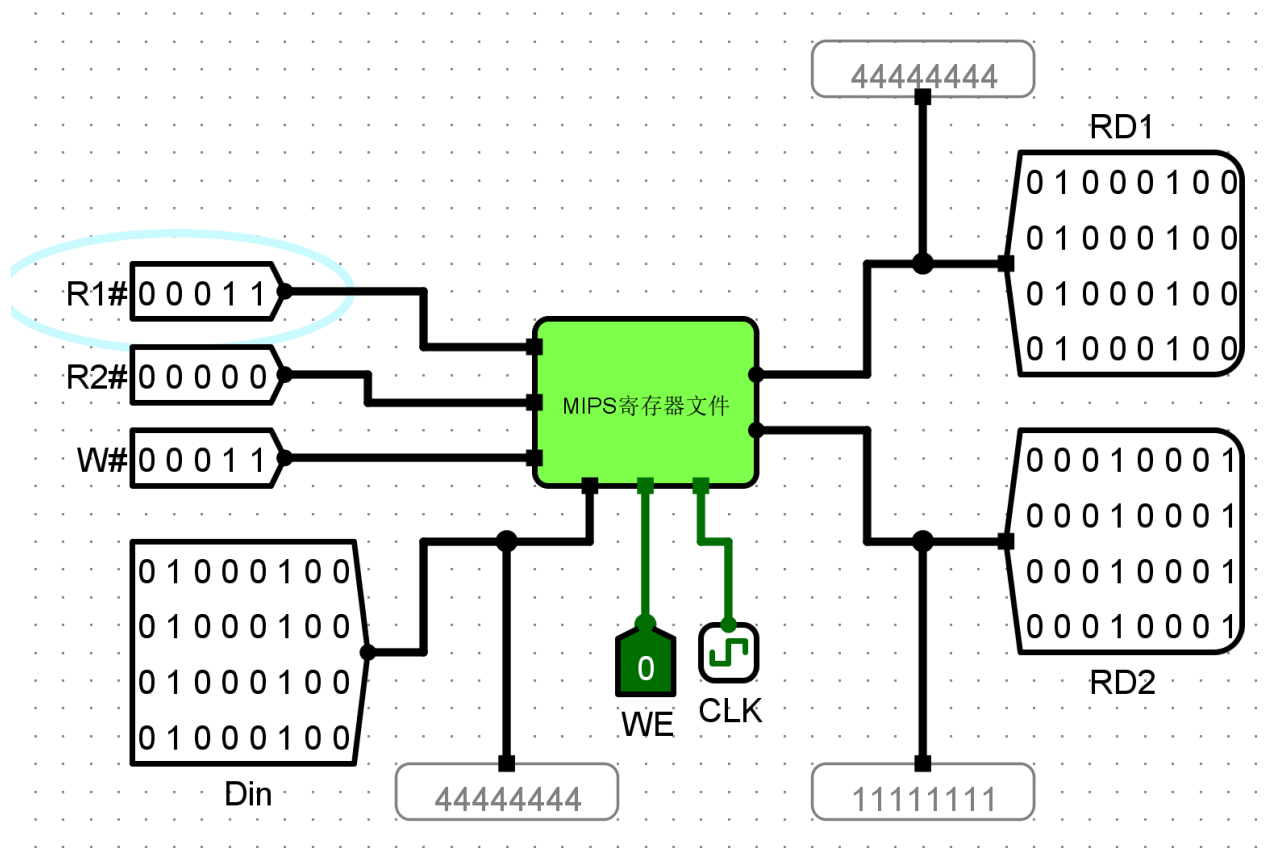
- 测试步骤:

- 第1步: 向寄存器0~寄存器3, 分别写入11111111、22222222、33333333、44444444 (均为十六进制, 下同)。

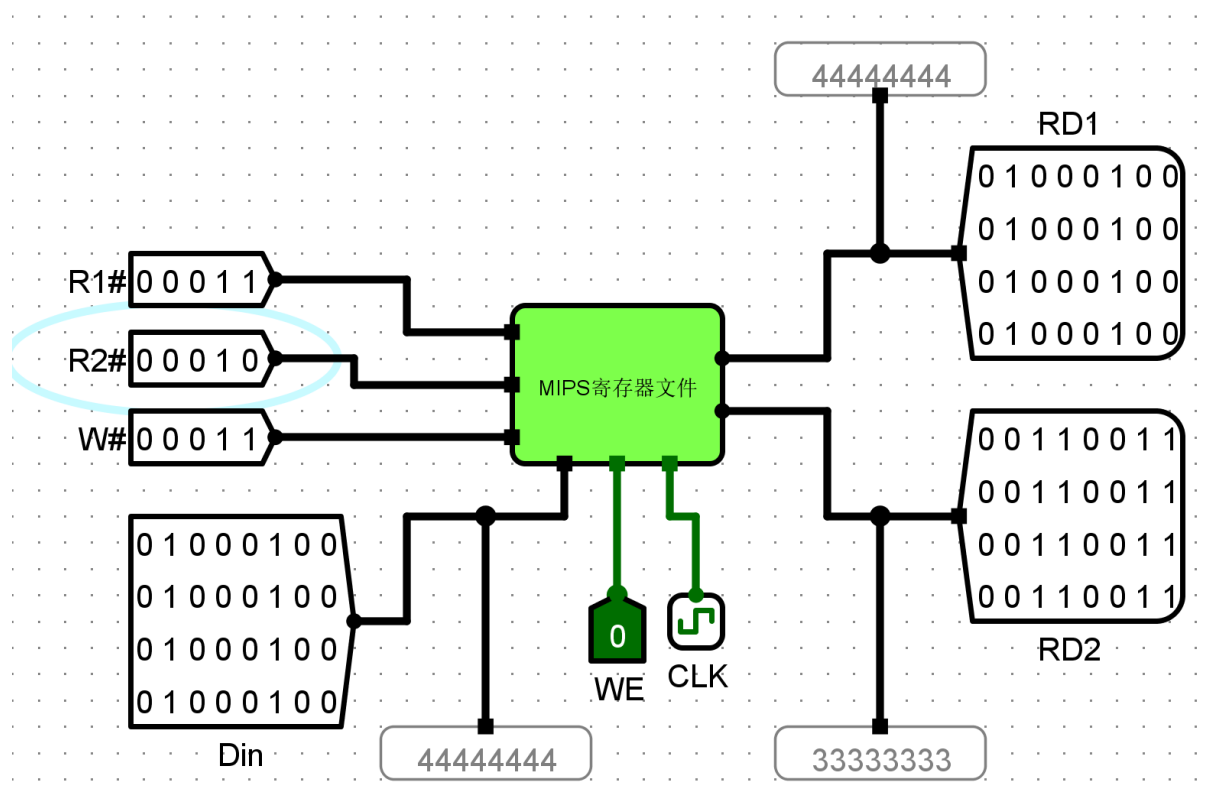
- 置W#=00000, Din=11111111H, WE=1, 按时钟CLK按钮2次
- 置W#=00001, Din=22222222H, WE=1, 按时钟CLK按钮2次
- 置W#=00010, Din=33333333H, WE=1, 按时钟CLK按钮2次
- 置W#=00011, Din=44444444H, WE=1, 按时钟CLK按钮2次



- 第2步：从RD1口读寄存器0~寄存器3。
- 置R1#=00000, WE=0, 观察RD1是否=11111111H
- 置R1#=00001, WE=0, 观察RD1是否=22222222H
- 置R1#=00010, WE=0, 观察RD1是否=33333333H
- 置R1#=00011, WE=0, 观察RD1是否=44444444H

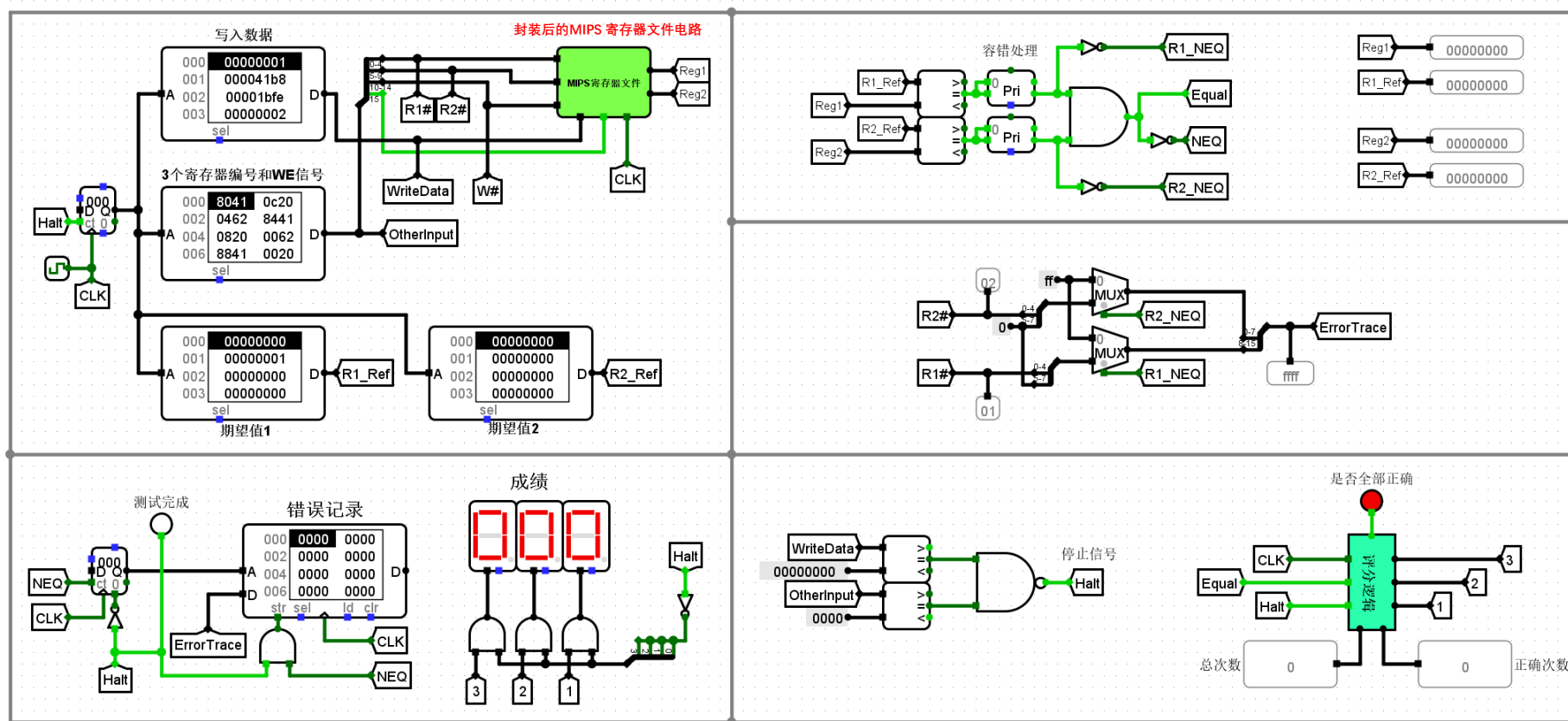


- 第3步：从RD2口读寄存器0~寄存器3。
- 置R2#=00000, WE=0, 观察RD2是否=11111111H
- 置R2#=00001, WE=0, 观察RD2是否=22222222H
- 置R2#=00010, WE=0, 观察RD2是否=33333333H
- 置R2#=00011, WE=0, 观察RD2是否=44444444H

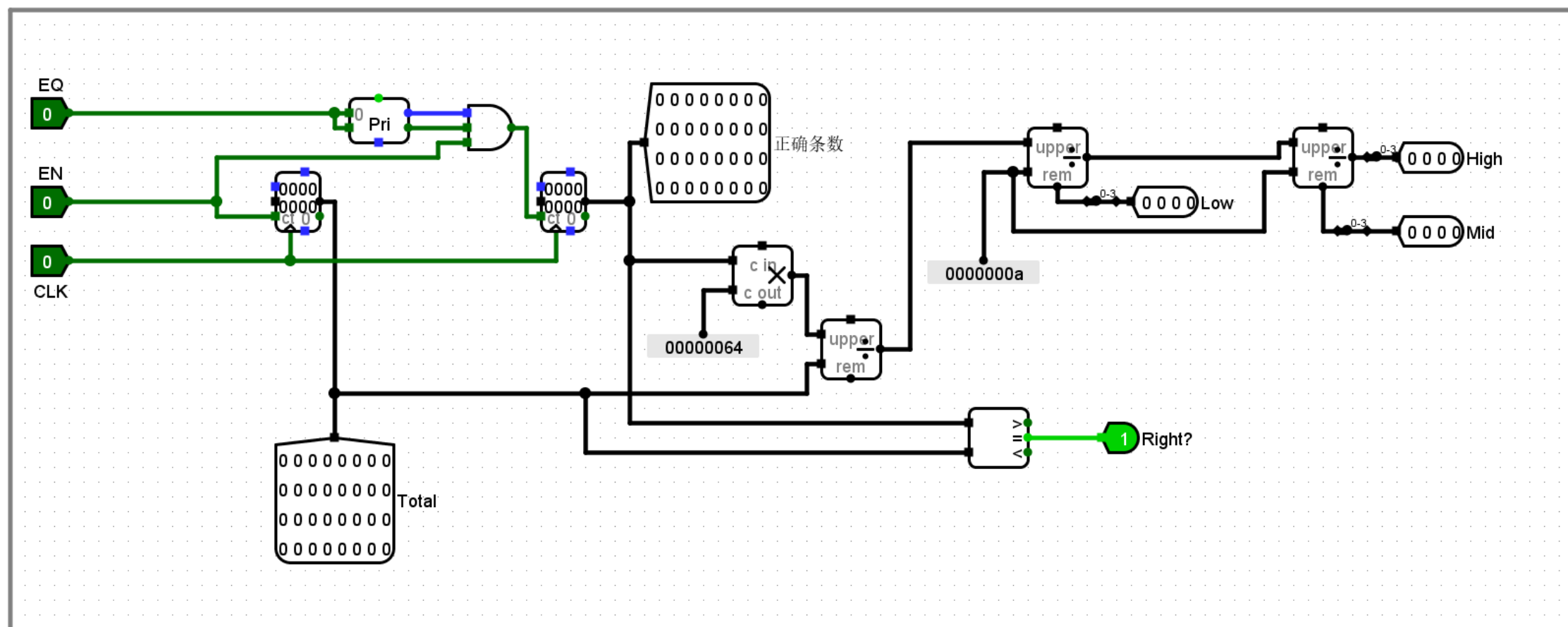


• 3、复杂的 MIPS 寄存器文件测试电路

- 测试步骤：先按Ctrl+R，使电路复位；然后按Ctrl+K，启动时钟；如果显示“100”，则表示测试通过。



复杂的 MIPS 寄存器文件测试电路



评分逻辑电路

4个存储器中的内容

8041=1000 0000 0100 0001

1 00000 00010 00001

WE W# R2# R1#

向寄存器0中写入1

MIPS寄存器文件测试用例——3个寄存器编号和WE信号.txt - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw

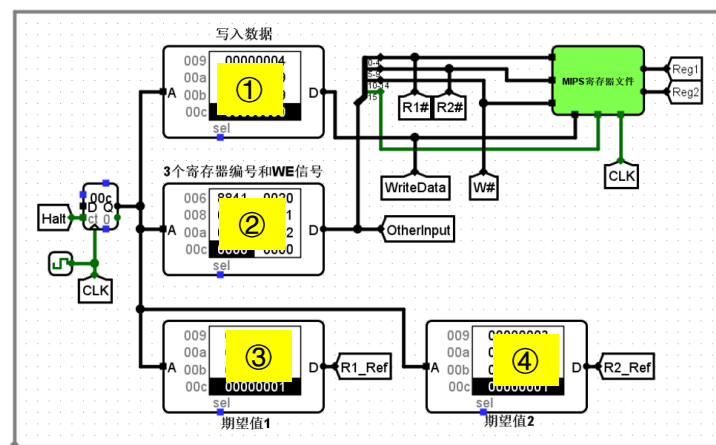
8041	c20	462
8441	820	62
8841	20	c62
8c41	820	462

MIPS寄存器文件测试用例——写入数据.txt - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw

1	41b8	1bfe
2	44a8	1ca7
3	5f7a	2f2a
4	3d39	6009

上述文本文件在“MIPS 寄存器文件测试用例.zip”中



4个存储器中的内容

c20=0000 1100 0010 0000

0	00011	00001	00000
WE	W#	R2#	R1#

R1#口读寄存器0、R2#口读寄存器1，读出的值分别是1、0

MIPS寄存器文件测试用例——3个寄存器编号和WE信号.txt - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw

8041	c20	462
8441	820	62
8841	20	c62
8c41	820	462

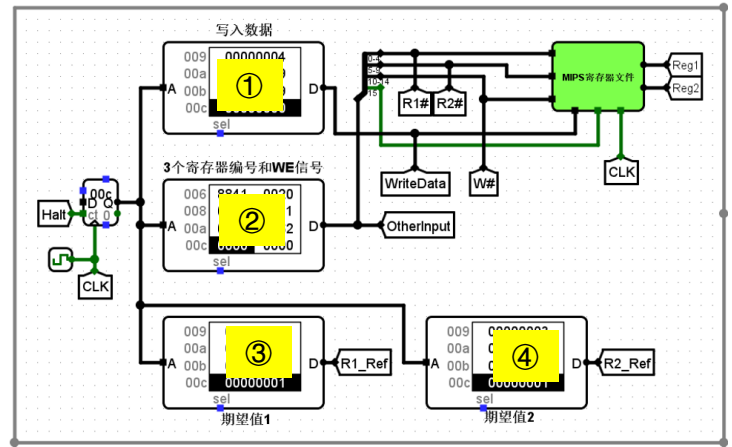
上述文本文件在“MIPS 寄存器文件测试用例.zip”中

MIPS寄存器文件测试用例——期望值1.txt - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw

0	1	0
0	1	0
2	1	3
2	1	3
1		



MIPS寄存器文件测试用例——期望值2.txt - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw

0	0	0
0	2	0
0	2	0
3	2	4
1		

4个存储器中的内容

462=0000 0100 0110 0010

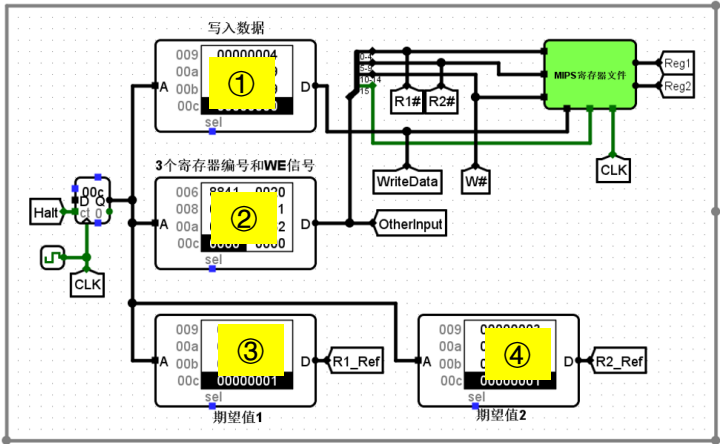
0	00001	00011	00010
WE	W#	R2#	R1#

R1#口读寄存器2、R2#口读寄存器3，读出的值分别是0、0

```
MIPS寄存器文件测试用例——3个寄存器编号和WE信号.txt - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
v2.0 raw
8041 c20 462
8441 820 62
8841 20 c62
8c41 820 462
```

上述文本文件在“MIPS 寄存器文件测试用例.zip”中

```
MIPS寄存器文件测试用例——期望值1.txt - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
v2.0 raw
0 1 0
0 1 0
2 1 3
2 1 3
1
```



```
MIPS寄存器文件测试用例——期望值2.txt - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)
v2.0 raw
0 0 0
0 2 0
0 2 0
3 2 4
1
```

4个存储器中的内容

8441=1000 0100 0100 0001

1 00001 00010 00001

WE W# R2# R1#

向寄存器1中写入2

MIPS寄存器文件测试用例——3个寄存器编号和WE信号.txt - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw

8041 c20 462

8441 820 62

8841 20 c62

8c41 820 462

MIPS寄存器文件测试用例——写入数据.txt - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw

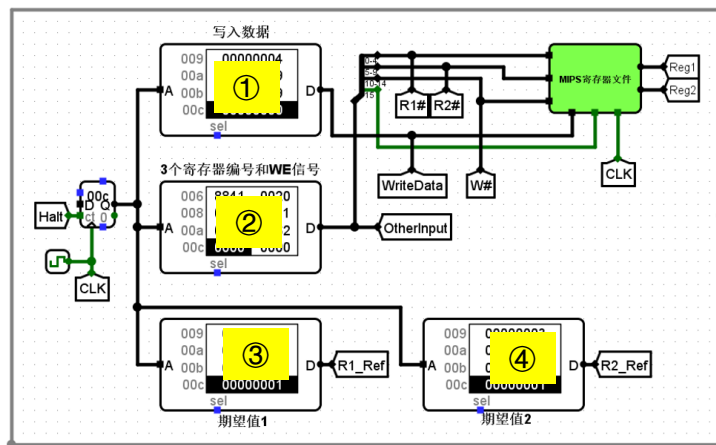
1 41b8 1bfe

2 44a8 1ca7

3 5f7a 2f2a

4 3d39 6009

上述文本文件在“**MIPS 寄存器文件测试用例.zip**”中



4个存储器中的内容

820=0000 1000 0010 0000

0 00010 00001 00000

WE W# R2# R1#

R1#口读寄存器0、R2#口读寄存器1，读出的值分别是1、0

MIPS寄存器文件测试用例——3个寄存器编号和WE信号.txt - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw

8041 c20 462

8441 820 62

8841 20 c62

8c41 820 462

上述文本文件在“MIPS 寄存器文件测试用例.zip”中

MIPS寄存器文件测试用例——期望值1.txt - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw

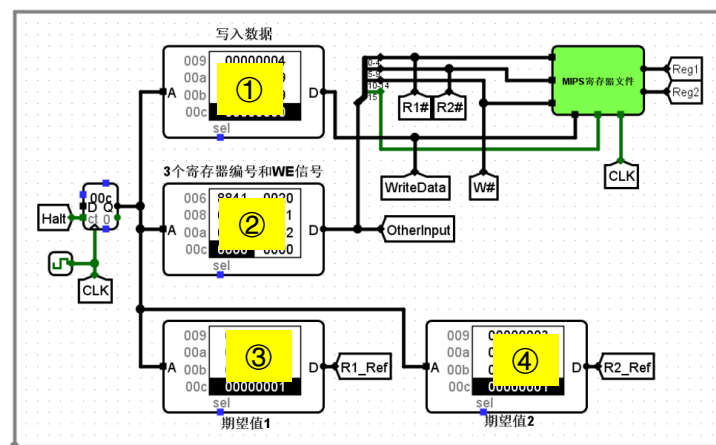
0 1 0

0 1 0

2 1 3

2 1 3

1



MIPS寄存器文件测试用例——期望值2.txt - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw

0 0 0

0 2 0

0 2 0

3 2 4

1

4个存储器中的内容

62=0000 0000 0110 0010

0 00000 00011 00010

WE W# R2# R1# R1#口读寄存器2、R2#口读寄存器3，读出的值分别是0、0

MIPS寄存器文件测试用例——3个寄存器编号和WE信号.txt - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw

8041 c20 462

8441 820 62

8841 20 c62

8c41 820 462

上述文本文件在“MIPS 寄存器文件测试用例.zip”中

MIPS寄存器文件测试用例——期望值1.txt - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw

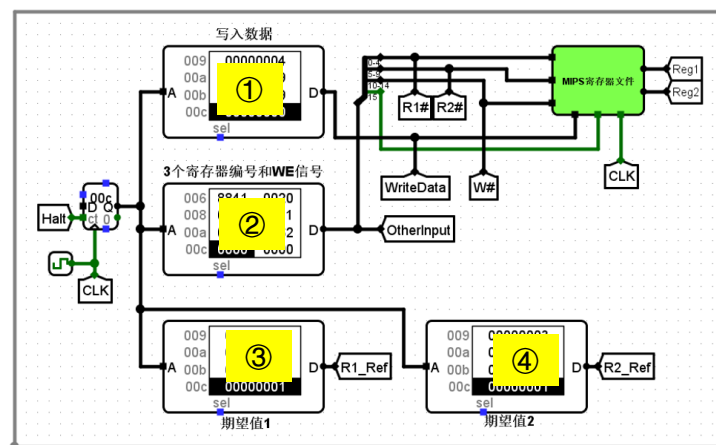
0 1 0

0 1 0

2 1 3

2 1 3

1



MIPS寄存器文件测试用例——期望值2.txt - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw

0 0 0

0 2 0

0 2 0

3 2 4

1

4个存储器中的内容

8841=1000 1000 0100 0001

WE	W#	R2#	R1#
1	00010	00010	00001

向寄存器2中写入3

MIPS寄存器文件测试用例——3个寄存器编号和WE信号.txt - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw

8041	c20	462
8441	820	62
8841	20	c62
8c41	820	462

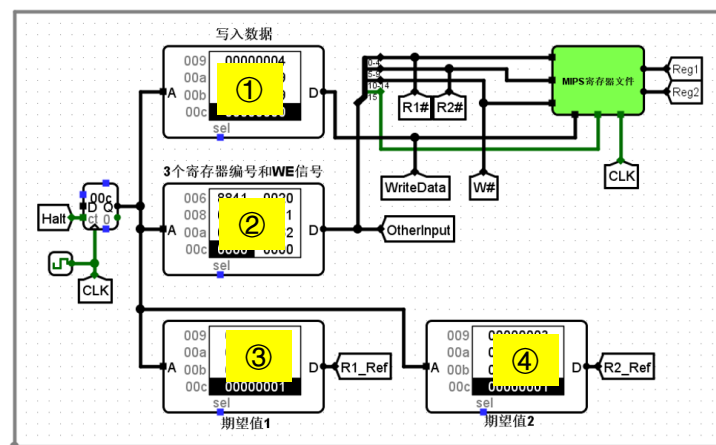
MIPS寄存器文件测试用例——写入数据.txt - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw

1	41b8	1bfe
2	44a8	1ca7
3	5f7a	2f2a
4	3d39	6009

上述文本文件在“MIPS 寄存器文件测试用例.zip”中



4个存储器中的内容

20=0000 0000 0010 0000

0 00000 00001 00000

WE W# R2# R1#

R1#口读寄存器0、R2#口读寄存器1，读出的值分别是1、2

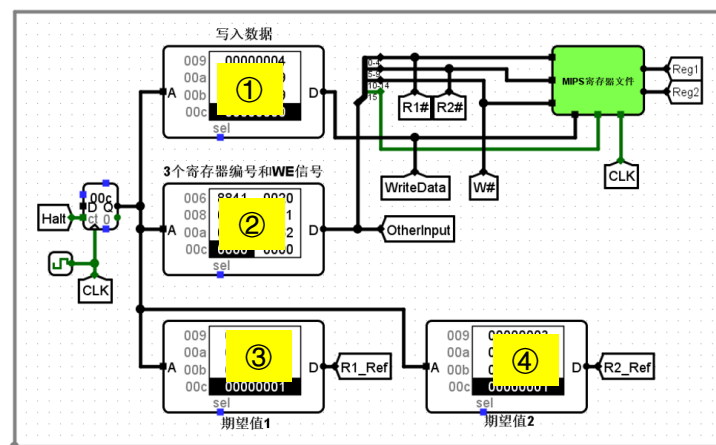
MIPS寄存器文件测试用例——3个寄存器编号和WE信号.txt - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw
8041 c20 462
8441 820 62
8841 20 c62
8c41 820 462

上述文本文件在“MIPS 寄存器文件测试用例.zip”中

MIPS寄存器文件测试用例——期望值1.txt - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw
0 1 0
0 1 0
2 1 3
2 1 3
1



MIPS寄存器文件测试用例——期望值2.txt - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw
0 0 0
0 2 0
0 2 0
3 2 4
1

4个存储器中的内容

c62=0000 1100 0110 0010

0 00011 00011 00010

WE W# R2# R1#

R1#口读寄存器2、R2#口读寄存器3，读出的值分别是3、0

MIPS寄存器文件测试用例——3个寄存器编号和WE信号.txt - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw

8041 c20 462

8441 820 62

8841 20 → c62

8c41 820 462

上述文本文件在“MIPS 寄存器文件测试用例.zip”中

MIPS寄存器文件测试用例——期望值1.txt - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw

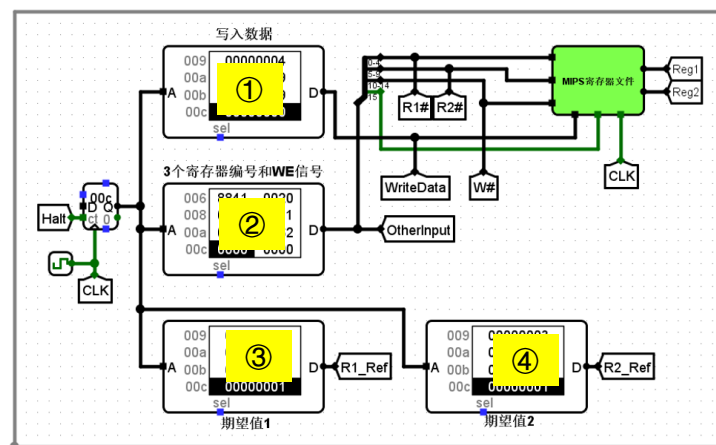
0 1 0

0 1 0

2 1 3

2 1 3

1



MIPS寄存器文件测试用例——期望值2.txt - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw

0 0 0

0 2 0

0 2 0

3 2 4

1

4个存储器中的内容

8c41=1000 1000 0100 0001

1 00011 00010 00001

WE W# R2# R1#

向寄存器3中写入4

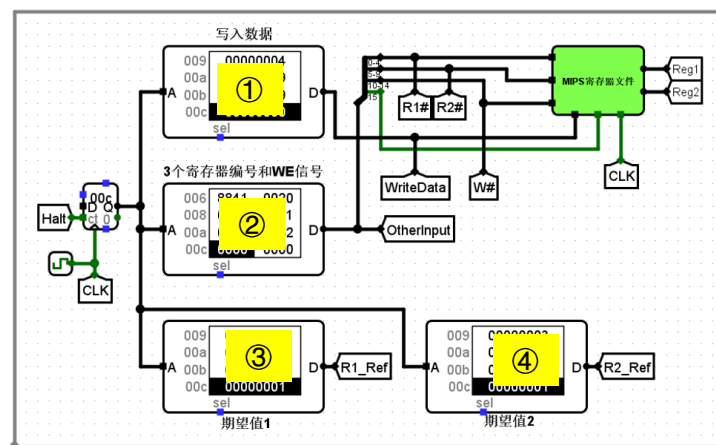
MIPS寄存器文件测试用例——3个寄存器编号和WE信号.txt - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw
8041 c20 462
8441 820 62
8841 20 c62
8c41 820 462

MIPS寄存器文件测试用例——写入数据.txt - 记事本
文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw
1 41b8 1bfe
2 44a8 1ca7
3 5f7a 2f2a
4 3d39 6009

上述文本文件在“MIPS 寄存器文件测试用例.zip”中



4个存储器中的内容

820=0000 1000 0010 0000

0 00010 00001 00000

WE W# R2# R1#

R1#口读寄存器0、R2#口读寄存器1，读出的值分别是1、2

MIPS寄存器文件测试用例——3个寄存器编号和WE信号.txt - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw

8041 c20 462

8441 820 62

8841 20 c62

8c41 820 462

上述文本文件在“MIPS 寄存器文件测试用例.zip”中

MIPS寄存器文件测试用例——期望值1.txt - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw

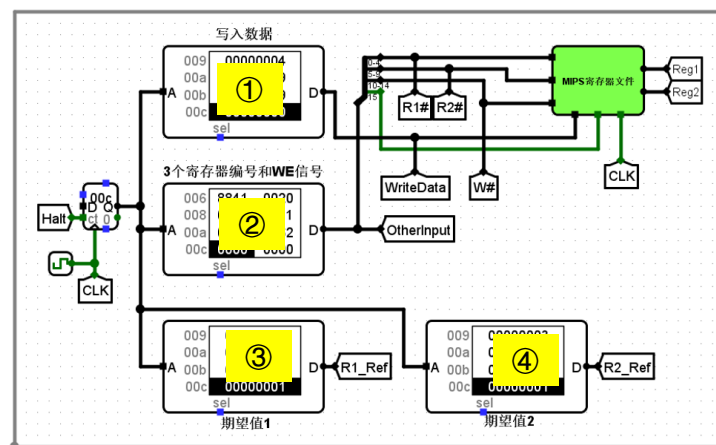
0 1 0

0 1 0

2 1 3

2 1 3

1



MIPS寄存器文件测试用例——期望值2.txt - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw

0 0 0

0 2 0

0 2 0

3 2 4

1

4个存储器中的内容

462=0000 0100 0110 0010

0 00001 00011 00010

WE W# R2# R1#

R1#口读寄存器2、R2#口读寄存器3，读出的值分别是3、4

MIPS寄存器文件测试用例——3个寄存器编号和WE信号.txt - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw

8041 c20 462

8441 820 62

8841 20 c62

8c41 820 462

上述文本文件在“MIPS 寄存器文件测试用例.zip”中

MIPS寄存器文件测试用例——期望值1.txt - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw

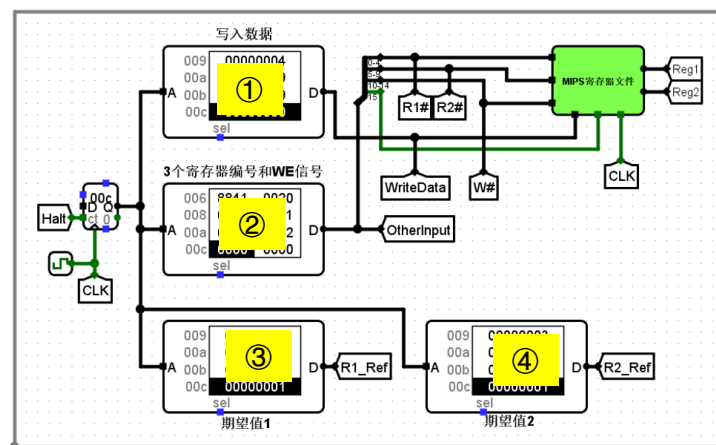
0 1 0

0 1 0

2 1 3

2 1 3

1



MIPS寄存器文件测试用例——期望值2.txt - 记事本

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw

0 0 0

0 2 0

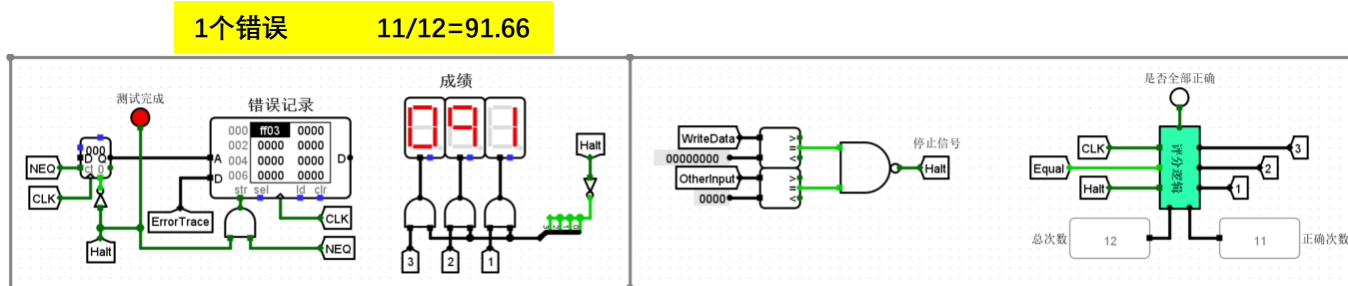
0 2 0

3 2 4

1

• 什么时候出现非“100”?

- (1) MIPS 寄存器文件电路有问题。
- (2) 修改写入数据存储器的内容。



MIPS寄存器文件测试用例——写入数据.txt

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw

1 41b8 1bfe
2 44a8 1ca7
3 5f7a 2f2a
→ 4 3d39 6009

“4”修改为“5”

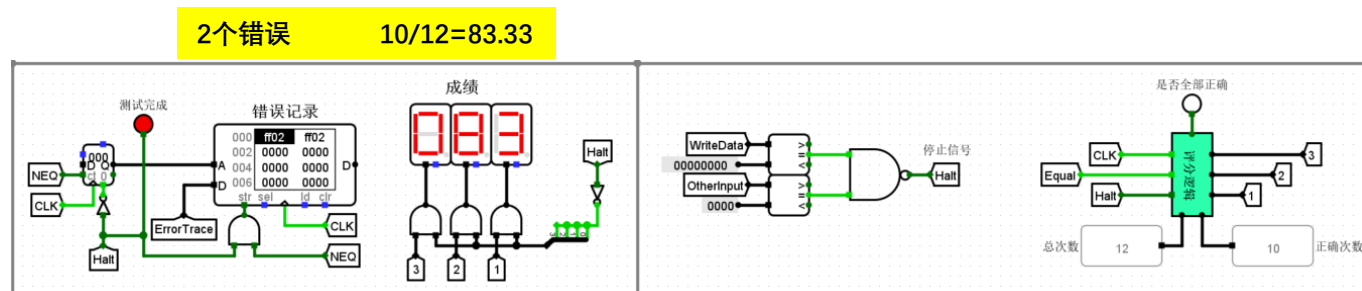
MIPS寄存器文件测试用例——修改后的写入数据.txt

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw

1 41b8 1bfe
2 44a8 1ca7
3 5f7a 2f2a
→ 5 3d39 6009

- (3) 修改期望值1存储器或期望值2存储器的内容。



MIPS寄存器文件测试用例——期望值1.txt

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw

0 1 0
0 1 0
→ 2 1 3
2 1 3
1

“2”修改为“1”

MIPS寄存器文件测试用例——修改后的期望值1.txt

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw

0 1 0
0 1 0
→ 1 1 3
2 1 3
1

MIPS寄存器文件测试用例——期望值2.txt

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw

→ 0 0 0
0 2 0
0 2 0
→ 3 2 4
1

“0”修改为“1”

“3”修改为“2”

MIPS寄存器文件测试用例——修改后的期望值2.txt

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw

→ 1 0 0
0 2 0
0 2 0
→ 2 2 4
1

上述文本文件在“MIPS 寄存器文件测试用例.zip”中

• 4、MIPS 寄存器文件电路 (R0=0)

- 在基于MIPS指令系统的CPU中，通常要求**R0寄存器恒等于0**；通过对前面的MIPS 寄存器文件电路进行适当修改，可以得到R0=0的MIPS 寄存器文件电路。

- MIPS 寄存器文件电路 (R0=0) 的输入、输出和控制信号为：

- **输入：**

- R1#：读出口1的寄存器编号（5位）
- R2#：读出口2的寄存器编号（5位）
- W#：写入口Din的寄存器编号（5位）
- Din：写入的数据（32位）

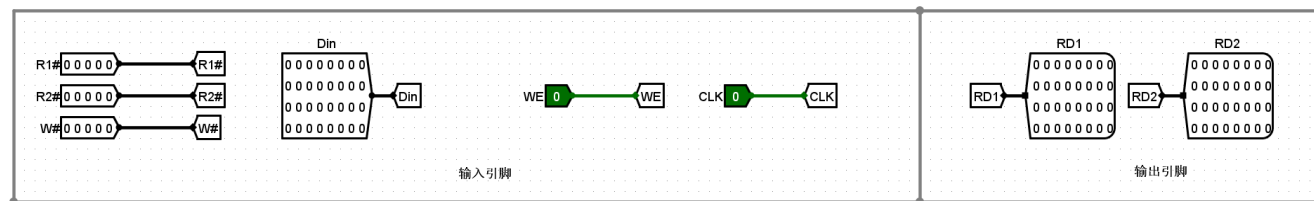
- **输出：**

- RD1：读出口1的输出（32位）
- RD2：读出口2的输出（32位）

- **控制信号（输入）：**

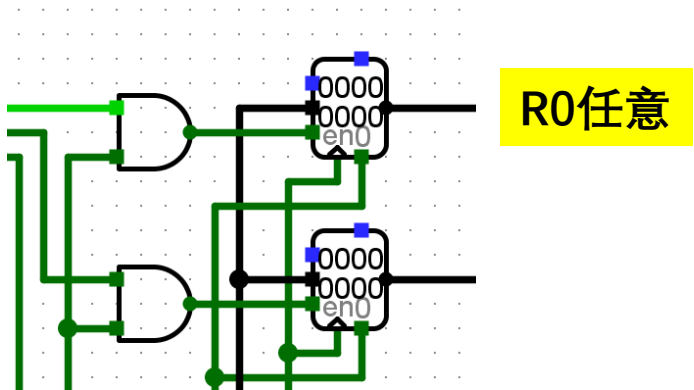
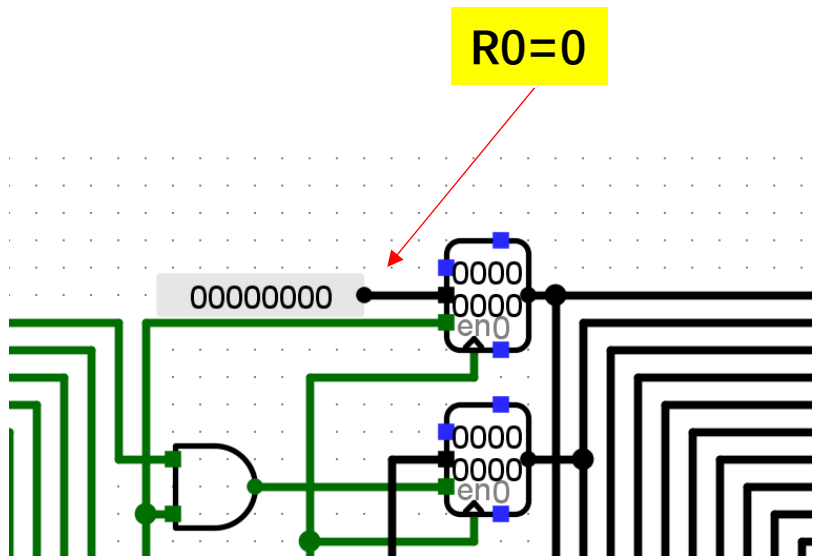
- WE：读写控制信号；WE=1，写入；WE=0，读出
- CLK：时钟信号

- 该电路没有RST信号（复位信号）



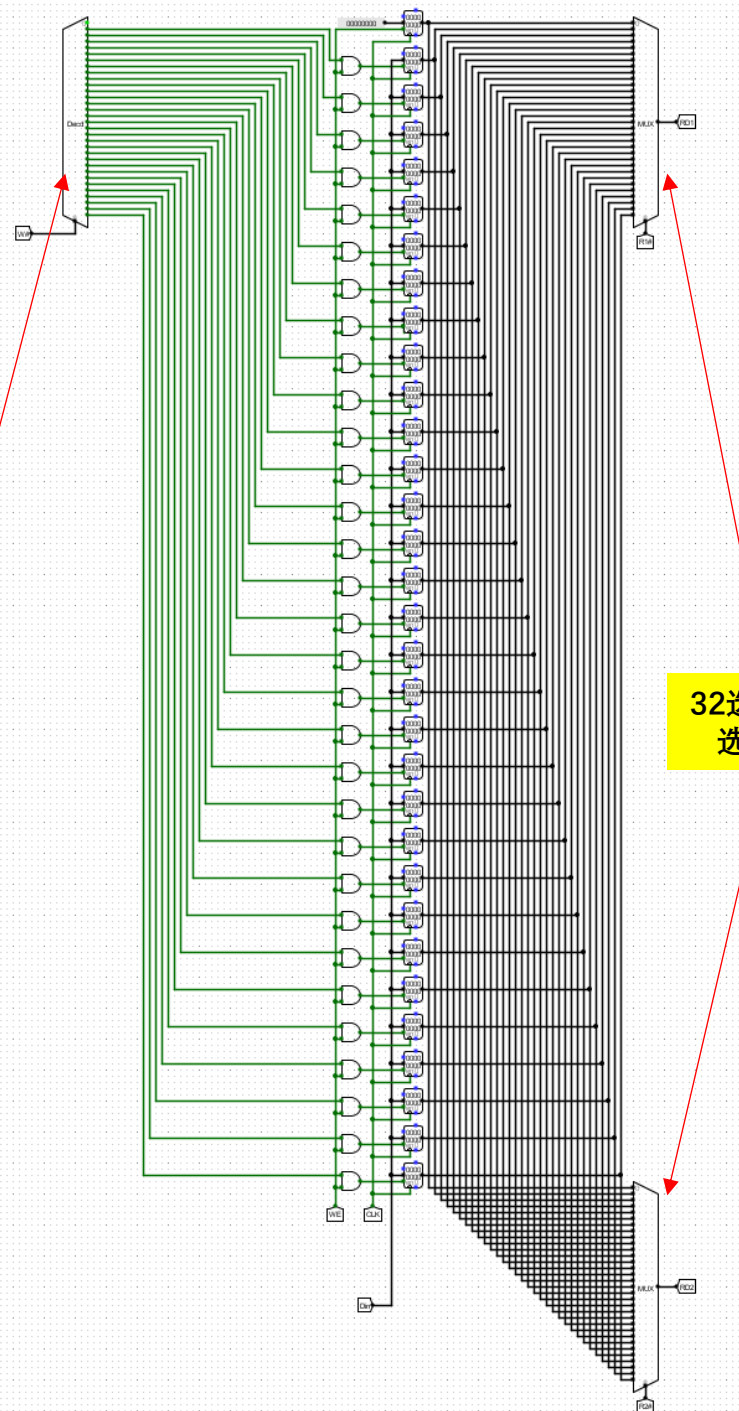
MIPS 寄存器文件电路 (R0=0) 的输入、输出和控制信号

MIPS 寄存器文件电路 (R0=0)



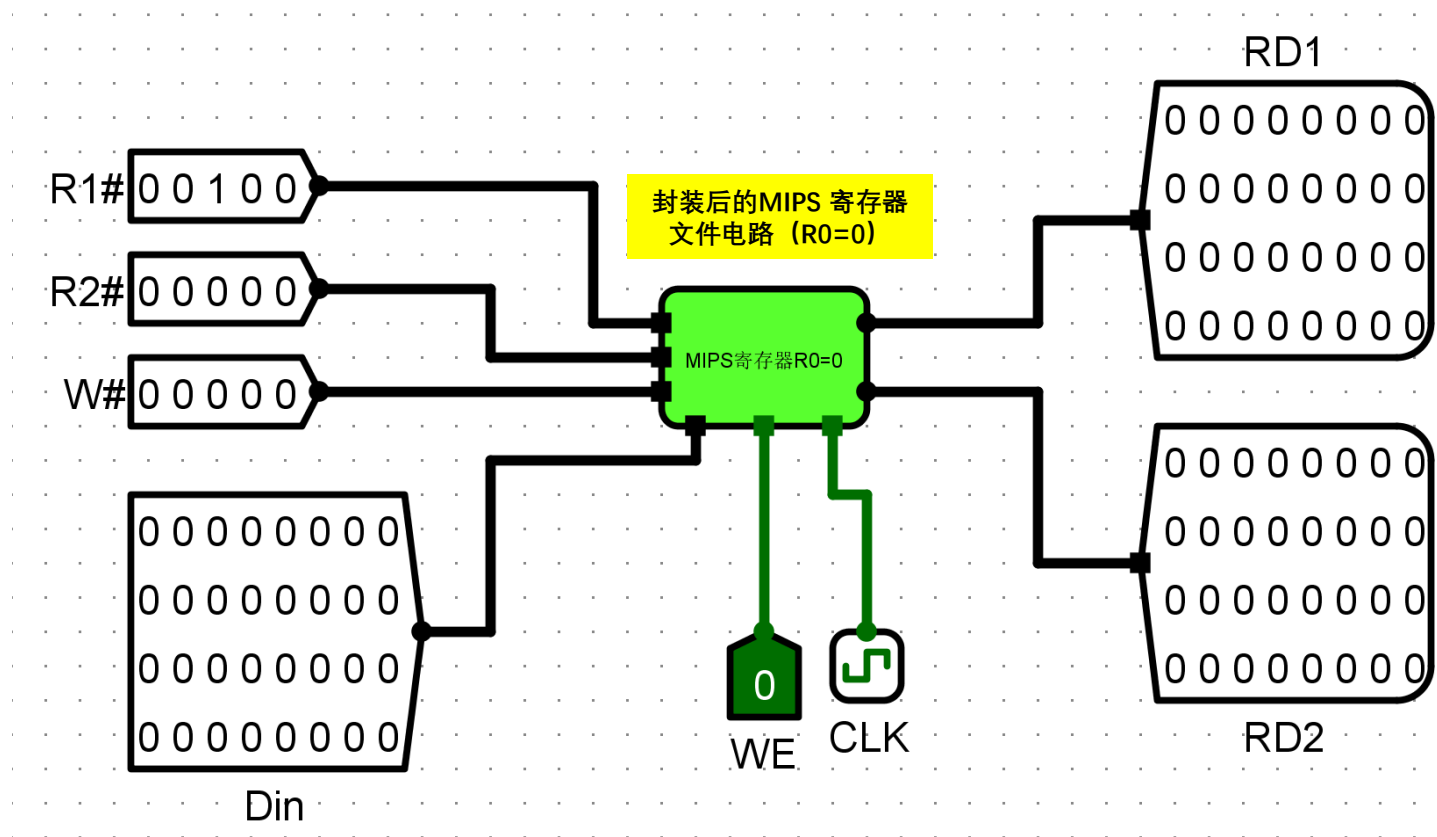
5-32译码器

32选1多路
选择器



• 5、简单的 MIPS 寄存器文件测试电路 (R0=0)

- 将前面的MIPS 寄存器文件电路 (R0=0) **封装**后, 再添加一些输入、输出及控制信号, 就可以构成一个简单的 MIPS 寄存器文件测试电路 (R0=0) :

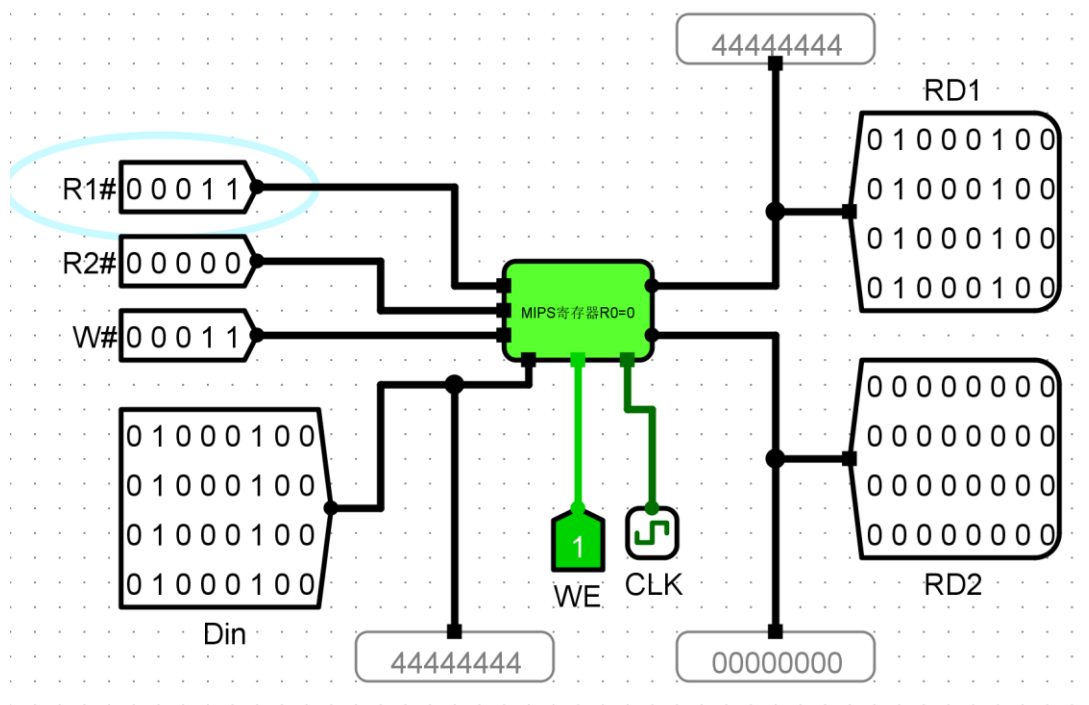


简单的 MIPS 寄存器文件测试电路 (R0=0)

- 测试步骤:

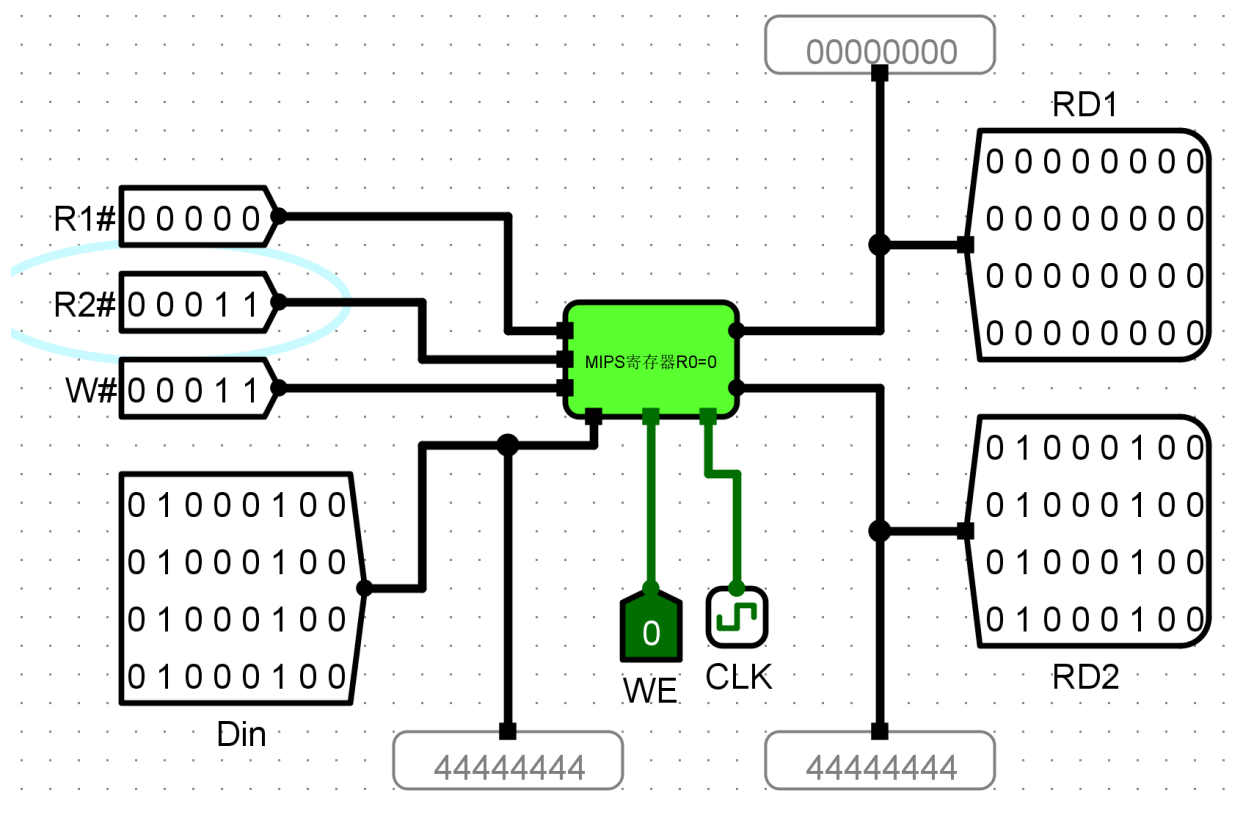
- 第1步: 向寄存器0~寄存器3, 分别写入11111111、22222222、33333333、44444444 (均为十六进制, 下同)。

- 置W#=00000, Din=11111111H, WE=1, 按时钟CLK按钮2次
- 置W#=00001, Din=22222222H, WE=1, 按时钟CLK按钮2次
- 置W#=00010, Din=33333333H, WE=1, 按时钟CLK按钮2次
- 置W#=00011, Din=44444444H, WE=1, 按时钟CLK按钮2次



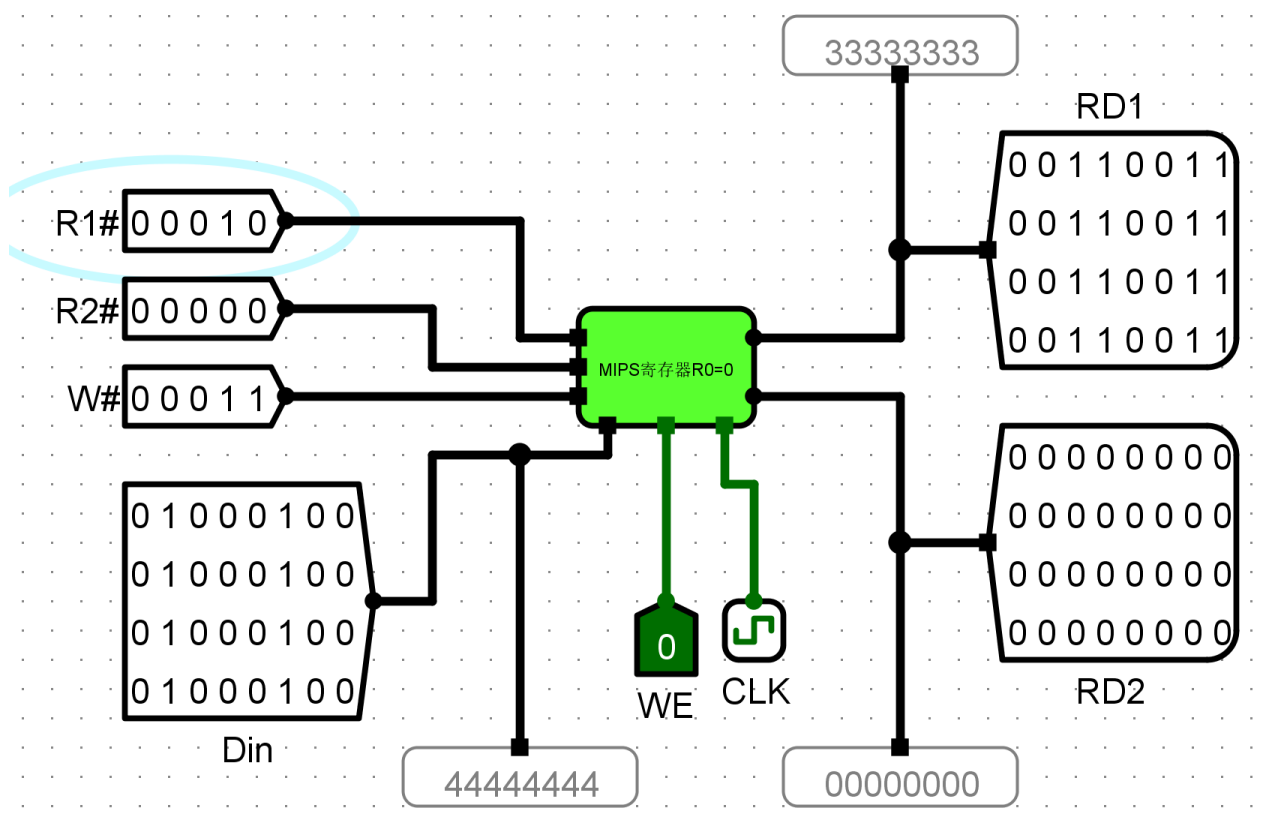
- 第2步：从RD1口读寄存器0~寄存器3。

- 置R1#=00000, WE=0, 观察RD1是否=00000000H (虽然前面向R0中写入了11111111H, 但是这里读出的R0仍然为0)
- 置R1#=00001, WE=0, 观察RD1是否=22222222H
- 置R1#=00010, WE=0, 观察RD1是否=33333333H
- 置R1#=00011, WE=0, 观察RD1是否=44444444H



- 第3步：从RD2口读寄存器0~寄存器3。

- 置R2#=00000, WE=0, 观察RD2是否=00000000H (虽然前面向R0中写入了11111111H, 但是这里读出的R0仍然为0)
- 置R2#=00001, WE=0, 观察RD2是否=22222222H
- 置R2#=00010, WE=0, 观察RD2是否=33333333H
- 置R2#=00011, WE=0, 观察RD2是否=44444444H



- 请同学们分析“**MIPS 寄存器文件电路**”的原理，指出该电路是如何实现：
 - ① 将数据写入到某个寄存器（R0 ~ R31）中；
 - ② 从RD1口读出某个寄存器（R0 ~ R31）的内容；
 - ③ 从RD2口读出某个寄存器（R0 ~ R31）的内容。
- 请同学们分析“**MIPS 寄存器文件电路（R0=0）**”的原理，指出该电路是如何实现：
 - ① 将数据写入到某个寄存器（R0 ~ R31）中；
 - ② 从RD1口读出某个寄存器（R0 ~ R31）的内容，并且一定是R0=0；
 - ③ 从RD2口读出某个寄存器（R0 ~ R31）的内容，并且一定是R0=0。
- 有兴趣的同学，请分析“复杂的 MIPS 寄存器文件测试电路”。

参考设计文件：寄存器文件设计实验.circ

五、寄存器文件设计实验

- 1、寄存器文件电路（4个寄存器）（设计实验）
- 2、寄存器文件电路（4个寄存器，R0=0）（设计实验）

• 1、寄存器文件电路（4个寄存器）（设计实验）

- 请同学们参考前面的 **MIPS 寄存器文件电路**，设计拥有4个32位寄存器的**寄存器文件电路**，该寄存器文件电路有2个读出口、1个写入口。

- 寄存器文件电路（4个寄存器）的输入、输出和控制信号为：

- **输入：**

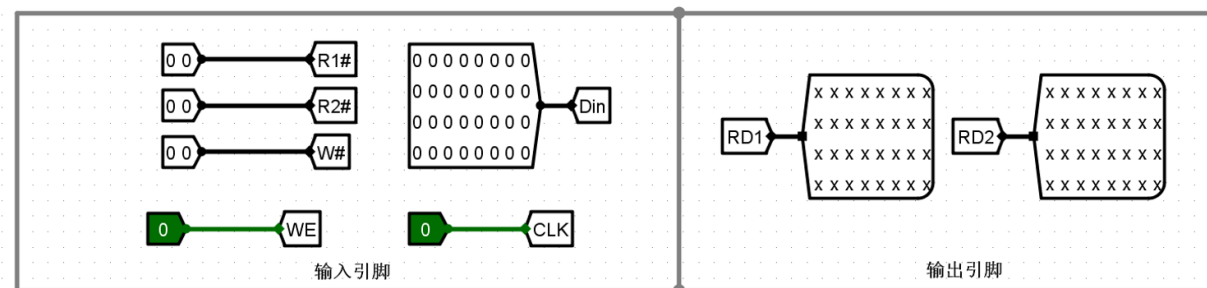
- R1#：读出口1的寄存器编号（2位）
- R2#：读出口2的寄存器编号（2位）
- W#：写入口Din的寄存器编号（2位）
- Din：写入的数据（32位）

- **输出：**

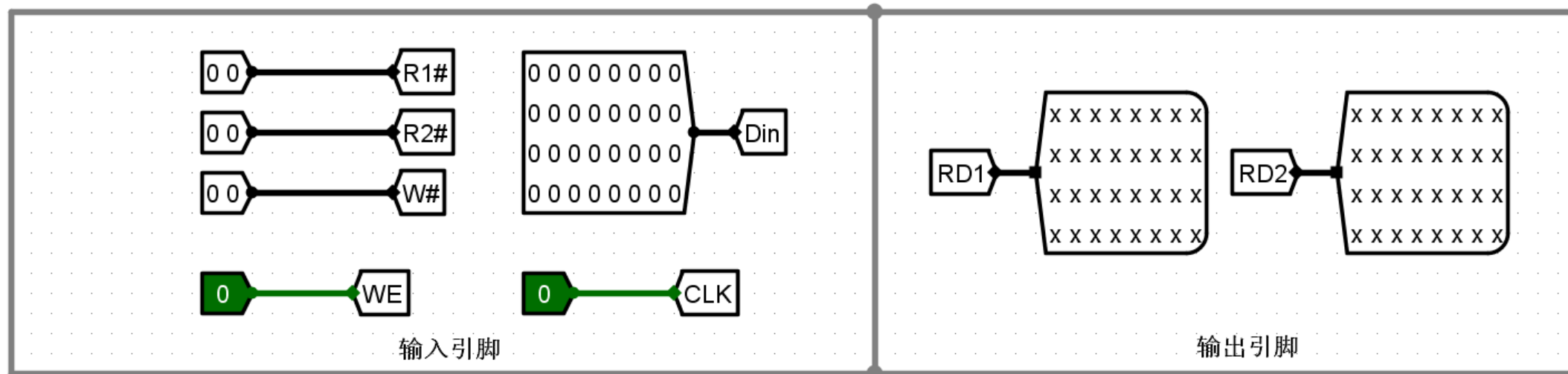
- RD1：读出口1的输出（32位）
- RD2：读出口2的输出（32位）

- **控制信号（输入）：**

- WE：读写控制信号；WE=1，写入；WE=0，读出
- CLK：时钟信号



寄存器文件电路（4个寄存器）的输入、输出和控制信号



请同学们在此设计电路！

寄存器文件电路（4个寄存器）

- 请同学们参考前面的简单的 MIPS 寄存器文件测试电路，设计一个**测试电路**。
- 然后，对设计好的**寄存器文件电路（4个寄存器）**的功能进行测试，包括：
 - ① 向不同的寄存器中写入内容。
 - ② 分别从两个不同的端口读出第①步写入的内容。

• 2、寄存器文件电路（4个寄存器，R0=0）（设计实验）

- 请同学们参考前面的 **MIPS 寄存器文件电路 (R0=0)**，设计拥有4个32位寄存器的**寄存器文件电路**，该寄存器文件电路有2个读出口、1个写入口，并且要求R0恒等于0。

- 寄存器文件电路（4个寄存器，R0=0）的输入、输出和控制信号为：

- **输入：**

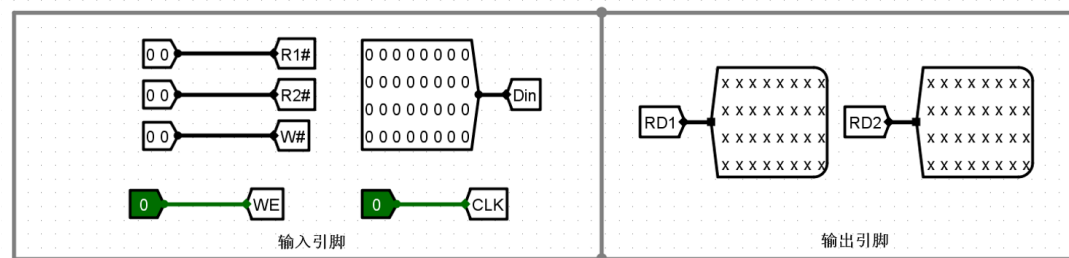
- R1#：读出口1的寄存器编号（2位）
- R2#：读出口2的寄存器编号（2位）
- W#：写入口Din的寄存器编号（2位）
- Din：写入的数据（32位）

- **输出：**

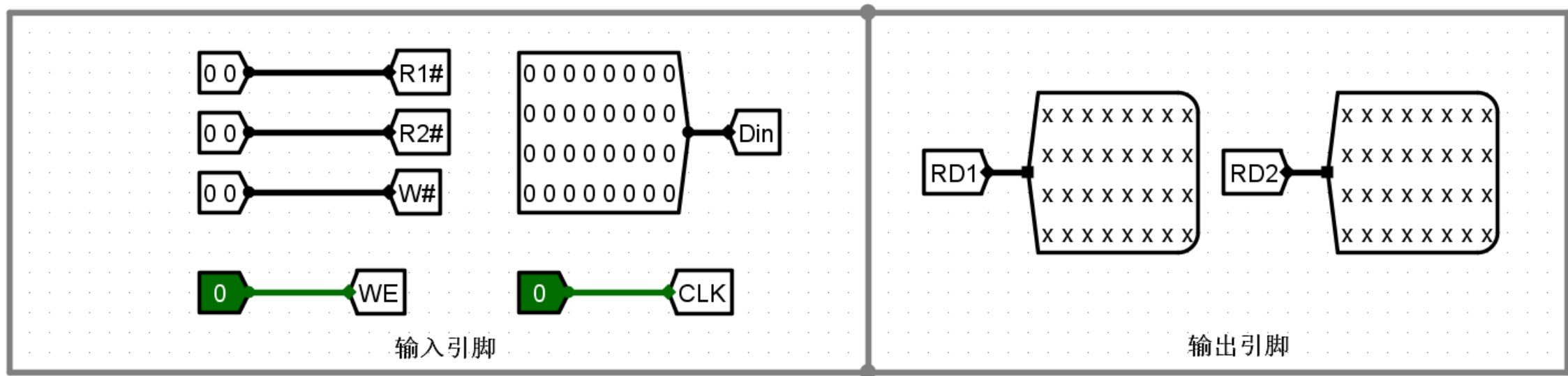
- RD1：读出口1的输出（32位）
- RD2：读出口2的输出（32位）

- **控制信号（输入）：**

- WE：读写控制信号；WE=1，写入；WE=0，读出
- CLK：时钟信号



寄存器文件电路（4个寄存器，R0=0）的输入、输出和控制信号



请同学们在此设计电路！

寄存器文件电路（4个寄存器，R0=0）

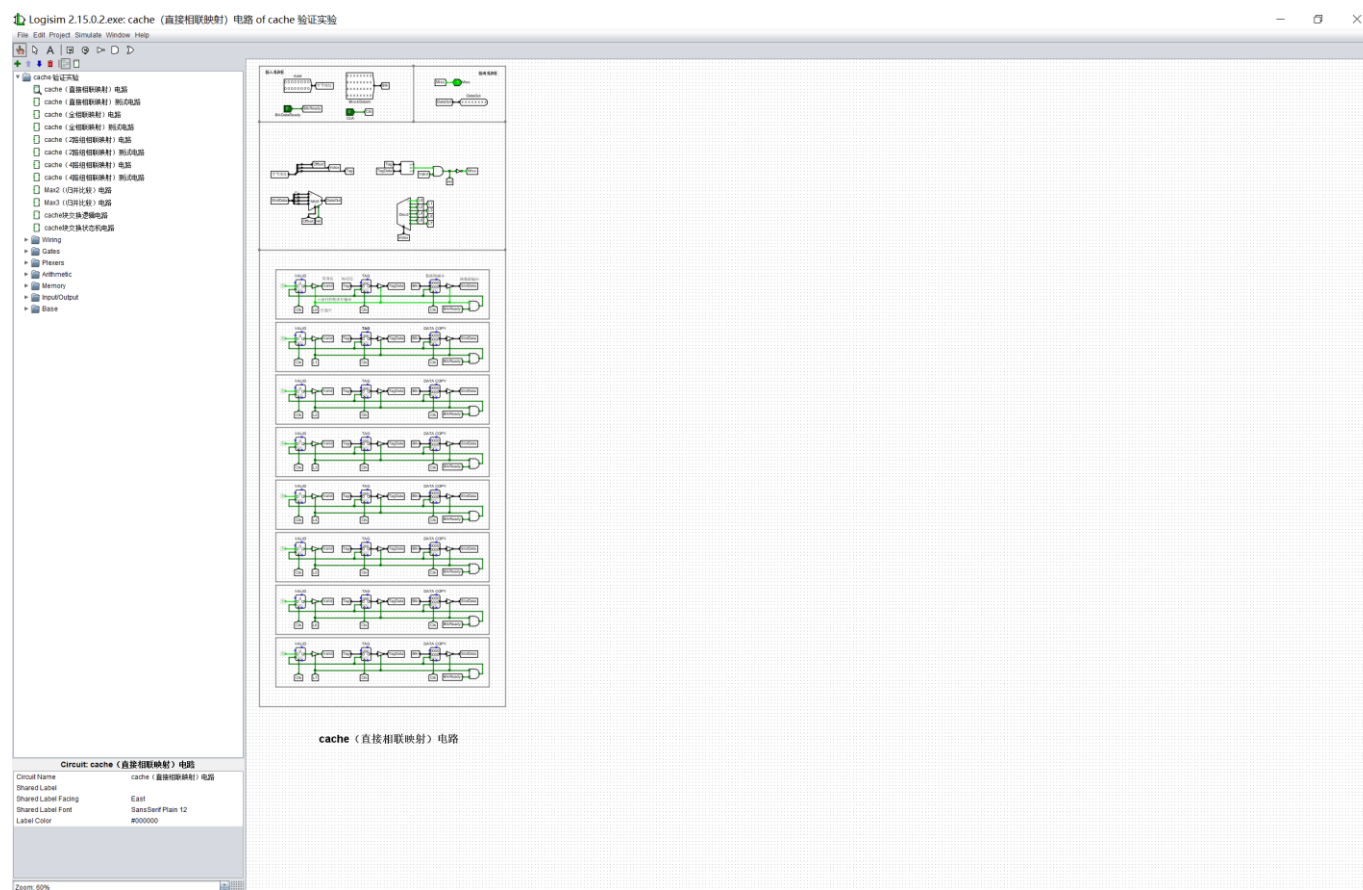
- 请同学们参考前面的简单的 MIPS 寄存器文件测试电路，设计一个测试电路。
- 然后，对设计好的寄存器文件电路（4个寄存器，R0=0）的功能进行测试，包括：
 - ① 向不同的寄存器中写入内容；
 - ② 分别从两个不同的端口读出第①步写入的内容；
 - ③ 验证R0是否恒等于0。

参考设计文件： [cache 验证实验.circ](#)

六、cache 验证实验

- 1、直接相联映射方式 cache 电路
- 2、直接相联映射方式 cache 测试电路
- 3、全相联映射方式 cache 电路
- 4、全相联映射方式 cache 测试电路
- 5、2路组相联映射方式 cache 电路
- 6、2路组相联映射方式 cache 测试电路
- 7、4路组相联映射方式 cache 电路
- 8、4路组相联映射方式 cache 测试电路

- 运行“Logisim”，在Logisim中打开“cache 验证实验.circ”。
- 也可以直接双击“cache 验证实验.circ”，打开该设计文件。



• 1、直接相联映射方式 cache 电路

- 直接相联映射方式：主存地址 = 区地址 (Tag) + 行索引 (Index) + 块内偏移 (Offset) 。
- 假设主存地址=16位；其中，区地址=11位，行索引=3位，块内偏移=2位；即cache有8行 (块) ($2^3=8$)，每行存放4个字节 ($2^2=4$) 。

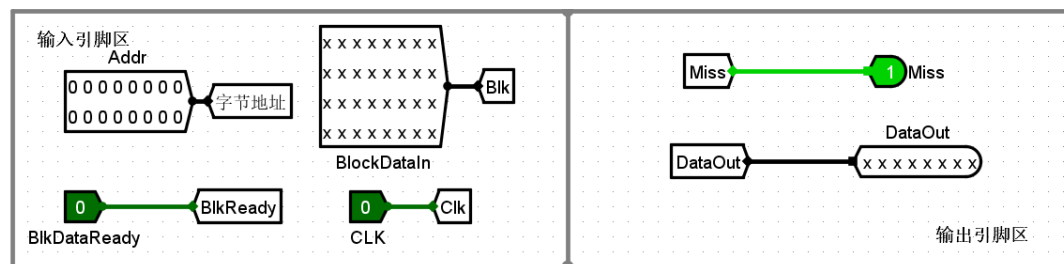
• 直接相联映射方式 cache 电路的输入和输出：

• 输入：

- 主存地址 (Addr, 字节地址)：16位
- 装入cache的数据块 (Blk)：因为cache每行存放4个字节，因此数据块大小=32位
- 数据块准备好信号 (BlkReady, 1位)
- 时钟信号 (CLK)

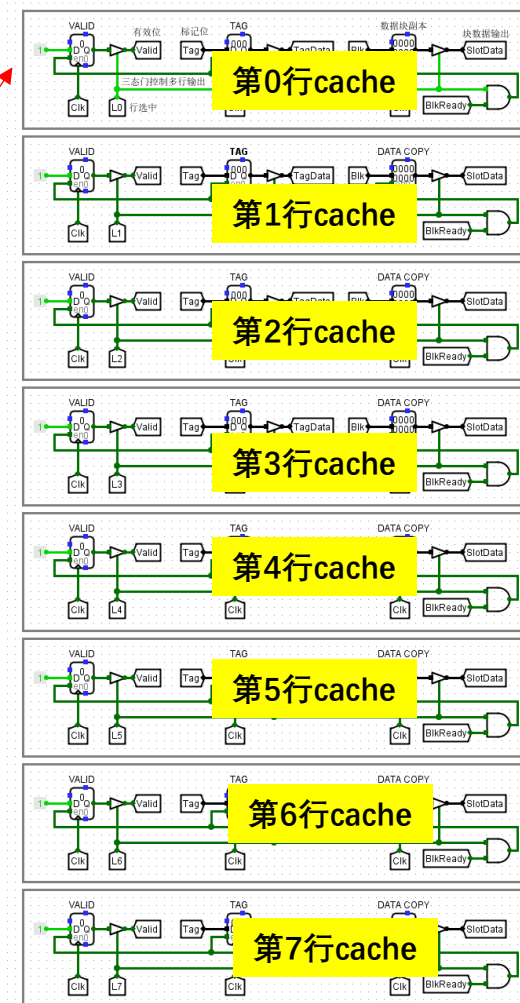
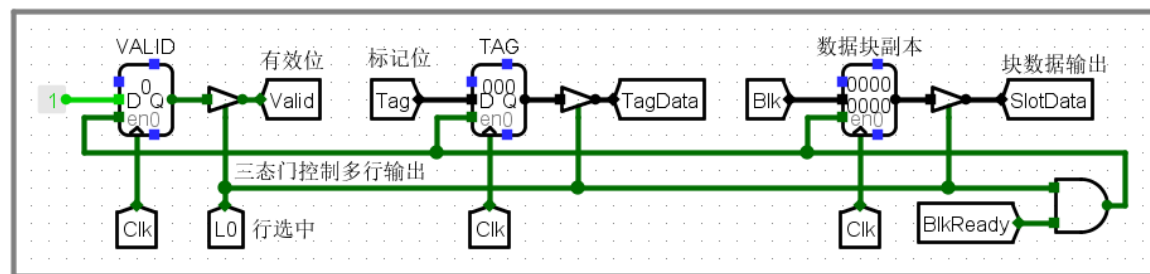
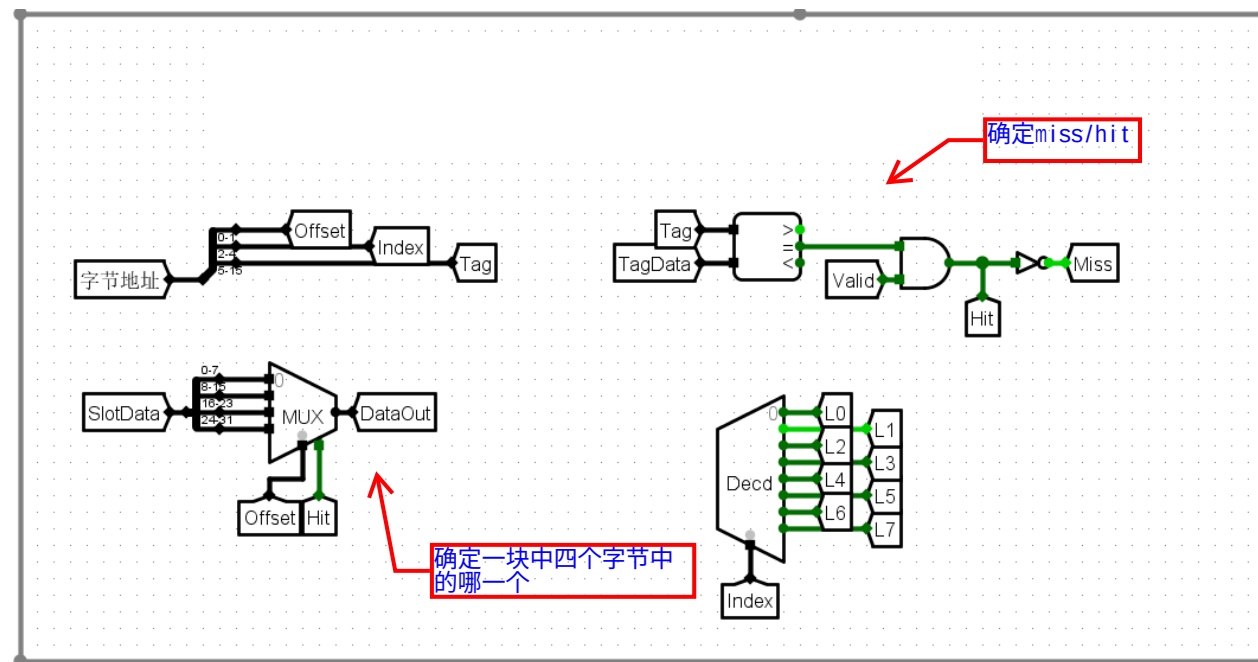
• 输出：

- 从cache中读出的数据 (DataOut)：1个字节，8位
- 是否命中 (Miss)：Miss=0，命中；Miss=1，缺失



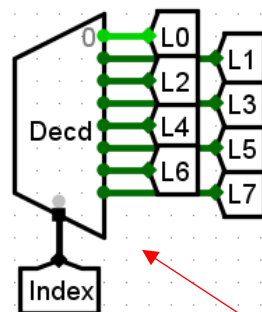
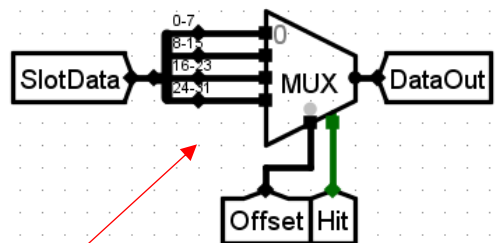
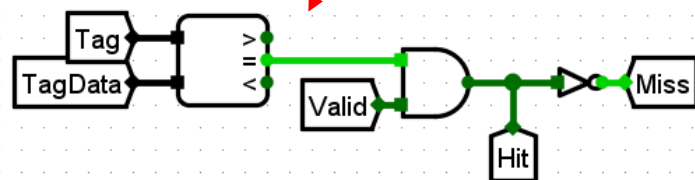
直接相联映射方式 cache 电路的输入和输出

直接相联映射方式 cache 电路



主存地址 (16位) = 区地址 (Tag) (11位) + 行索引 (Index) (3位) + 块内偏移 (Offset) (2位)

主存地址中的区地址 (Tag) 和cache中的标志位 (TagData) 进行比较, 如果相等, 并且有效位 (Valid) =1, 则命中 (Hit), 否则, 不命中 (Miss)



如果命中 (Hit=1), 则将cache中的数据块副本 (SlotData, 共32位) 中的某个字节送DataOut (由主存的块内偏移 (Offset) 决定是哪个字节)

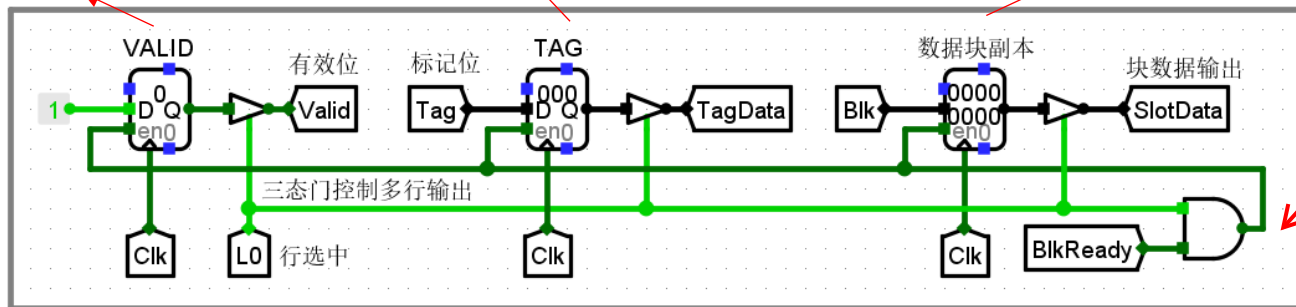
根据行索引 (Index) (3位) 选择8行cache中的某一行

1位寄存器 (存放有效位)

11位寄存器 (存放区地址tag)

32位寄存器 (存放数据块副本)

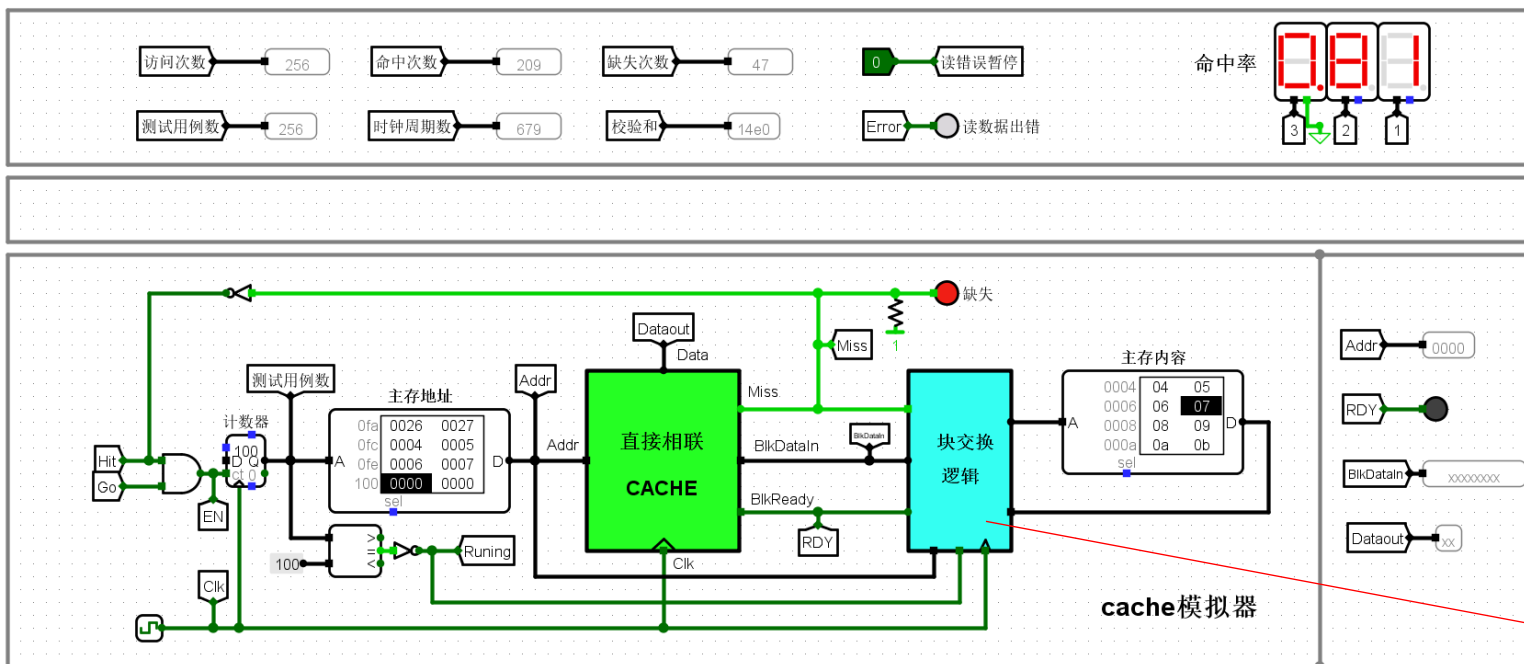
第0行cache



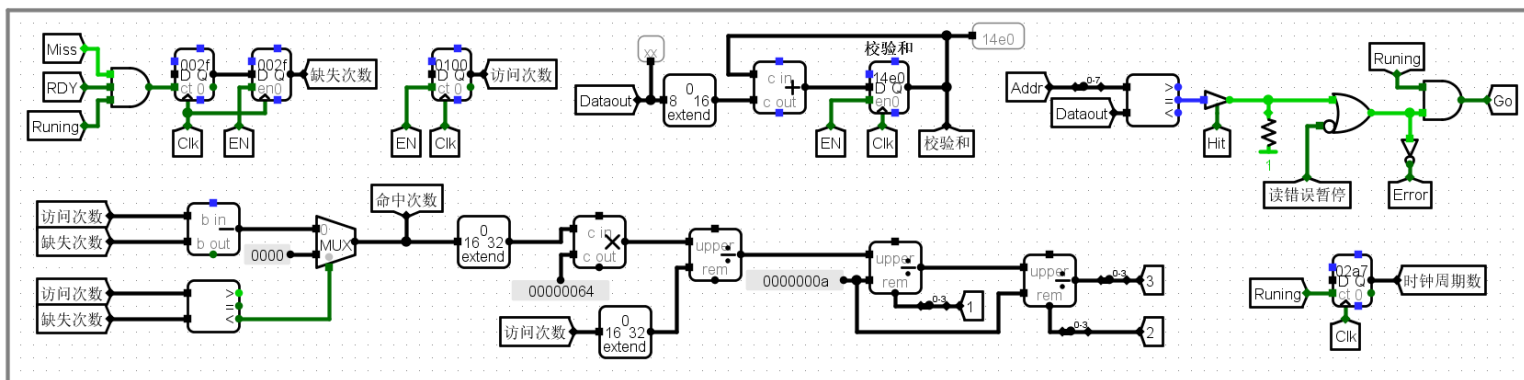
该行被选中并且 blkready=1, 此位与块交换逻辑有关, 表示miss后从主存调入块已准备好, 会触发各个寄存器值的修改

• 2、直接相联映射方式 cache 测试电路

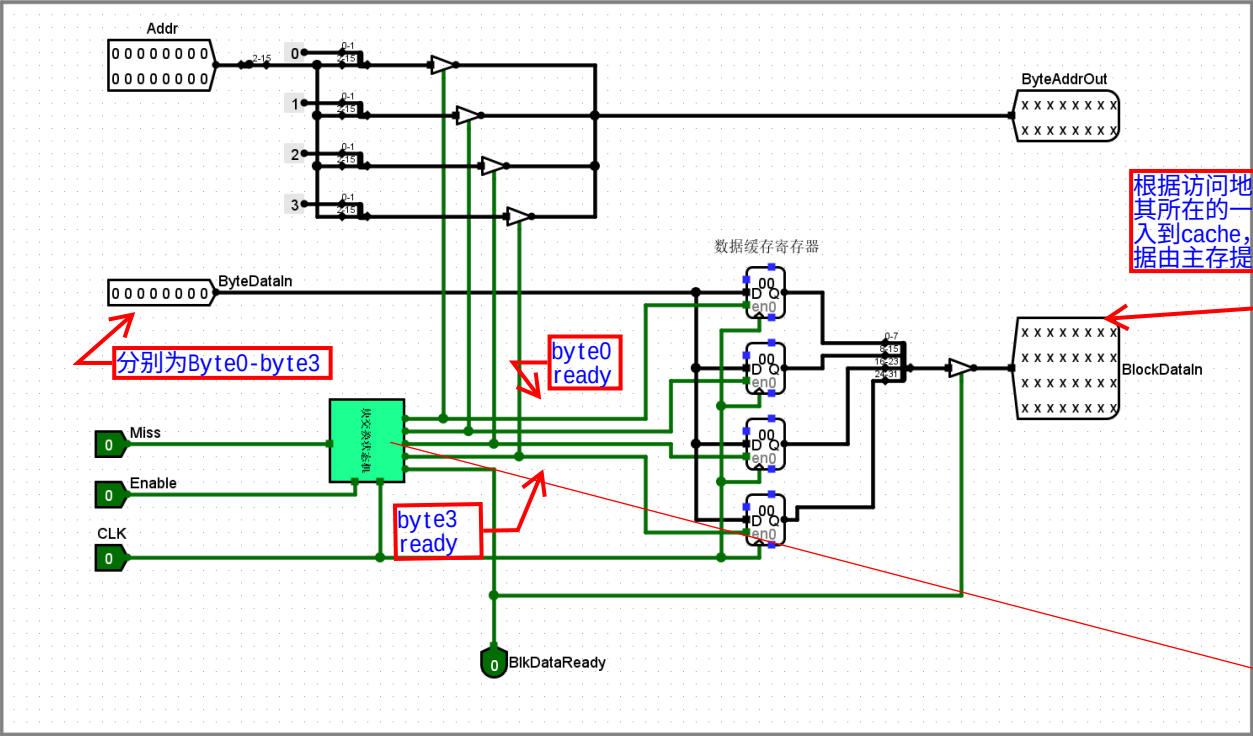
- 测试步骤：先按Ctrl+R，使电路复位；然后按Ctrl+K，启动时钟；等到命中率数字不变（计数器=100），表示测试完成。



具体电路见下一页



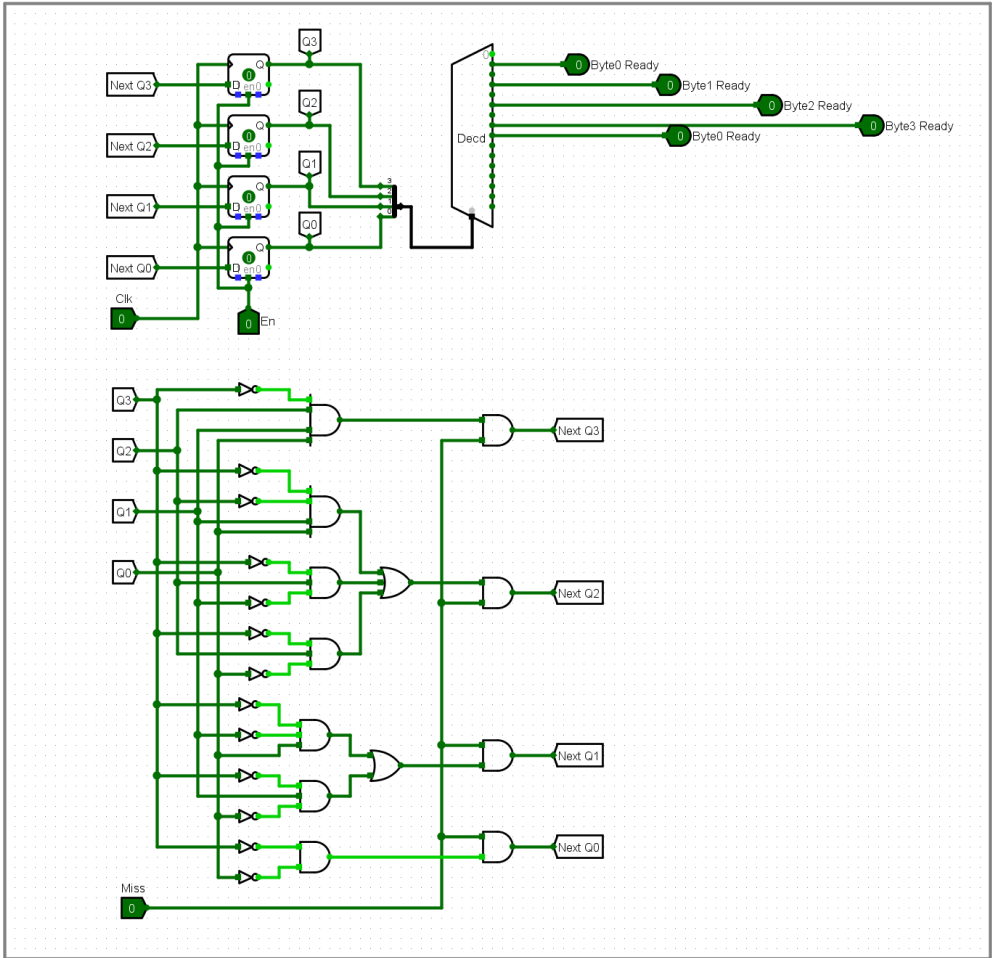
cache块交换逻辑电路



cache发生Miss,则需从主存提取所在的一个字块写入cache

根据访问地址去主存将其所在的一块数据都写入到cache,此时访问数据由主存提供

cache块交换状态机电路



cache块交换状态机电路

- 前32次访问情况分析（访问32次，命中25次，不命中（缺失）7次）：
 - (1) 访问 0 1 2 3
 - 访问地址为 0 存储单元，不命中，从主存中装入 0 1 2 3 四个单元，之后访问 1 2 3 存储单元，全部命中；命中3次、不命中1次
 - (2) 访问 8 9 a b
 - 访问地址为 8 存储单元，不命中，从主存中装入 8 9 a b 四个单元，之后访问 9 a b 存储单元，全部命中；命中3次、不命中1次
 - (3) 访问 10 11 12 13
 - 访问地址为 10 存储单元，不命中，从主存中装入 10 11 12 13 四个单元，之后访问 11 12 13 存储单元，全部命中；命中3次、不命中1次
 - (4) 访问 20 21 22 23
 - 访问地址为 20 存储单元，不命中，从主存中装入 20 21 22 23 四个单元，之后访问 21 22 23 存储单元，全部命中；命中3次、不命中1次
 - (5) 访问 8 9 a b
 - 访问地址为 8 存储单元，命中（第（2）步已经将8 9 a b装入cache中了），之后访问 9 a b 存储单元，全部命中；命中4次、不命中0次
 - (6) 访问 4 5 6 7
 - 访问地址为 4 存储单元，不命中，从主存中装入 4 5 6 7 四个单元，之后访问 5 6 7 存储单元，全部命中；命中3次、不命中1次
 - (7) 访问 24 25 26 27
 - 访问地址为 24 存储单元，不命中，从主存中装入 24 25 26 27 四个单元，之后访问 25 26 27 存储单元，全部命中；命中3次、不命中1次
 - (8) 访问 4 5 6 7
 - 访问地址为 4 存储单元，不命中；原因是：第（6）步将4 5 6 7装入cache中的第1行，但是第（7）步又将24 25 26 27装入cache中的第1行，这样导致不命中；从主存中装入 4 5 6 7 四个单元，之后访问 5 6 7 存储单元，全部命中；命中3次、不命中1次

cacache (直接相联映射) 测试用例——主存地址.txt

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw

0 1 2 3 8 9 a b

10 11 12 13 20 21 22 23

8 9 a b 4 5 6 7

24 25 26 27 4 5 6 7

0 1 2 3 8 9 a b

8 9 a b 20 21 22 23

28 29 2a 2b 24 25 26 27

4 5 6 7 24 25 26 27

0 1 2 3 8 9 a b

20 21 22 23 20 21 22 23

30 31 32 33 4 5 6 7

24 25 26 27 4 5 6 7

0 1 2 3 8 9 a b

20 21 22 23 20 21 22 23

28 29 2a 2b 4 5 6 7

24 25 26 27 24 25 26 27

0 1 2 3 8 9 a b

8 9 a b 20 21 22 23

30 31 32 33 4 5 6 7

24 25 26 27 24 25 26 27

0 1 2 3 0 1 2 3

8 9 a b 20 21 22 23

28 29 2a 2b 4 5 6 7

24 25 26 27 4 5 6 7

0 1 2 3 8 9 a b

0 1 2 3 20 21 22 23

30 31 32 33 24 25 26 27

4 5 6 7 24 25 26 27

主存地址=4=0000 0000 0000 0100=000000000000 001 00 行索引=1

主存地址=24=0000 0000 0010 0100=000000000001 001 00 行索引=1

即4 5 6 7要装入cache的第1行，24 25 26 27也要装入cache的第1行

cacache (直接相联映射) 测试用例——主存内容.txt

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw

0 1 2 3 4 5 6 7

8 9 a b c d e f

10 11 12 13 14 15 16 17

18 19 1a 1b 1c 1d 1e 1f

20 21 22 23 24 25 26 27

28 29 2a 2b 2c 2d 2e 2f

30 31 32 33 34 35 36 37

38 39 3a 3b 3c 3d 3e 3f

40 41 42 43 44 45 46 47

48 49 4a 4b 4c 4d 4e 4f

50 51 52 53 54 55 56 57

58 59 5a 5b 5c 5d 5e 5f

60 61 62 63 64 65 66 67

68 69 6a 6b 6c 6d 6e 6f

70 71 72 73 74 75 76 77

78 79 7a 7b 7c 7d 7e 7f

80 81 82 83 84 85 86 87

88 89 8a 8b 8c 8d 8e 8f

90 91 92 93 94 95 96 97

98 99 9a 9b 9c 9d 9e 9f

a0 a1 a2 a3 a4 a5 a6 a7

a8 a9 aa ab ac ad ae af

b0 b1 b2 b3 b4 b5 b6 b7

b8 b9 ba bb bc bd be bf

c0 c1 c2 c3 c4 c5 c6 c7

c8 c9 ca cb cc cd ce cf

d0 d1 d2 d3 d4 d5 d6 d7

d8 d9 da db dc dd de df

e0 e1 e2 e3 e4 e5 e6 e7

e8 e9 ea eb ec ed ee ef

f0 f1 f2 f3 f4 f5 f6 f7

f8 f9 fa fb fc fd fe ff

0 1 2 3 4 5 6 7

8 9 a b c d e f

10 11 12 13 14 15 16 17

18 19 1a 1b 1c 1d 1e 1f

20 21 22 23 24 25 26 27

28 29 2a 2b 2c 2d 2e 2f

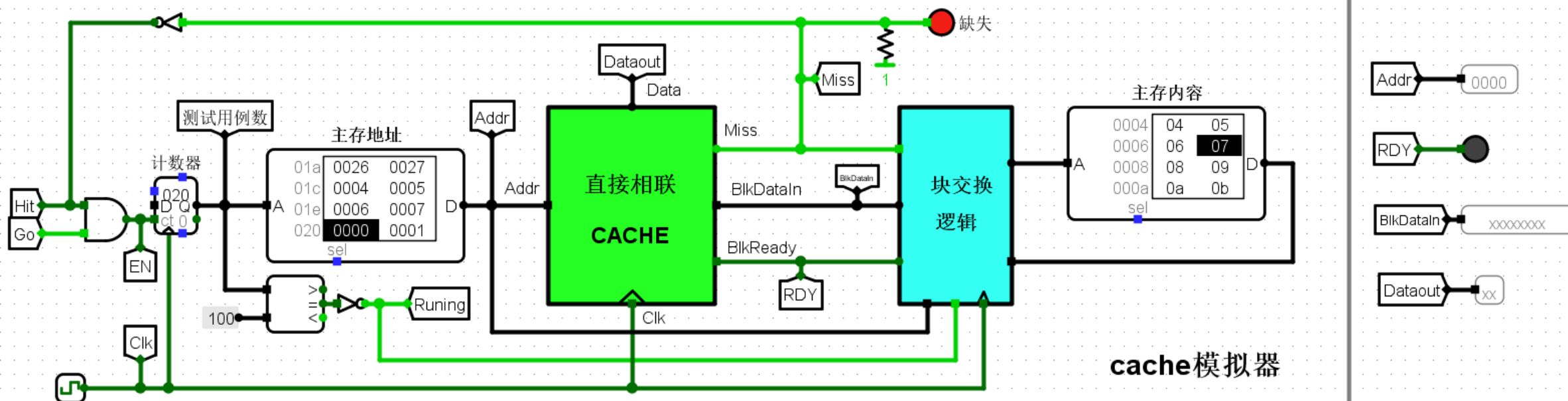
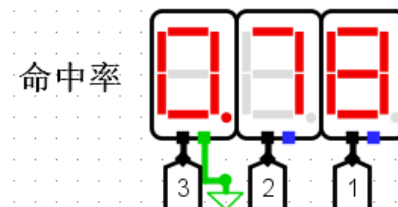
30 31 32 33 34 35 36 37

38 39 3a 3b 3c 3d 3e 3f

40 41 42 43 44 45 46 47

访问32次，命中25次，不命中（缺失）7次

命中率=25/32=78.125%



访问256次，命中209次，不命中（缺失）47次

命中率=209/256=81.64%

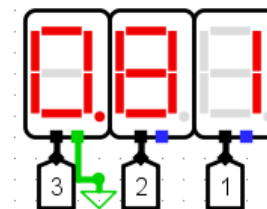
访问次数 256

命中次数 209

缺失次数 47

0 读错误暂停

命中率

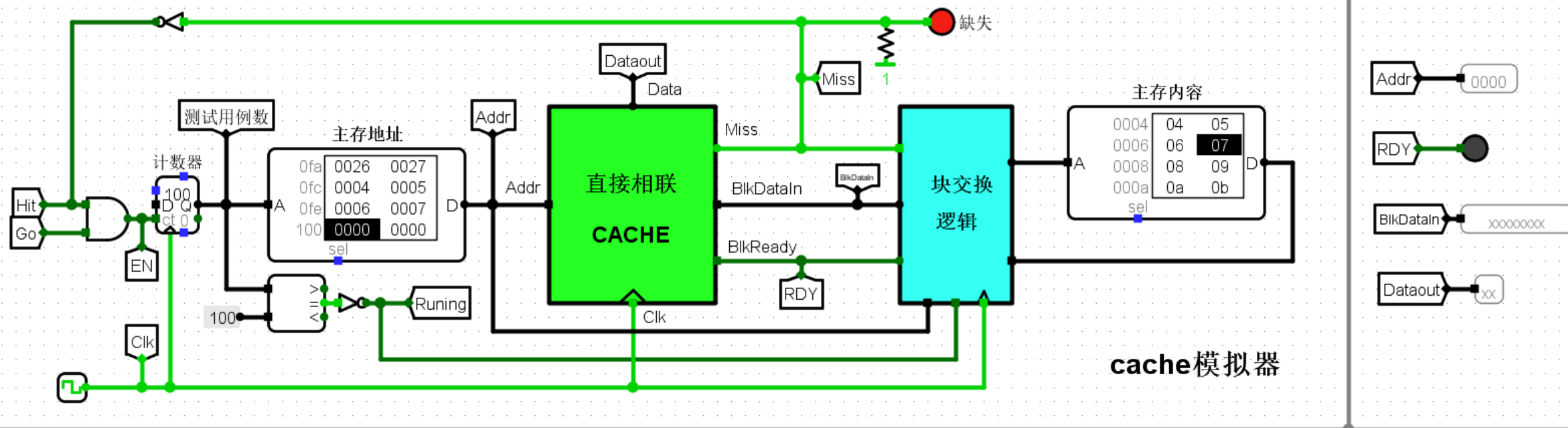


测试用例数 256

时钟周期数 679

校验和 14e0

Error 读数据出错



v2.0 raw

0 1 2 3 8 9 a b
10 11 12 13 20 21 22 23
8 9 a b 4 5 6 7
24 25 26 27 4 5 6 7

0 1 2 3 8 9 a b
8 9 a b 20 21 22 23
28 29 2a 2b 24 25 26 27
4 5 6 7 24 25 26 27

0 1 2 3 8 9 a b
20 21 22 23 20 21 22 23
30 31 32 33 4 5 6 7
24 25 26 27 4 5 6 7

0 1 2 3 8 9 a b
20 21 22 23 20 21 22 23
28 29 2a 2b 4 5 6 7
24 25 26 27 24 25 26 27

0 1 2 3 8 9 a b
8 9 a b 20 21 22 23
30 31 32 33 4 5 6 7
24 25 26 27 24 25 26 27

0 1 2 3 0 1 2 3
8 9 a b 20 21 22 23
28 29 2a 2b 4 5 6 7
24 25 26 27 4 5 6 7

0 1 2 3 8 9 a b
0 1 2 3 20 21 22 23
30 31 32 33 24 25 26 27
4 5 6 7 24 25 26 27

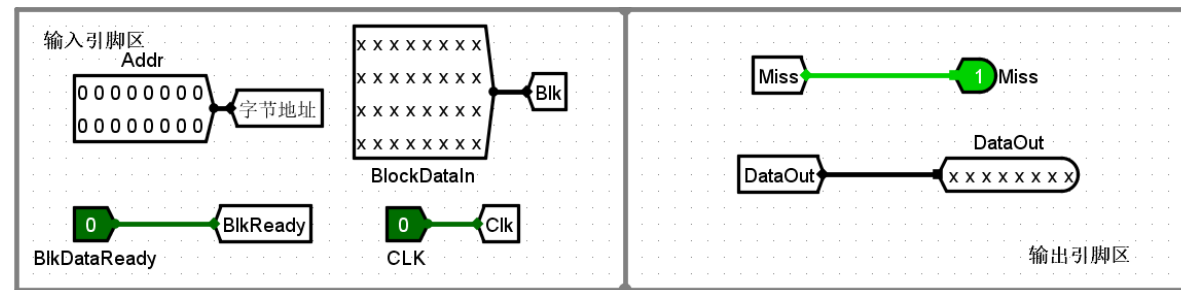
0 1 2 3 8 9 a b
20 21 22 23 20 21 22 23
28 29 2a 2b 4 5 6 7
24 25 26 27 4 5 6 7

- 请同学们分析“直接相联映射方式 cache 电路”的原理。
- 请同学们分析第33～第64次cache访问的情况：命中多少次？不命中（缺失）多少次？
- 有兴趣的同学，请分析“cache 测试电路”的原理。

• 3、全相联映射方式 cache 电路

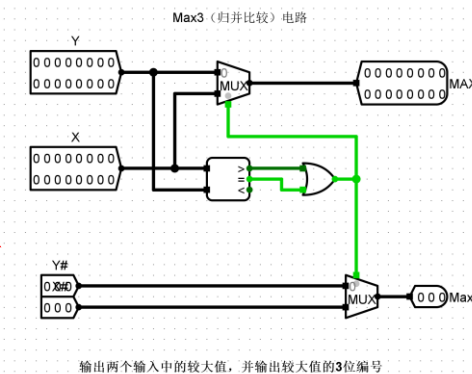
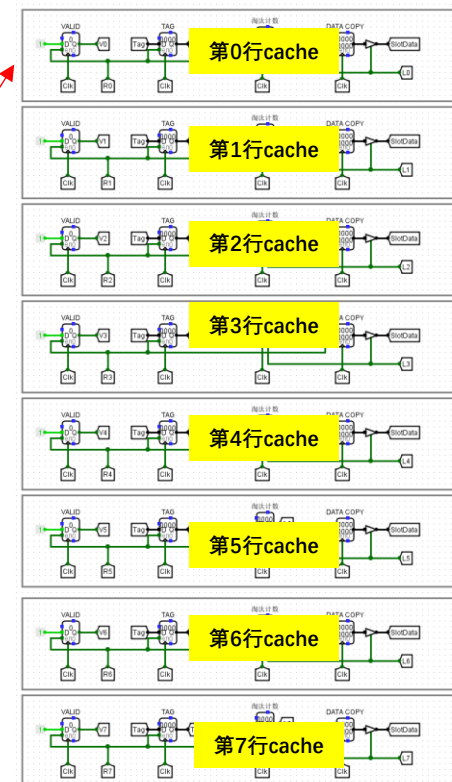
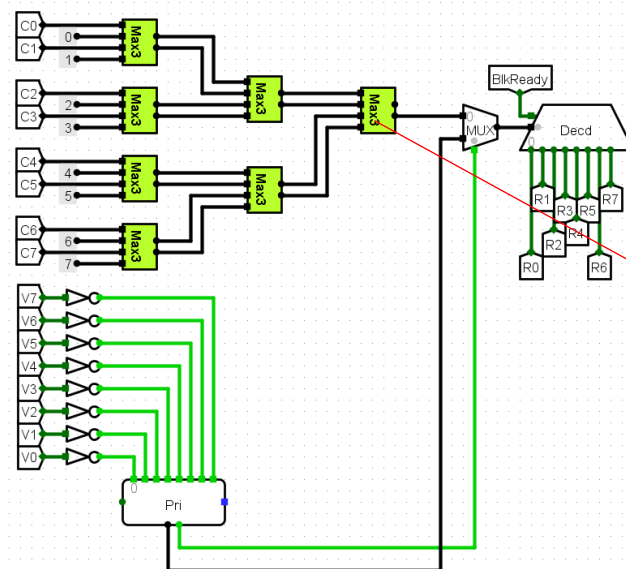
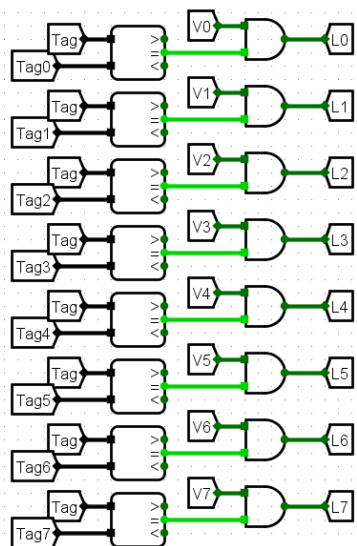
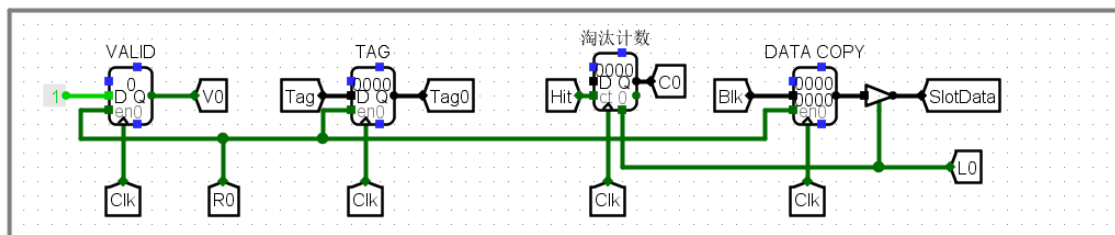
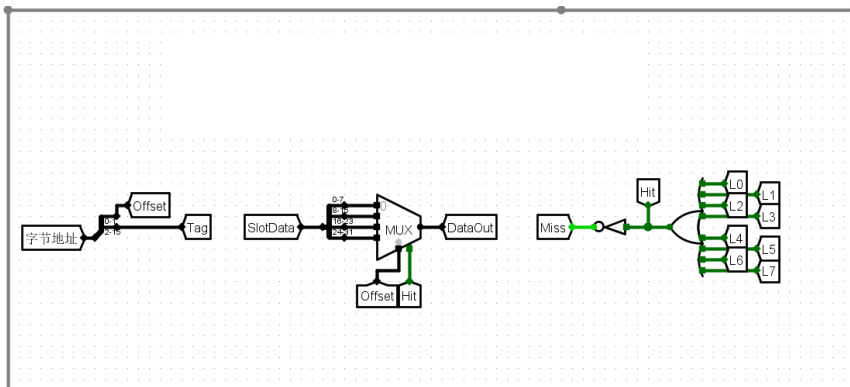
- 全相联映射方式：主存地址 = 主存块地址（Tag）+ 块内偏移（Offset）。
- 假设主存地址=16位；其中，主存块地址=14位，块内偏移=2位。
- cache有8行（块），每行存放4个字节（ $2^2=4$ ）。
- 全相联映射方式 cache 电路的输入和输出：
 - 输入：
 - 主存地址（Addr，字节地址）：16位
 - 装入cache的数据块（Blk）：因为cache每行存放4个字节，因此数据块大小=32位
 - 数据块准备好信号（BlkReady，1位）
 - 时钟信号（CLK）

- 输出：
 - 从cache中得到的数据（DataOut）：1个字节，8位
 - 是否命中（Miss）：Miss=1，缺失；Miss=0，命中



全相联映射方式 cache 电路的输入和输出

全相联映射方式 cache 电路



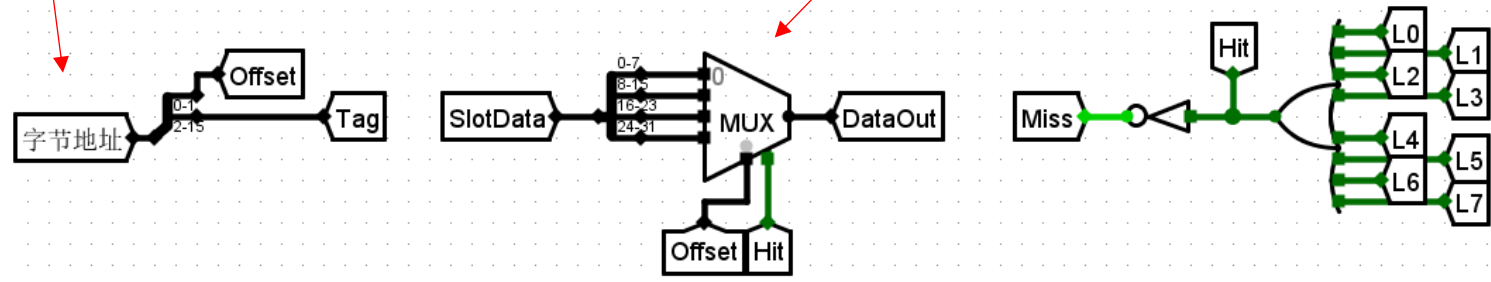
输出两个输入中的较大值，并输出较大值的3位编号

Max3（归并比较）电路

主存地址 (16位) = 主存块地址 (Tag) (14位) + 块内偏移 (Offset) (2位)

如果命中 (Hit=1), 则将
cache中的数据块副本
(SlotData, 共32位) 中的某个
字节送DataOut (由主存的块内
偏移 (Offset) 决定是哪个字节)

如果主存的块地址 (Tag) 与8个cache块中的
主存块地址 (Tag0 ~ Tag7) 中的任一个
比较相等, 并且对应的有效位 (V0 ~ V7)
=1, 则对应的L0 ~ L7输出高电平, 命中
(Hit), 否则, 不命中 (Miss)

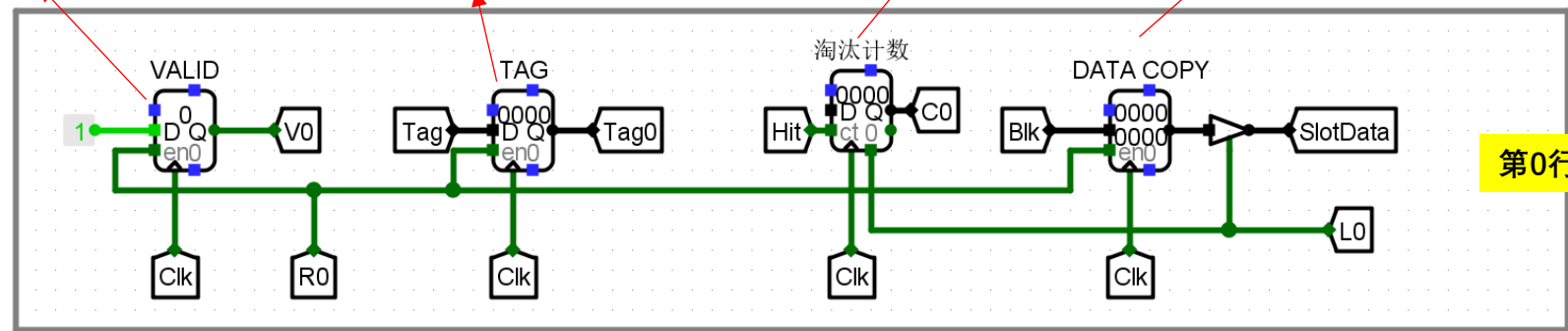


1位寄存器
存放有效位

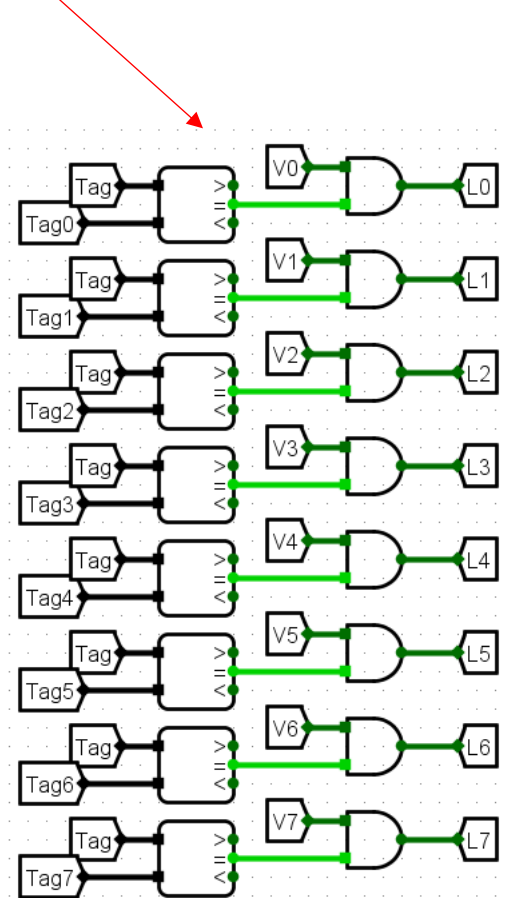
14位寄存器
存放块地址Tag

16位计数器
用于LRU替换算法的计数

32位寄存器
存放数据块副本



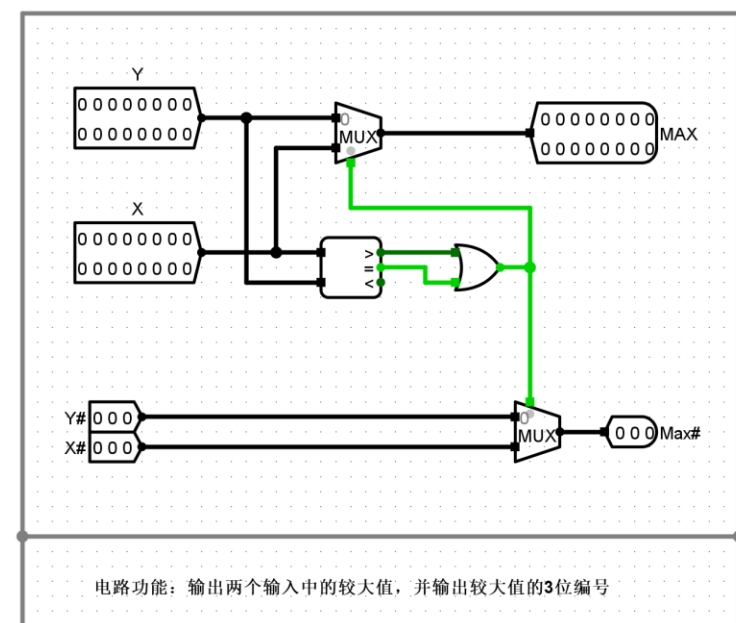
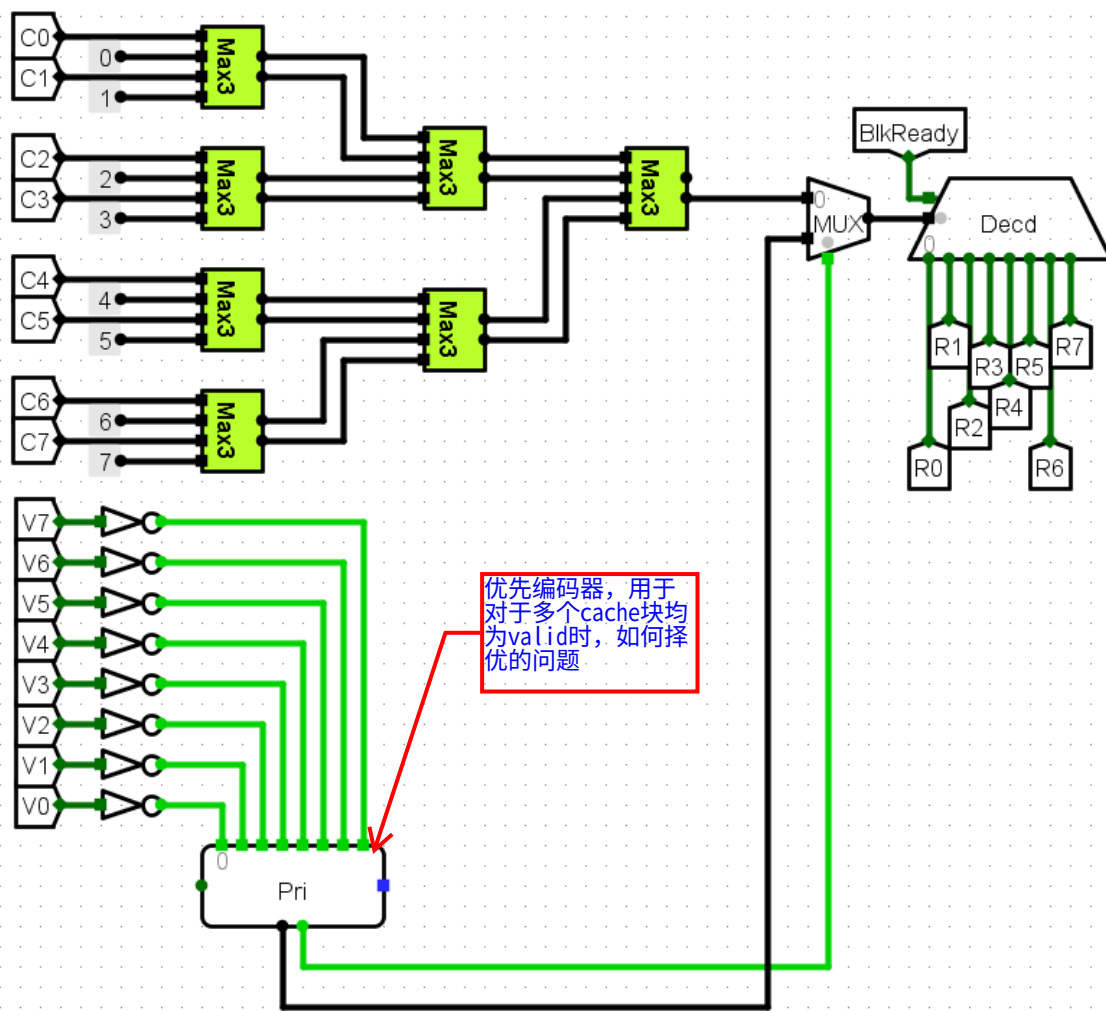
第0行cache



该电路用于实现LRU（近期最少使用）替换算法

输入：C0 ~ C7, V0 ~ V7, BlkReady

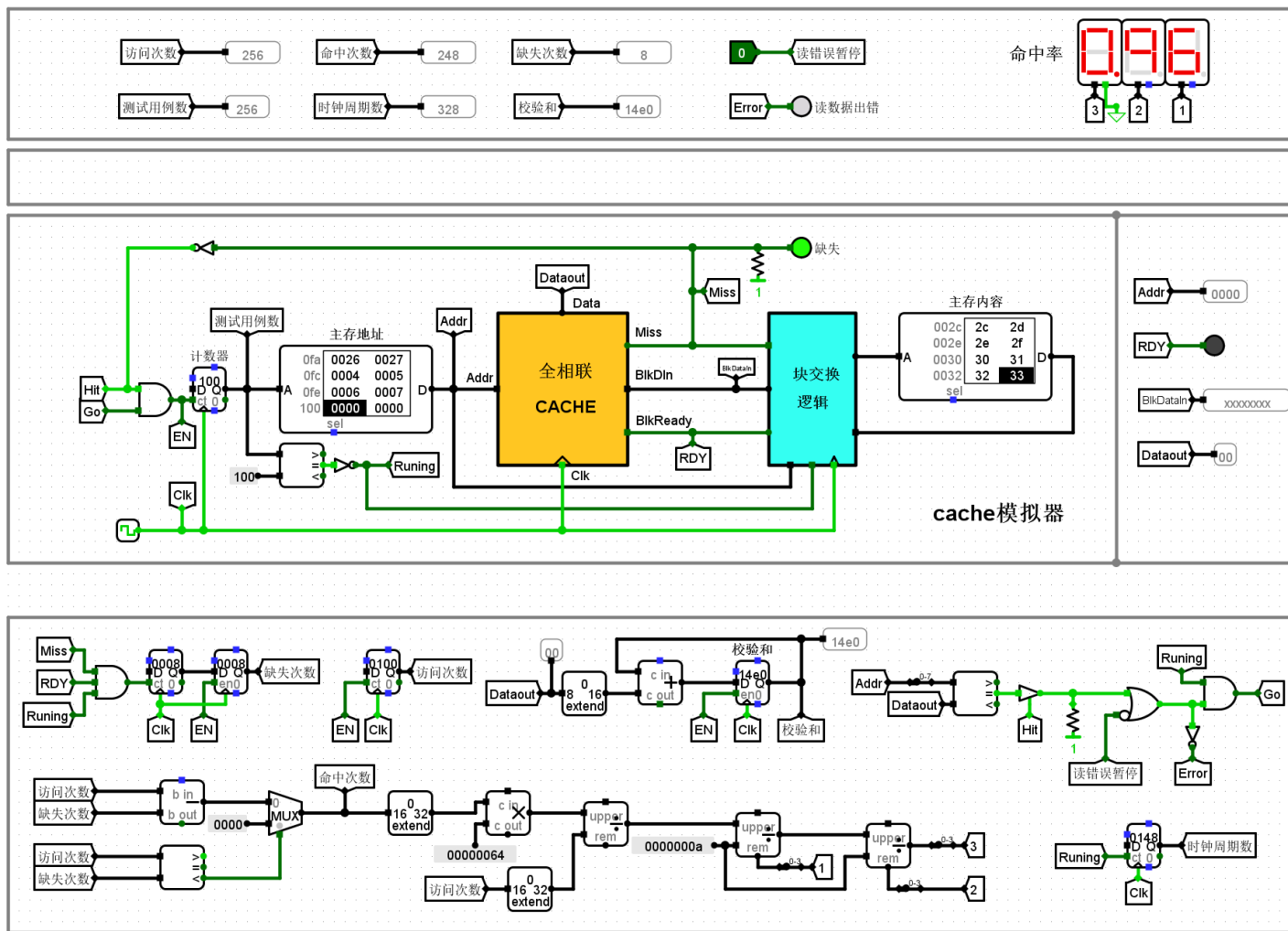
输出：R0 ~ R7



Max3（归并比较）电路

• 4、全相联映射方式 cache 测试电路（与直接相联映射方式的cache测试电路相同）

- 测试步骤：先按Ctrl+R，使电路复位；然后按Ctrl+K，启动时钟；等到命中率数字不变（计数器=100），表示表示测试完成。



- 前32次访问情况分析（访问32次，命中26次，不命中（缺失）6次）：
 - (1) 访问 0 1 2 3
 - 访问地址为 0 存储单元，不命中，从主存中装入 0 1 2 3 四个单元，之后访问 1 2 3 存储单元，全部命中；命中3次、不命中1次
 - (2) 访问 8 9 a b
 - 访问地址为 8 存储单元，不命中，从主存中装入 8 9 a b 四个单元，之后访问 9 a b 存储单元，全部命中；命中3次、不命中1次
 - (3) 访问 10 11 12 13
 - 访问地址为 10 存储单元，不命中，从主存中装入 10 11 12 13 四个单元，之后访问 11 12 13 存储单元，全部命中；命中3次、不命中1次
 - (4) 访问 20 21 22 23
 - 访问地址为 20 存储单元，不命中，从主存中装入 20 21 22 23 四个单元，之后访问 21 22 23 存储单元，全部命中；命中3次、不命中1次
 - (5) 访问 8 9 a b
 - 访问地址为 8 存储单元，命中（第（2）步已经将8 9 a b装入cache中了），之后访问 9 a b 存储单元，全部命中；命中4次、不命中0次
 - (6) 访问 4 5 6 7
 - 访问地址为 4 存储单元，不命中，从主存中装入 4 5 6 7 四个单元，之后访问 5 6 7 存储单元，全部命中；命中3次、不命中1次
 - (7) 访问 24 25 26 27
 - 访问地址为 24 存储单元，不命中，从主存中装入 24 25 26 27 四个单元，之后访问 25 26 27 存储单元，全部命中；命中3次、不命中1次
 - (8) 访问 4 5 6 7
 - 访问地址为 4 存储单元，命中（第（6）步已经将8 9 a b装入cache中了），之后访问 5 6 7 存储单元，全部命中；命中4次、不命中0次

cacache（全相联映射）测试用例——主存地址.txt

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw

0 1 2 3 8 9 a b

10 11 12 13 20 21 22 23

8 9 a b 4 5 6 7

24 25 26 27 4 5 6 7

0 1 2 3 8 9 a b

8 9 a b 20 21 22 23

28 29 2a 2b 24 25 26 27

4 5 6 7 24 25 26 27

0 1 2 3 8 9 a b

20 21 22 23 20 21 22 23

30 31 32 33 4 5 6 7

24 25 26 27 4 5 6 7

0 1 2 3 8 9 a b

20 21 22 23 20 21 22 23

28 29 2a 2b 4 5 6 7

24 25 26 27 24 25 26 27

0 1 2 3 8 9 a b

8 9 a b 20 21 22 23

30 31 32 33 4 5 6 7

24 25 26 27 24 25 26 27

0 1 2 3 0 1 2 3

8 9 a b 20 21 22 23

28 29 2a 2b 4 5 6 7

24 25 26 27 4 5 6 7

0 1 2 3 8 9 a b

0 1 2 3 20 21 22 23

30 31 32 33 24 25 26 27

4 5 6 7 24 25 26 27

0 1 2 3 8 9 a b

20 21 22 23 20 21 22 23

28 29 2a 2b 4 5 6 7

24 25 26 27 4 5 6 7

共256个地址

与直接相联映射方式相同

cacache（全相联映射）测试用例——主存内容.txt

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw

0 1 2 3 4 5 6 7

8 9 a b c d e f

10 11 12 13 14 15 16 17

18 19 1a 1b 1c 1d 1e 1f

20 21 22 23 24 25 26 27

28 29 2a 2b 2c 2d 2e 2f

30 31 32 33 34 35 36 37

38 39 3a 3b 3c 3d 3e 3f

40 41 42 43 44 45 46 47

48 49 4a 4b 4c 4d 4e 4f

50 51 52 53 54 55 56 57

58 59 5a 5b 5c 5d 5e 5f

60 61 62 63 64 65 66 67

68 69 6a 6b 6c 6d 6e 6f

70 71 72 73 74 75 76 77

78 79 7a 7b 7c 7d 7e 7f

80 81 82 83 84 85 86 87

88 89 8a 8b 8c 8d 8e 8f

90 91 92 93 94 95 96 97

98 99 9a 9b 9c 9d 9e 9f

a0 a1 a2 a3 a4 a5 a6 a7

a8 a9 aa ab ac ad ae af

b0 b1 b2 b3 b4 b5 b6 b7

b8 b9 ba bb bc bd be bf

c0 c1 c2 c3 c4 c5 c6 c7

c8 c9 ca cb cc cd ce cf

d0 d1 d2 d3 d4 d5 d6 d7

d8 d9 da db dc dd de df

e0 e1 e2 e3 e4 e5 e6 e7

e8 e9 ea eb ec ed ee ef

f0 f1 f2 f3 f4 f5 f6 f7

f8 f9 fa fb fc fd fe ff

0 1 2 3 4 5 6 7

8 9 a b c d e f

10 11 12 13 14 15 16 17

18 19 1a 1b 1c 1d 1e 1f

20 21 22 23 24 25 26 27

28 29 2a 2b 2c 2d 2e 2f

30 31 32 33 34 35 36 37

38 39 3a 3b 3c 3d 3e 3f

40 41 42 43 44 45 46 47

48 49 4a 4b 4c 4d 4e 4f

50 51 52 53 54 55 56 57

58 59 5a 5b 5c 5d 5e 5f

60 61 62 63 64 65 66 67

68 69 6a 6b 6c 6d 6e 6f

70 71 72 73 74 75 76 77

访问32次，命中26次，不命中（缺失）6次

命中率=26/32=81.25%

访问次数 32

命中次数 26

缺失次数 6

0 读错误暂停

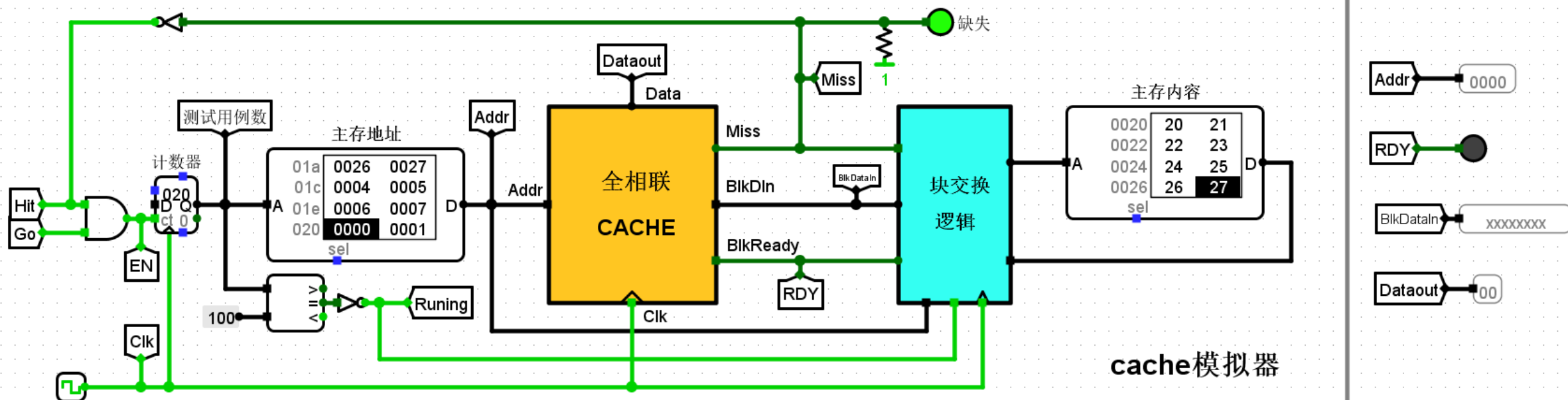
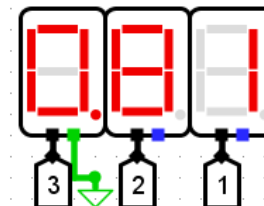
测试用例数 32

时钟周期数 86

校验和 01e0

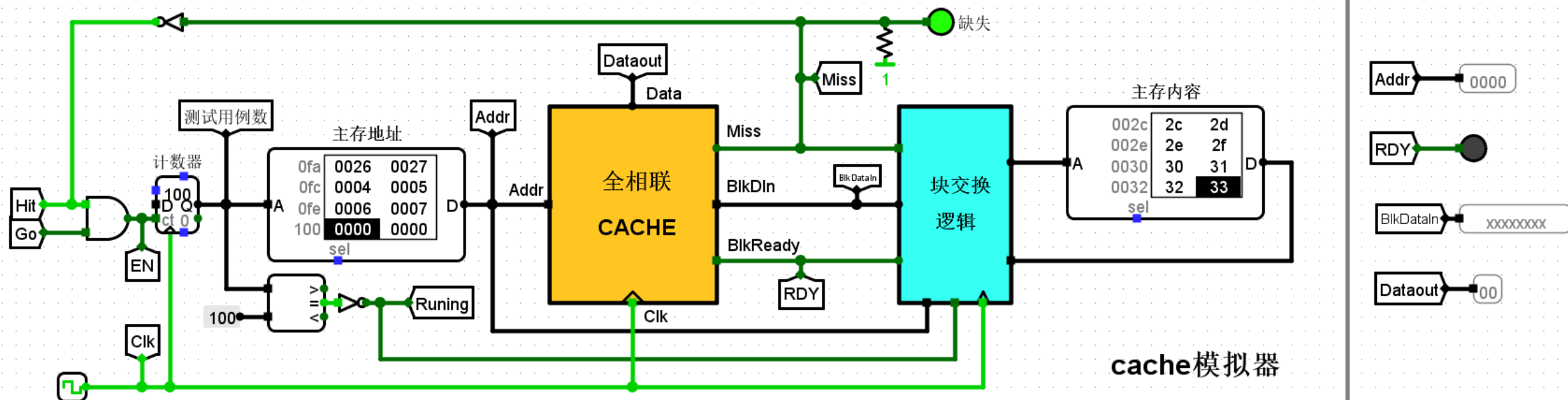
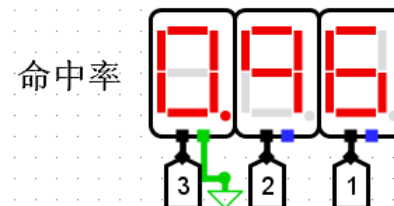
Error 读数据出错

命中率



访问256次，命中248次，不命中（缺失）8次

命中率=248/256=96.875%



- 请同学们分析“全相联映射方式 cache 电路”的原理。

- 请同学们分析第33～第64次cache访问的情况：命中多少次？不命中（缺失）多少次？

v2.0 raw

0 1 2 3 8 9 a b

10 11 12 13 20 21 22 23

8 9 a b 4 5 6 7

24 25 26 27 4 5 6 7

0 1 2 3 8 9 a b

8 9 a b 20 21 22 23

28 29 2a 2b 24 25 26 27

4 5 6 7 24 25 26 27

0 1 2 3 8 9 a b

20 21 22 23 20 21 22 23

30 31 32 33 4 5 6 7

24 25 26 27 4 5 6 7

0 1 2 3 8 9 a b

20 21 22 23 20 21 22 23

28 29 2a 2b 4 5 6 7

24 25 26 27 24 25 26 27

0 1 2 3 8 9 a b

8 9 a b 20 21 22 23

30 31 32 33 4 5 6 7

24 25 26 27 24 25 26 27

0 1 2 3 0 1 2 3

8 9 a b 20 21 22 23

28 29 2a 2b 4 5 6 7

24 25 26 27 4 5 6 7

0 1 2 3 8 9 a b

0 1 2 3 20 21 22 23

30 31 32 33 24 25 26 27

4 5 6 7 24 25 26 27

0 1 2 3 8 9 a b

20 21 22 23 20 21 22 23

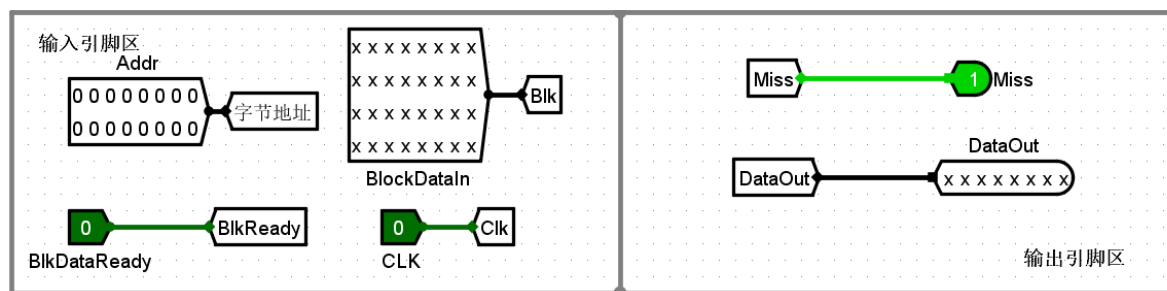
28 29 2a 2b 4 5 6 7

24 25 26 27 4 5 6 7

• 5、2路组相联映射方式 cache 电路

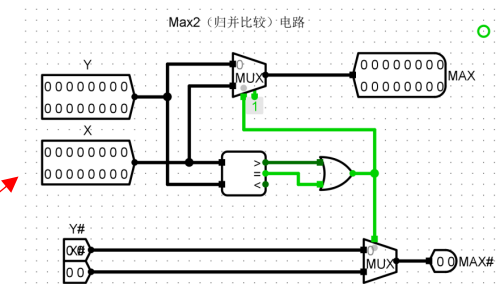
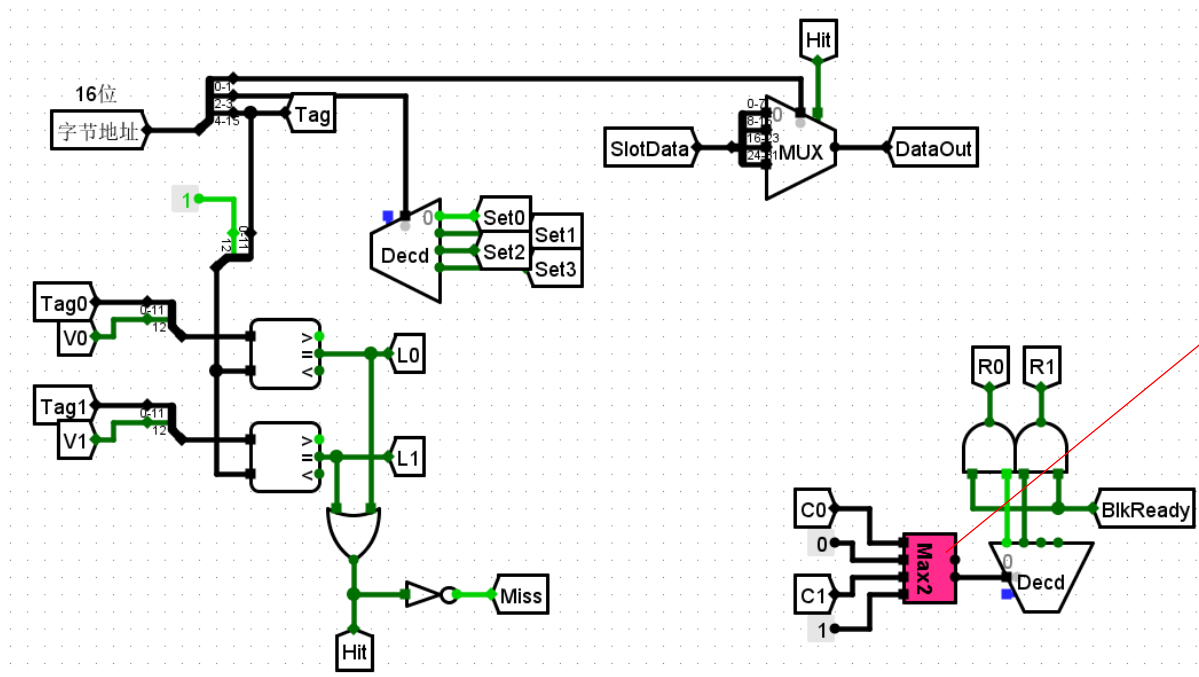
- 2路组相联映射方式：主存地址 = 标记（Tag）+ 组索引（Index）+ 块内偏移（Offset）。
- 假设主存地址=16位；其中，标记=12位，组索引=2位，块内偏移=2位；即cache有4组（ $2^2=4$ ）、每组2行。
- cache共有8行（块）（分为4个组、每组2行），每行存放4个字节（ $2^2=4$ ）。
- 2路组相联映射方式 cache 电路的输入和输出：
 - 输入：
 - 主存地址（Addr，字节地址）：16位
 - 装入cache的数据块（Blk）：因为cache每行存放4个字节，因此数据块大小=32位
 - 数据块准备好信号（BlkReady，1位）
 - 时钟信号（CLK）

- 输出：
 - 从cache中得到的数据（DataOut）：1个字节，8位
 - 是否命中（Miss）：Miss=1，缺失；Miss=0，命中

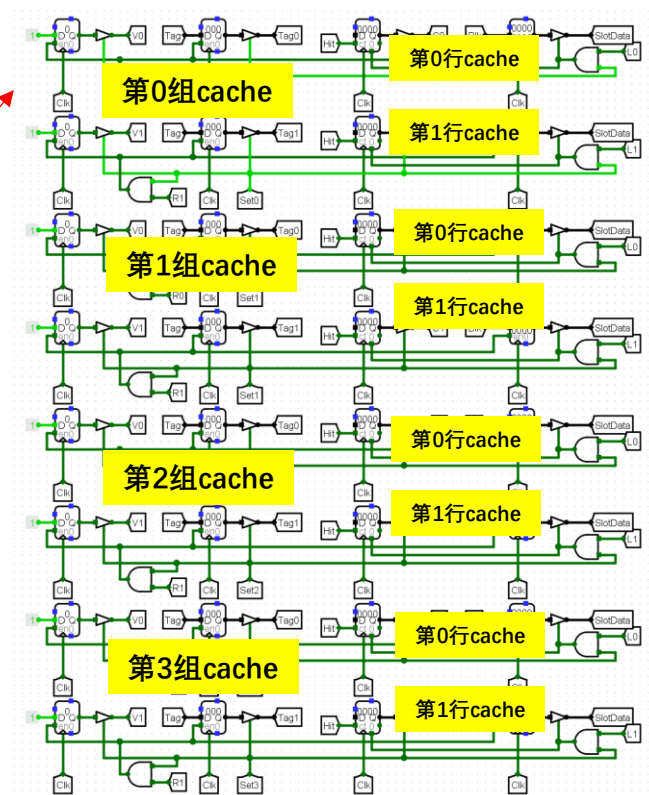
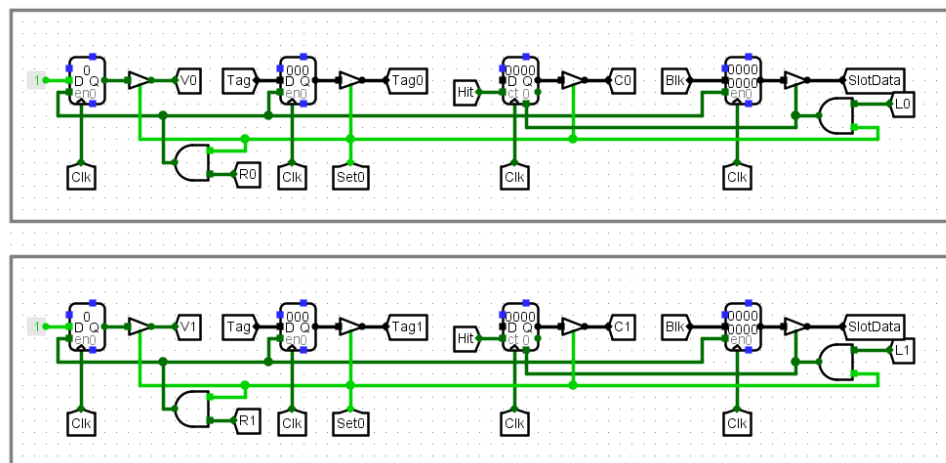


2路组相联映射方式 cache 电路的输入和输出

2路组相联映射方式cache电路



Max2 (归并比较) 电路

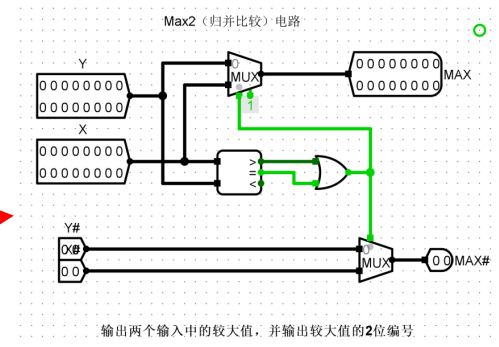


主存地址 (16位) = 标记 (Tag) (12位) + 组索引 (Index, 2位) + 块内偏移 (Offset) (2位)

如果命中 (Hit=1)，则将cache中的数据块副本 (SlotData, 共32位) 中的某个字节送DataOut (由主存的块内偏移 (Offset) 决定是哪个字节)

根据组索引 (Index) (2位) 选择4组cache中的某一组

这个电路用于LRU替换算法



将主存的标记 (Tag) 与 cache组中的2个块的标记 (Tag0、Tag1) 进行比较，如果有一个比较相等，并且对应的有效位 (V0、V1) =1，则对应的L0、L1输出高电平，命中 (Hit)，否则，不命中 (Miss)

12位寄存器 (存放标记, Tag)

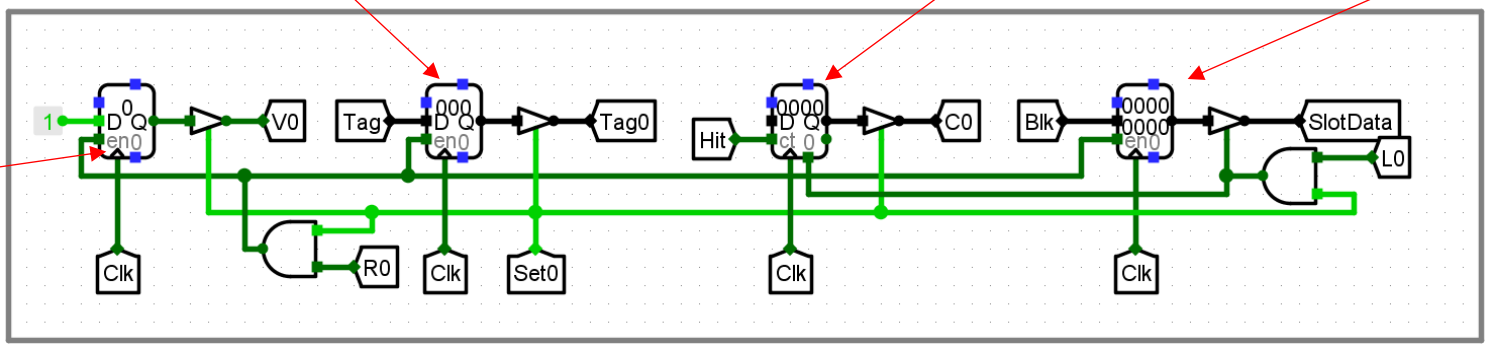
16位计数器 (用于LRU替换算法的计数)

32位寄存器 (存放数据块副本)

1位寄存器 (存放有效位)

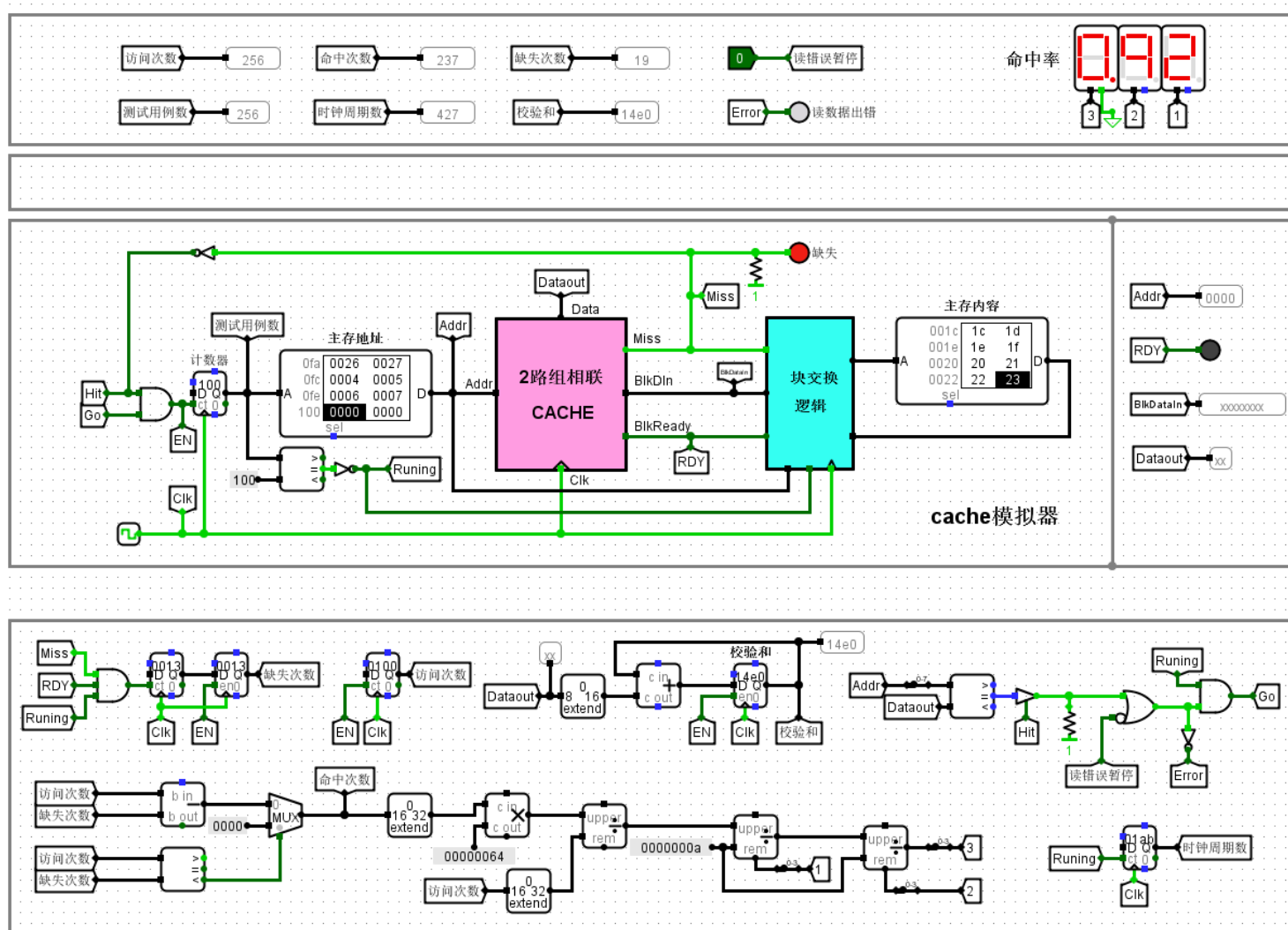
第0组cache

第0行cache



- 6、2路组相联映射方式 cache 测试电路（与直接相联映射方式的cache测试电路相同）

- 测试步骤：先按Ctrl+R，使电路复位；然后按Ctrl+K，启动时钟；等到命中率数字不变（计数器=100），表示表示测试完成。



- 前32次访问情况分析（访问32次，命中26次，不命中（缺失）6次）：
 - (1) 访问 0 1 2 3
 - 访问地址为 0 存储单元，不命中，从主存中装入 0 1 2 3 四个单元，之后访问 1 2 3 存储单元，全部命中；命中3次、不命中1次
 - (2) 访问 8 9 a b
 - 访问地址为 8 存储单元，不命中，从主存中装入 8 9 a b 四个单元，之后访问 9 a b 存储单元，全部命中；命中3次、不命中1次
 - (3) 访问 10 11 12 13
 - 访问地址为 10 存储单元，不命中，从主存中装入 10 11 12 13 四个单元，之后访问 11 12 13 存储单元，全部命中；命中3次、不命中1次
 - (4) 访问 20 21 22 23
 - 访问地址为 20 存储单元，不命中，从主存中装入 20 21 22 23 四个单元，之后访问 21 22 23 存储单元，全部命中；命中3次、不命中1次
 - (5) 访问 8 9 a b
 - 访问地址为 8 存储单元，命中（第（2）步已经将8 9 a b装入cache中了），之后访问 9 a b 存储单元，全部命中；命中4次、不命中0次
 - (6) 访问 4 5 6 7
 - 访问地址为 4 存储单元，不命中，从主存中装入 4 5 6 7 四个单元，之后访问 5 6 7 存储单元，全部命中；命中3次、不命中1次
 - (7) 访问 24 25 26 27
 - 访问地址为 24 存储单元，不命中，从主存中装入 24 25 26 27 四个单元，之后访问 25 26 27 存储单元，全部命中；命中3次、不命中1次
 - (8) 访问 4 5 6 7
 - 访问地址为 4 存储单元，命中（第（6）步已经将8 9 a b装入cache中了），之后访问 5 6 7 存储单元，全部命中；命中4次、不命中0次

cacache (2路组相联映射) 测试用例——主存地址.txt

文件(F)

编辑(E)

格式(O)

查看(V)

帮助(H)

v2.0 raw

0 1 2 3 8 9 a b

10 11 12 13 20 21 22 23

8 9 a b 4 5 6 7

24 25 26 27 4 5 6 7

0 1 2 3 8 9 a b

8 9 a b 20 21 22 23

共256个地址

0 1 2 3

装入第0组、第0行

000000000000 00 00

8 9 a b

装入第2组、第0行

000000000000 10 00

10 11 12 13

装入第0组、第1行

000000000001 00 00

20 21 22 23

装入第0组、第0行

000000000010 00 00

8 9 a b

已经在第2组、第0行

4 5 6 7

装入第1组、第0行

000000000000 01 00

24 25 26 27

装入第1组、第1行

000000000010 01 00

4 5 6 7

已经在第1组、第0行

000000000000 01 00

30 31 32 33 4 5 6 7

24 25 26 27 24 25 26 27

0 1 2 3 0 1 2 3

8 9 a b 20 21 22 23

28 29 2a 2b 4 5 6 7

24 25 26 27 4 5 6 7

0 1 2 3 8 9 a b

0 1 2 3 20 21 22 23

30 31 32 33 24 25 26 27

4 5 6 7 24 25 26 27

0 1 2 3 8 9 a b

20 21 22 23 20 21 22 23

28 29 2a 2b 4 5 6 7

24 25 26 27 4 5 6 7

v2.0 raw

0 1 2 3 4 5 6 7

8 9 a b c d e f

10 11 12 13 14 15 16 17

18 19 1a 1b 1c 1d 1e 1f

20 21 22 23 24 25 26 27

28 29 2a 2b 2c 2d 2e 2f

30 31 32 33 34 35 36 37

38 39 3a 3b 3c 3d 3e 3f

40 41 42 43 44 45 46 47

48 49 4a 4b 4c 4d 4e 4f

56 57

5e 5f

66 67

6e 6f

76 77

7e 7f

86 87

8e 8f

96 97

9e 9f

a6 a7

ae af

b6 b7

d be bf

c6 c7

ce cf

d6 d7

a8 a9 aa ab ac ad de df

e0 e1 e2 e3 e4 e5 e6 e7

e8 e9 ea eb ec ed ee ef

f0 f1 f2 f3 f4 f5 f6 f7

f8 f9 fa fb fc fd fe ff

0 1 2 3 4 5 6 7

8 9 a b c d e f

10 11 12 13 14 15 16 17

18 19 1a 1b 1c 1d 1e 1f

20 21 22 23 24 25 26 27

28 29 2a 2b 2c 2d 2e 2f

30 31 32 33 34 35 36 37

38 39 3a 3b 3c 3d 3e 3f

40 41 42 43 44 45 46 47

48 49 4a 4b 4c 4d 4e 4f

50 51 52 53 54 55 56 57

58 59 5a 5b 5c 5d 5e 5f

60 61 62 63 64 65 66 67

68 69 6a 6b 6c 6d 6e 6f

70 71 72 73 74 75 76 77

访问32次，命中26次，不命中（缺失）6次

命中率=26/32=81.25%

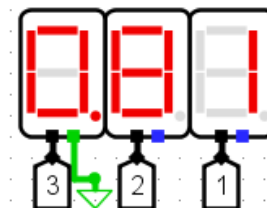
访问次数 32

命中次数 26

缺失次数 6

0 读错误暂停

命中率

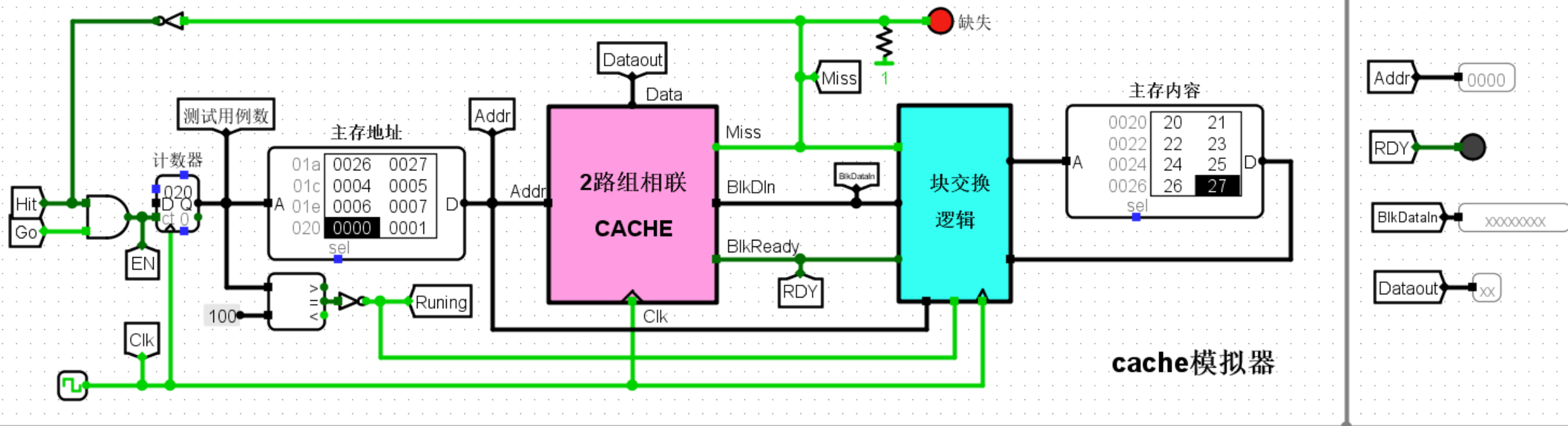


测试用例数 32

时钟周期数 86

校验和 01e0

Error 读数据出错

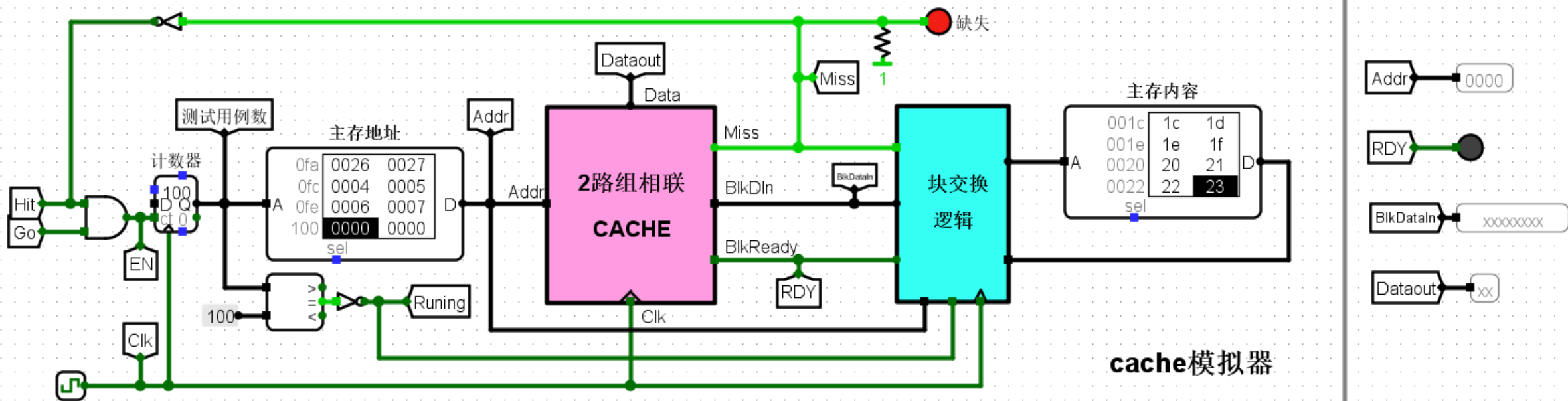
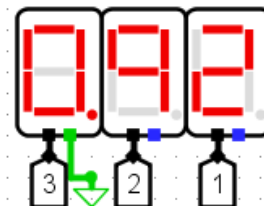


访问256次，命中237次，不命中（缺失）19次

命中率=237/256=92.578%

访问次数 256 命中次数 237 缺失次数 19 0 读错误暂停
测试用例数 256 时钟周期数 427 校验和 14e0 Error 读数据出错

命中率



v2.0 raw

0 1 2 3 8 9 a b

10 11 12 13 20 21 22 23

8 9 a b 4 5 6 7

24 25 26 27 4 5 6 7

0 1 2 3 8 9 a b

8 9 a b 20 21 22 23

28 29 2a 2b 24 25 26 27

4 5 6 7 24 25 26 27

0 1 2 3 8 9 a b

20 21 22 23 20 21 22 23

30 31 32 33 4 5 6 7

24 25 26 27 4 5 6 7

0 1 2 3 8 9 a b

20 21 22 23 20 21 22 23

28 29 2a 2b 4 5 6 7

24 25 26 27 24 25 26 27

0 1 2 3 8 9 a b

8 9 a b 20 21 22 23

30 31 32 33 4 5 6 7

24 25 26 27 24 25 26 27

0 1 2 3 0 1 2 3

8 9 a b 20 21 22 23

28 29 2a 2b 4 5 6 7

24 25 26 27 4 5 6 7

0 1 2 3 8 9 a b

0 1 2 3 20 21 22 23

30 31 32 33 24 25 26 27

4 5 6 7 24 25 26 27

0 1 2 3 8 9 a b

20 21 22 23 20 21 22 23

28 29 2a 2b 4 5 6 7

24 25 26 27 4 5 6 7

- 请同学们分析“2路组相联映射方式 cache 电路”的原理。

- 请同学们分析第33 ~ 第64次cache访问的情况：命中
多少次？不命中（缺失）多少次？

• 7、4路组相联映射方式 cache 电路

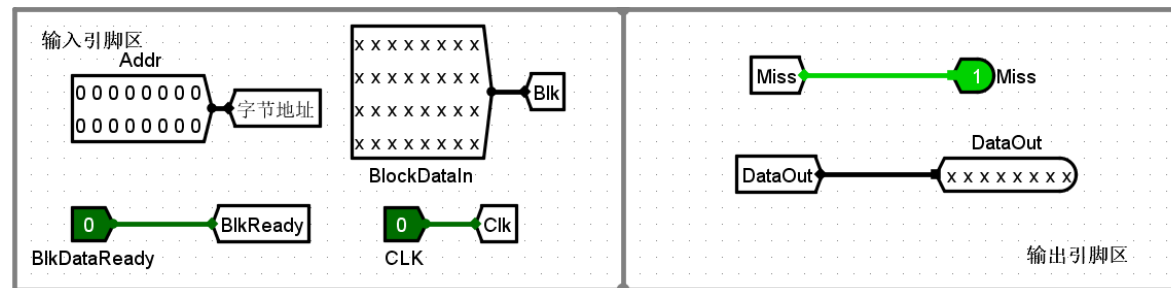
- 4路组相联映射方式：主存地址 = 标记（Tag）+ 组索引（Index）+ 块内偏移（Offset）。
- 假设主存地址=16位；其中，标记=13位，组索引=1位，块内偏移=2位；即cache有2组（ $2^1=2$ ）、每组4行。
- cache共有8行（块）（分为2个组、每组4行），每行存放4个字节（ $2^2=4$ ）。
- 4路组相联映射方式 cache 电路的输入和输出：

- **输入：**

- 主存地址（Addr，字节地址）：16位
- 装入cache的数据块（Blk）：因为cache每行存放4个字节，因此数据块大小=32位
- 数据块准备好信号（BlkReady，1位）
- 时钟信号（CLK）

- **输出：**

- 从cache中得到的数据（DataOut）：1个字节，8位
- 是否命中（Miss）：Miss=1，缺失；Miss=0，命中



4路组相联映射方式 cache 电路的输入和输出

主存地址 (16位) = 标记 (Tag) (13位) + 组索引 (Index, 1位) + 块内偏移 (Offset) (2位)

将主存的标记 (Tag) 与cache组中的4个块的标记 (Tag0~Tag3) 进行比较, 如果有一个比较相等, 并且对应的有效位 (V0~V3) = 1, 则对应的L0~L3输出高电平, 命中 (Hit), 否则, 不命中 (Miss)

如果命中 (Hit=1), 则将cache中的数据块副本 (SlotData, 共32位) 中的某个字节送DataOut (由主存的块内偏移 (Offset) 决定是哪个字节)

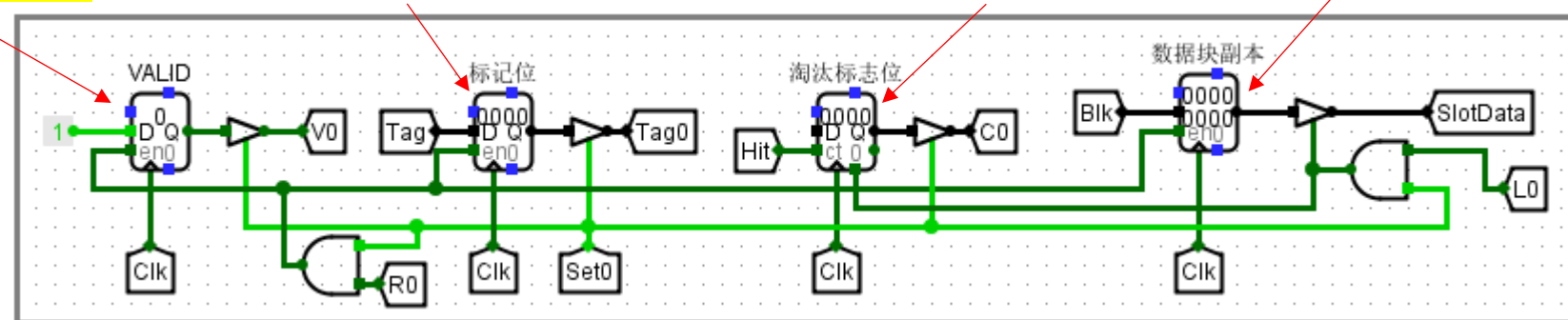
根据组索引 (Index) (1位) 选择2组cache中的某一组

1位寄存器 (存放有效位)

13位寄存器 (存放标记, Tag)

16位计数器 (用于LRU替换算法的计数)

32位寄存器 (存放数据块副本)



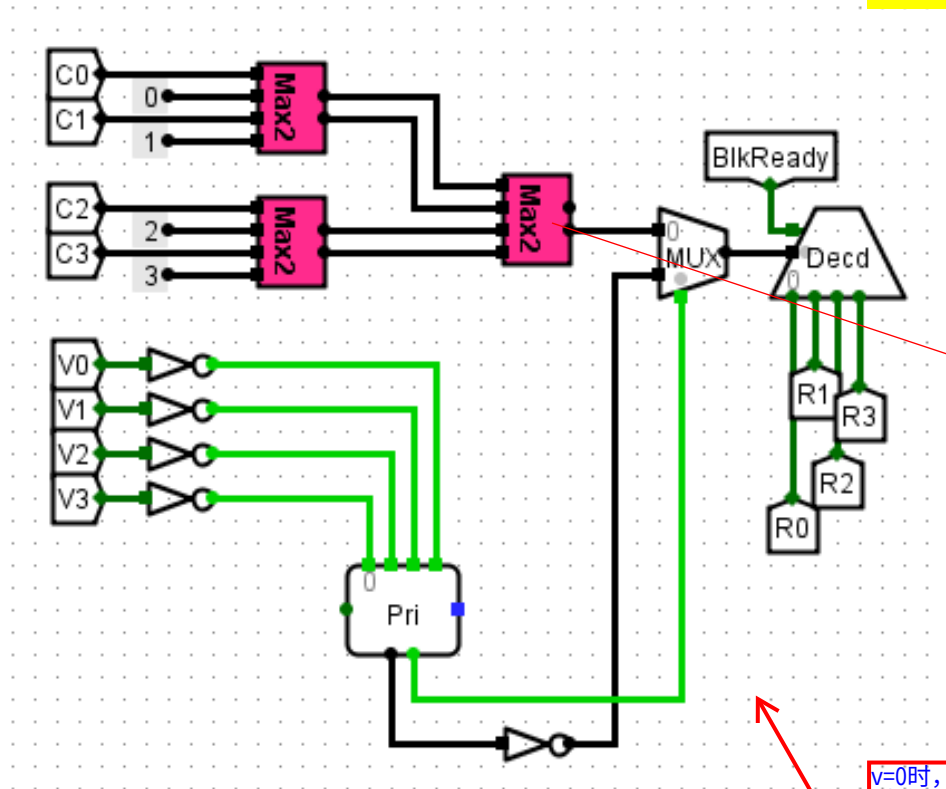
第0组cache

第0行cache

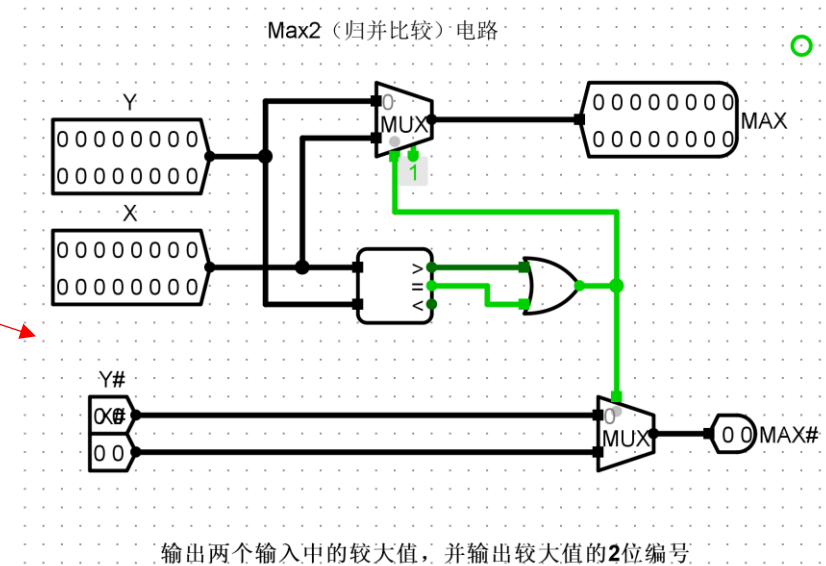
该电路用于实现LRU（近期最少使用）替换算法

输入：C0 ~ C3, V0 ~ V3, BlkReady

输出：R0 ~ R3



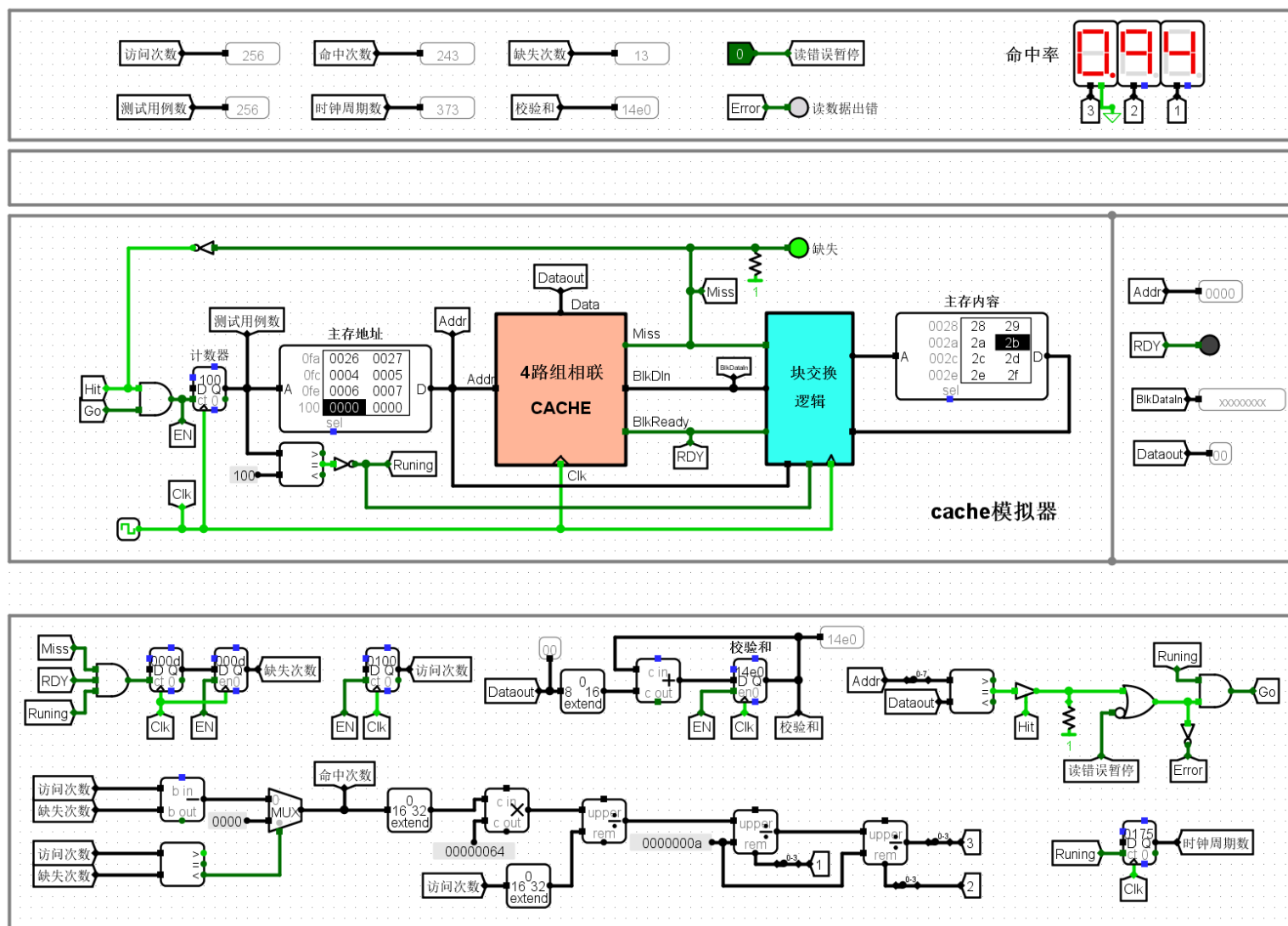
v=0时，表示该块还没有装入，此时不需要根据LRU进行计算
v=1时，说明都占满了，此时才需要采用LRU替换
v从1变成0，应该是换到另一组了



输出两个输入中的较大值，并输出较大值的2位编号

- 8、4路组相联映射方式 cache 测试电路（与直接相联映射方式的cache测试电路相同）

- 测试步骤：先按Ctrl+R，使电路复位；然后按Ctrl+K，启动时钟；等到命中率数字不变（计数器=100），表示表示测试完成。



- 前32次访问情况分析（访问32次，命中26次，不命中（缺失）6次）：
 - (1) 访问 0 1 2 3
 - 访问地址为 0 存储单元，不命中，从主存中装入 0 1 2 3 四个单元，之后访问 1 2 3 存储单元，全部命中；命中3次、不命中1次
 - (2) 访问 8 9 a b
 - 访问地址为 8 存储单元，不命中，从主存中装入 8 9 a b 四个单元，之后访问 9 a b 存储单元，全部命中；命中3次、不命中1次
 - (3) 访问 10 11 12 13
 - 访问地址为 10 存储单元，不命中，从主存中装入 10 11 12 13 四个单元，之后访问 11 12 13 存储单元，全部命中；命中3次、不命中1次
 - (4) 访问 20 21 22 23
 - 访问地址为 20 存储单元，不命中，从主存中装入 20 21 22 23 四个单元，之后访问 21 22 23 存储单元，全部命中；命中3次、不命中1次
 - (5) 访问 8 9 a b
 - 访问地址为 8 存储单元，命中（第（2）步已经将8 9 a b装入cache中了），之后访问 9 a b 存储单元，全部命中；命中4次、不命中0次
 - (6) 访问 4 5 6 7
 - 访问地址为 4 存储单元，不命中，从主存中装入 4 5 6 7 四个单元，之后访问 5 6 7 存储单元，全部命中；命中3次、不命中1次
 - (7) 访问 24 25 26 27
 - 访问地址为 24 存储单元，不命中，从主存中装入 24 25 26 27 四个单元，之后访问 25 26 27 存储单元，全部命中；命中3次、不命中1次
 - (8) 访问 4 5 6 7
 - 访问地址为 4 存储单元，命中（第（6）步已经将8 9 a b装入cache中了），之后访问 5 6 7 存储单元，全部命中；命中4次、不命中0次

cacahe (4路组相联映射) 测试用例——主存地址.txt

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw
0 1 2 3 8 9 a b
10 11 12 13 20 21 22 23
8 9 a b 4 5 6 7
24 25 26 27 4 5 6 7
0 1 2 3 8 9 a b
8 9 a b 20 21 22 23
28 29 2a 2b 24 25 26 27
4 5 6 7 24 25 26 27
0 1 2 3 8 9 a b
20 21 22 23 20 21 22 23
30 31 32 33 4 5 6 7
24 25 26 27 4 5 6 7
0 1 2 3 8 9 a b
20 21 22 23 20 21 22 23
28 29 2a 2b 4 5 6 7
24 25 26 27 24 25 26 27
0 1 2 3 8 9 a b
8 9 a b 20 21 22 23
30 31 32 33 4 5 6 7
24 25 26 27 24 25 26 27
0 1 2 3 0 1 2 3
8 9 a b 20 21 22 23
28 29 2a 2b 4 5 6 7
24 25 26 27 4 5 6 7
0 1 2 3 8 9 a b
0 1 2 3 20 21 22 23
30 31 32 33 24 25 26 27
4 5 6 7 24 25 26 27
0 1 2 3 8 9 a b
20 21 22 23 20 21 22 23
28 29 2a 2b 4 5 6 7
24 25 26 27 4 5 6 7

共256个地址

与直接相联映射方式相同

cacahe (4路组相联映射) 测试用例——主存内容.txt

文件(F) 编辑(E) 格式(O) 查看(V) 帮助(H)

v2.0 raw
0 1 2 3 4 5 6 7
8 9 a b c d e f
10 11 12 13 14 15 16 17
18 19 1a 1b 1c 1d 1e 1f
20 21 22 23 24 25 26 27
28 29 2a 2b 2c 2d 2e 2f
30 31 32 33 34 35 36 37
38 39 3a 3b 3c 3d 3e 3f
40 41 42 43 44 45 46 47
48 49 4a 4b 4c 4d 4e 4f
50 51 52 53 54 55 56 57
58 59 5a 5b 5c 5d 5e 5f
60 61 62 63 64 65 66 67
68 69 6a 6b 6c 6d 6e 6f
70 71 72 73 74 75 76 77
78 79 7a 7b 7c 7d 7e 7f
80 81 82 83 84 85 86 87
88 89 8a 8b 8c 8d 8e 8f
90 91 92 93 94 95 96 97
98 99 9a 9b 9c 9d 9e 9f
a0 a1 a2 a3 a4 a5 a6 a7
a8 a9 aa ab ac ad ae af
b0 b1 b2 b3 b4 b5 b6 b7
b8 b9 ba bb bc bd be bf
c0 c1 c2 c3 c4 c5 c6 c7
c8 c9 ca cb cc cd ce cf
d0 d1 d2 d3 d4 d5 d6 d7
d8 d9 da db dc dd de df
e0 e1 e2 e3 e4 e5 e6 e7
e8 e9 ea eb ec ed ee ef
f0 f1 f2 f3 f4 f5 f6 f7
f8 f9 fa fb fc fd fe ff
0 1 2 3 4 5 6 7
8 9 a b c d e f
10 11 12 13 14 15 16 17
18 19 1a 1b 1c 1d 1e 1f
20 21 22 23 24 25 26 27
28 29 2a 2b 2c 2d 2e 2f
30 31 32 33 34 35 36 37
38 39 3a 3b 3c 3d 3e 3f
40 41 42 43 44 45 46 47
48 49 4a 4b 4c 4d 4e 4f
50 51 52 53 54 55 56 57
58 59 5a 5b 5c 5d 5e 5f
60 61 62 63 64 65 66 67
68 69 6a 6b 6c 6d 6e 6f
70 71 72 73 74 75 76 77

访问32次，命中26次，不命中（缺失）6次

命中率=26/32=81.25%

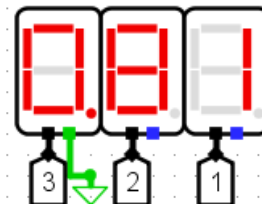
访问次数 32

命中次数 26

缺失次数 6

0 读错误暂停

命中率

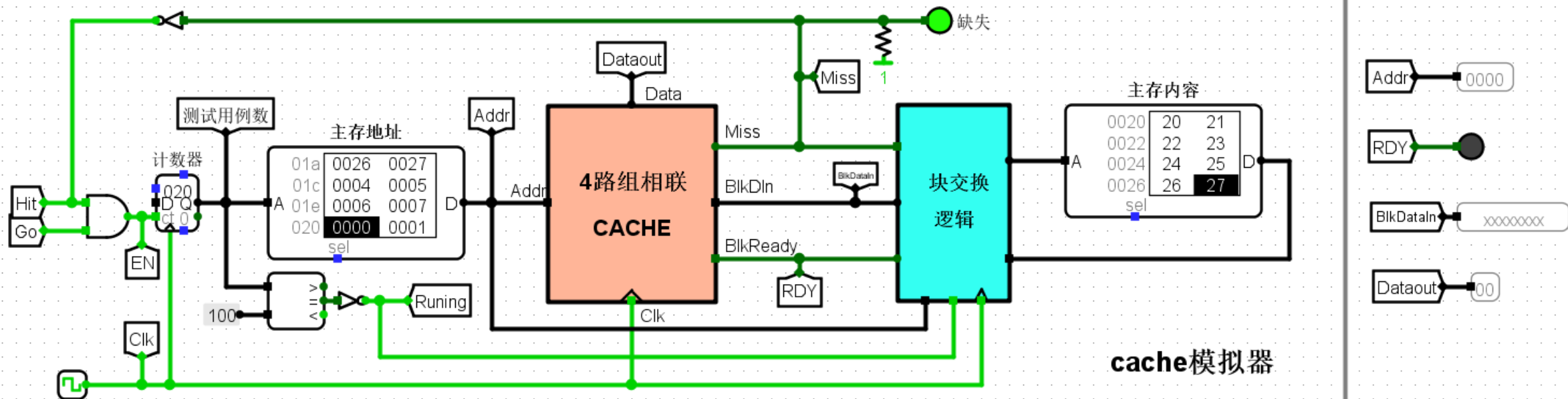


测试用例数 32

时钟周期数 86

校验和 01e0

Error 读数据出错



访问256次，命中243次，不命中（缺失）13次

命中率=243/256=94.92%

访问次数 256

命中次数 243

缺失次数 13

0 读错误暂停

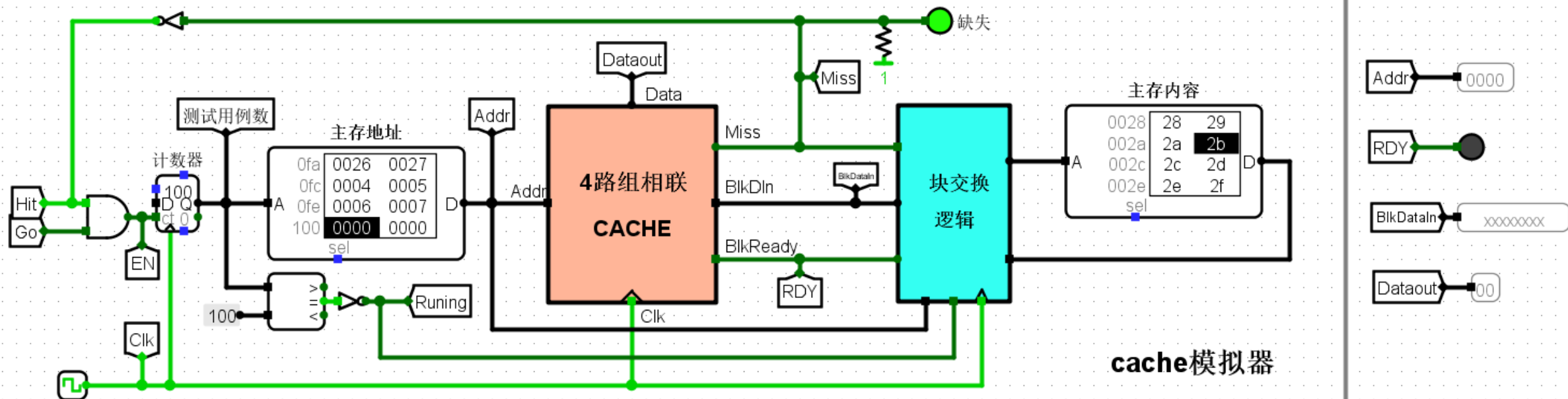
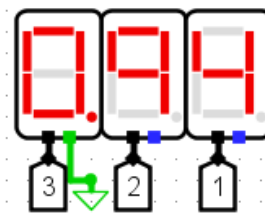
测试用例数 256

时钟周期数 373

校验和 14e0

Error 读数据出错

命中率



v2.0 raw

0 1 2 3 8 9 a b
10 11 12 13 20 21 22 23
8 9 a b 4 5 6 7
24 25 26 27 4 5 6 7

0 1 2 3 8 9 a b
8 9 a b 20 21 22 23
28 29 2a 2b 24 25 26 27
4 5 6 7 24 25 26 27

0 1 2 3 8 9 a b
20 21 22 23 20 21 22 23
30 31 32 33 4 5 6 7
24 25 26 27 4 5 6 7

0 1 2 3 8 9 a b
20 21 22 23 20 21 22 23
28 29 2a 2b 4 5 6 7
24 25 26 27 24 25 26 27

0 1 2 3 8 9 a b
8 9 a b 20 21 22 23
30 31 32 33 4 5 6 7
24 25 26 27 24 25 26 27

0 1 2 3 0 1 2 3
8 9 a b 20 21 22 23
28 29 2a 2b 4 5 6 7
24 25 26 27 4 5 6 7

0 1 2 3 8 9 a b
0 1 2 3 20 21 22 23
30 31 32 33 24 25 26 27
4 5 6 7 24 25 26 27

0 1 2 3 8 9 a b
20 21 22 23 20 21 22 23
28 29 2a 2b 4 5 6 7
24 25 26 27 4 5 6 7

- 请同学们分析“4路组相联映射方式 cache 电路”的原理。

- 请同学们分析第33～第64次cache访问的情况：命中多少次？不命中（缺失）多少次？

• 小结:

- ① 直接相联映射方式: 访问256次, 命中209次, 不命中 (缺失) 47次, 命中率= $209/256=81.64\%$ No.4
- ② 全相联映射方式: 访问256次, 命中248次, 不命中 (缺失) 8次, 命中率= $248/256=96.875\%$ No.1
- ③ 二路组相联映射方式: 访问256次, 命中237次, 不命中 (缺失) 19次, 命中率= $237/256=92.578\%$ No.3
- ④ 四路组相联映射方式: 访问256次, 命中243次, 不命中 (缺失) 13次, 命中率= $243/256=94.92\%$ No.2

参考设计文件： cache 设计实验.circ

七、cache 设计实验

请同学们从以下4组中**任选1组**，设计相应的电路（**挑战性实验**）

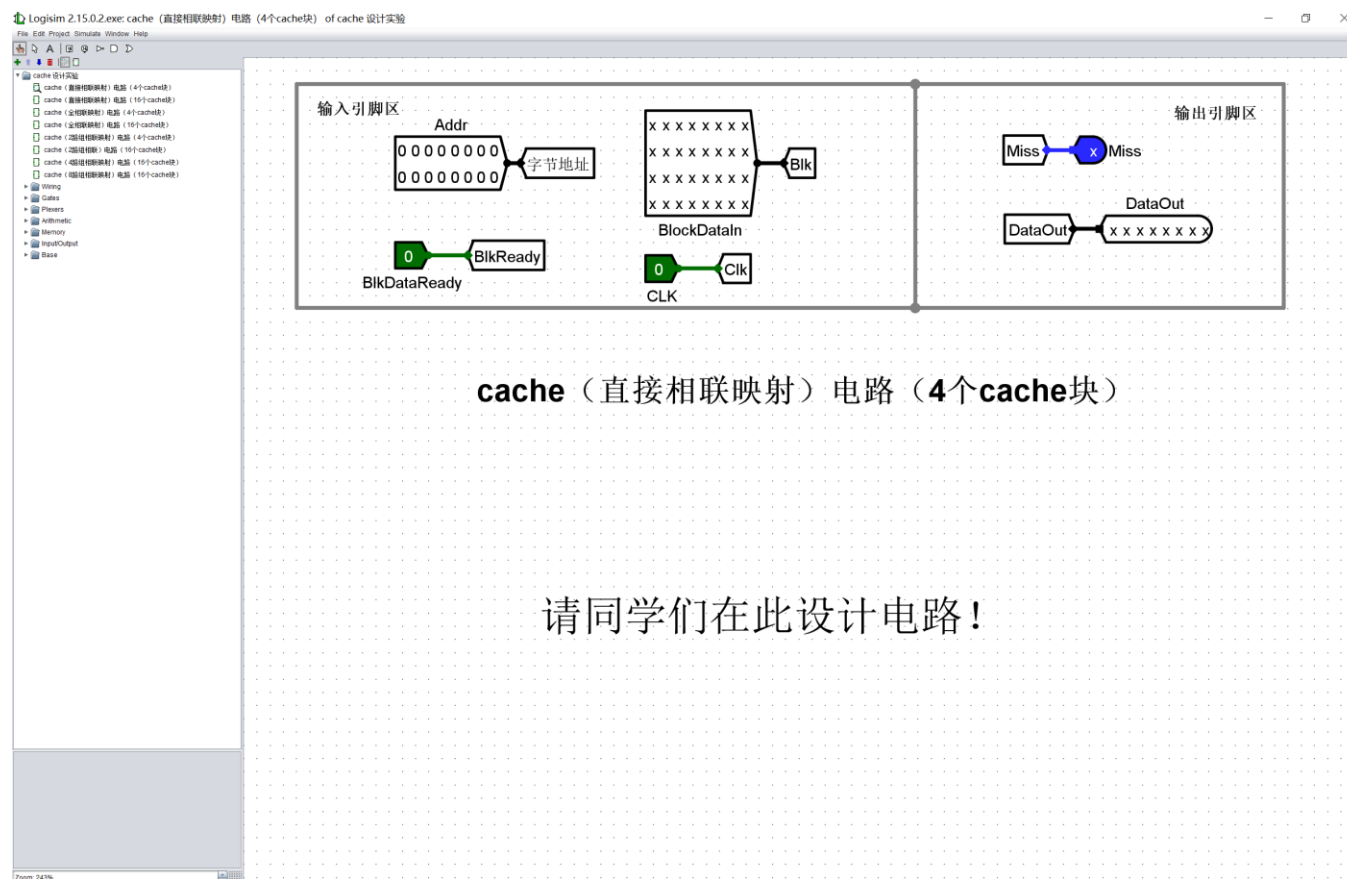
第1组：直接相联映射方式 cache 电路（4个cache块） 加6分
全相联映射方式 cache 电路（16个cache块）

第2组: 直接相联映射方式 cache 电路（16个cache块） 加6分
全相联映射方式 cache 电路（4个cache块）

第3组: 2路组相联映射方式 cache 电路（16个cache块）
直接相联映射方式 cache 电路（16个cache块） 加8分

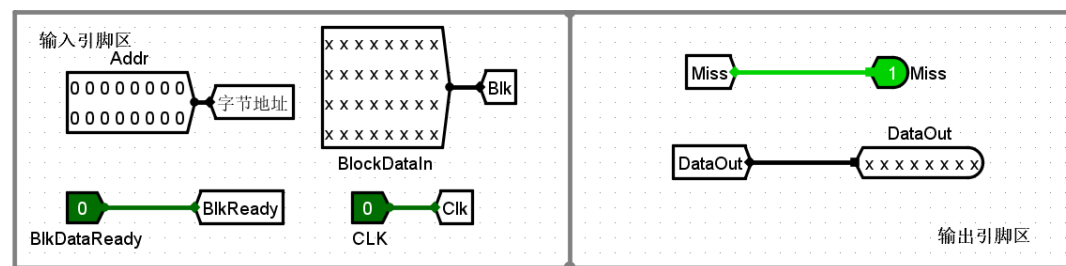
第4组：全相联映射方式 cache 电路（16个cache块） 加8分
8路组相联映射方式 cache 电路（16个cache块）

- 运行“Logisim”，在Logisim中打开“**cache 设计实验.circ**”。
- 也可以直接双击“**cache 设计实验.circ**”，打开该设计文件。



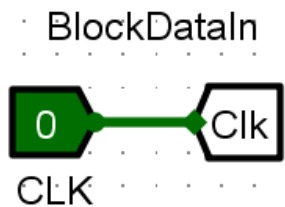
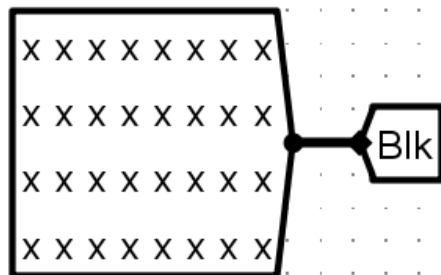
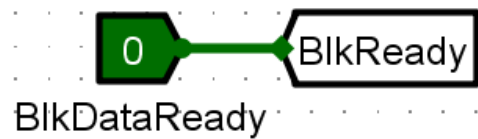
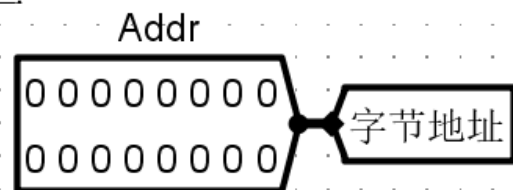
• 1、直接相联映射方式 cache 电路（4个cache块）（挑战性实验）（8个简化为4个）

- 直接相联映射方式：主存地址 = 区地址 (Tag) + 行索引 (Index) + 块内偏移 (Offset)。
- 假设主存地址=16位；其中，区地址=12位，行索引=2位，块内偏移=2位；即cache有4行（块）（ $2^2=4$ ），每行存放4个字节（ $2^2=4$ ）。
- 直接相联映射方式 cache 电路的输入和输出：
 - 输入：
 - 主存地址 (Addr, 字节地址)：16位
 - 装入cache的数据块 (Blk)：因为cache每行存放4个字节，因此数据块大小=32位
 - 数据块准备好信号 (BlkReady, 1位)
 - 时钟信号 (CLK)
 - 输出：
 - 从cache中读出的数据 (DataOut)：1个字节，8位
 - 是否命中 (Miss)：Miss=0，命中；Miss=1，缺失

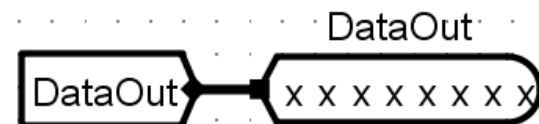


直接相联映射方式 cache 电路的输入和输出（4个cache块）

输入引脚区



输出引脚区



cache（直接相联映射）电路（4个**cache**块）

请同学们在此设计电路！

• 2、直接相联映射方式 cache 电路（16个cache块）（挑战性实验）（8个增加到16个）

- 直接相联映射方式：主存地址 = 区地址 (Tag) + 行索引 (Index) + 块内偏移 (Offset) 。
- 假设主存地址=16位；其中，区地址=10位，行索引=4位，块内偏移=2位；即cache有**16行（块）**（ $2^4=16$ ），每行存放4个字节（ $2^2=4$ ）。

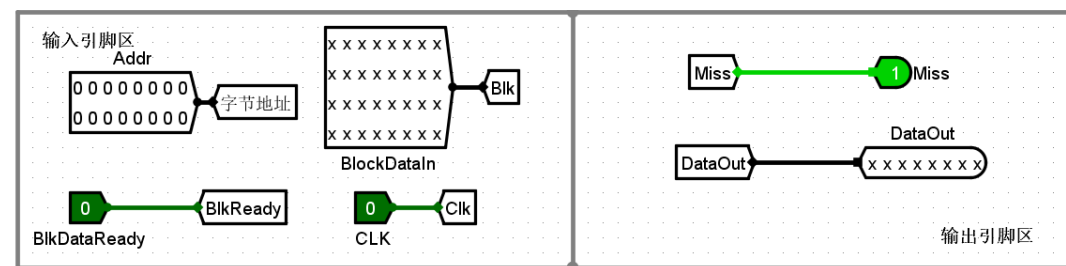
• 直接相联映射方式 cache 电路的输入和输出：

• 输入：

- 主存地址 (Addr, 字节地址)：16位
- 装入cache的数据块 (Blk)：因为cache每行存放4个字节，因此数据块大小=32位
- 数据块准备好信号 (BlkReady, 1位)
- 时钟信号 (CLK)

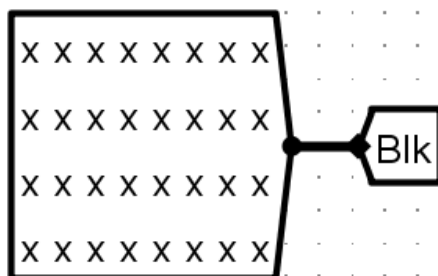
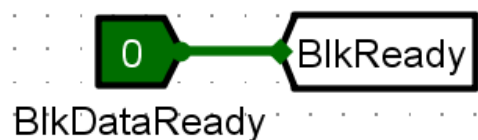
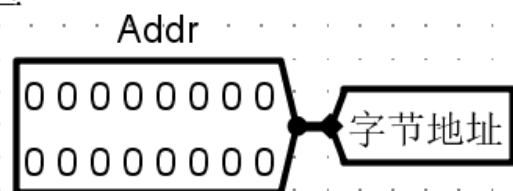
• 输出：

- 从cache中读出的数据 (DataOut)：1个字节，8位
- 是否命中 (Miss)：Miss=0，命中；Miss=1，缺失

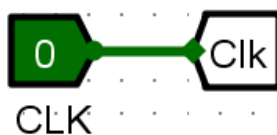


直接相联映射方式 cache 电路的输入和输出（16个cache块）

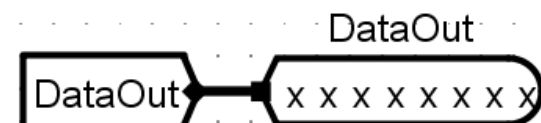
输入引脚区



BlockDataIn



输出引脚区



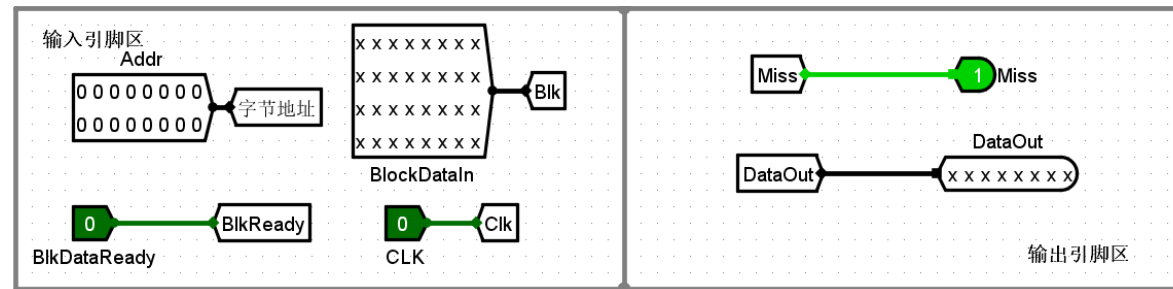
cache（直接相联映射）电路（**16个cache块**）

请同学们在此设计电路！

• 3、全相联映射方式 cache 电路（4个cache块）（挑战性实验）（8个简化为4个）

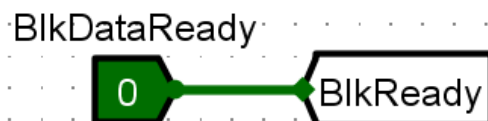
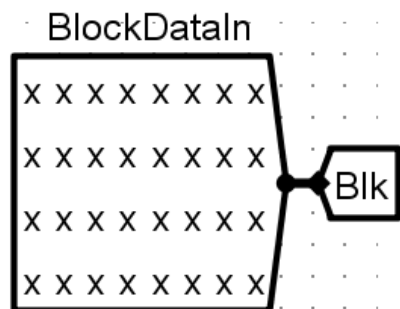
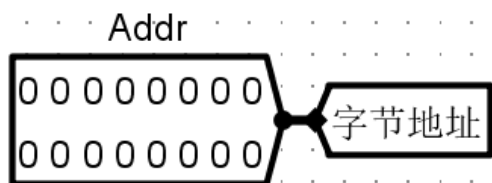
- 全相联映射方式：主存地址 = 主存块地址（Tag）+ 块内偏移（Offset）。
- 假设主存地址=16位；其中，主存块地址=14位，块内偏移=2位。
- cache有4行（块），每行存放4个字节（ $2^2=4$ ）。
- 全相联映射方式 cache 电路的输入和输出：
 - 输入：
 - 主存地址（Addr，字节地址）：16位
 - 装入cache的数据块（Blk）：因为cache每行存放4个字节，因此数据块大小=32位
 - 数据块准备好信号（BlkReady，1位）
 - 时钟信号（CLK）

- 输出：
 - 从cache中得到的数据（DataOut）：1个字节，8位
 - 是否命中（Miss）：Miss=1，缺失；Miss=0，命中

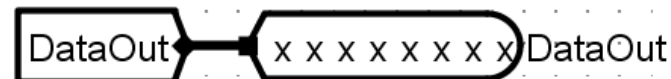


全相联映射方式 cache 电路的输入和输出（4个cache块）

输入引脚区



输出引脚区



cache（全相联映射）电路（4个cache块）

请同学们在此设计电路！

• 4、全相联映射方式 cache 电路（16个cache块）（挑战性实验）（8个增加到16个）

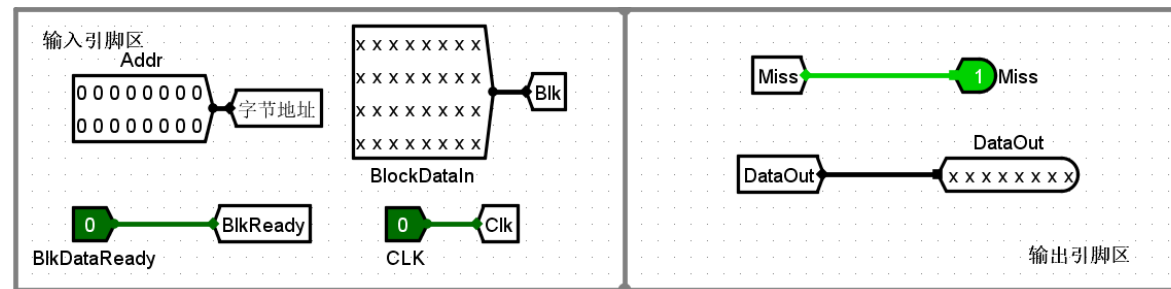
- 全相联映射方式：主存地址 = 主存块地址（Tag）+ 块内偏移（Offset）。
- 假设主存地址=16位；其中，主存块地址=14位，块内偏移=2位。
- cache有16行（块），每行存放4个字节（ $2^2=4$ ）。
- 全相联映射方式 cache 电路的输入和输出：

- 输入：

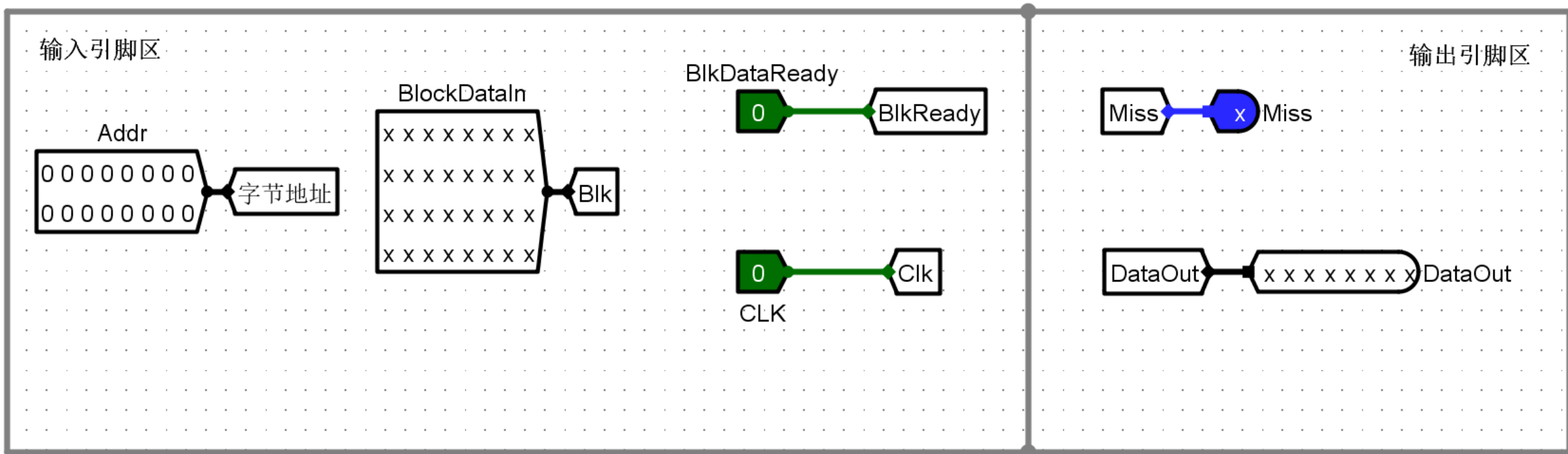
- 主存地址（Addr，字节地址）：16位
- 装入cache的数据块（Blk）：因为cache每行存放4个字节，因此数据块大小=32位
- 数据块准备好信号（BlkReady，1位）
- 时钟信号（CLK）

- 输出：

- 从cache中得到的数据（DataOut）：1个字节，8位
- 是否命中（Miss）：Miss=1，缺失；Miss=0，命中



全相联映射方式 cache 电路的输入和输出（16个cache块）



cache（全相联映射）电路（16个cache块）

请同学们在此设计电路！

• 5、2路组相联映射方式 cache 电路（16个cache块）（挑战性实验）（8个增加到16个）

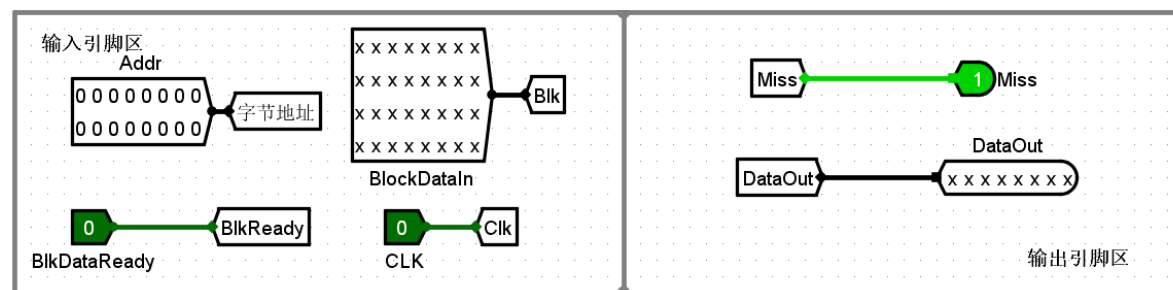
- 2路组相联映射方式：主存地址 = 标记（Tag）+ 组索引（Index）+ 块内偏移（Offset）。
- 假设主存地址=16位；其中，标记=11位，组索引=3位，块内偏移=2位；即cache有8组（ $2^3=8$ ）、每组2行（块）。
- cache共有16行（块）（分为8个组、每组2行（块）），每行存放4个字节（ $2^2=4$ ）。
- 2路组相联映射方式 cache 电路的输入和输出：

- **输入：**

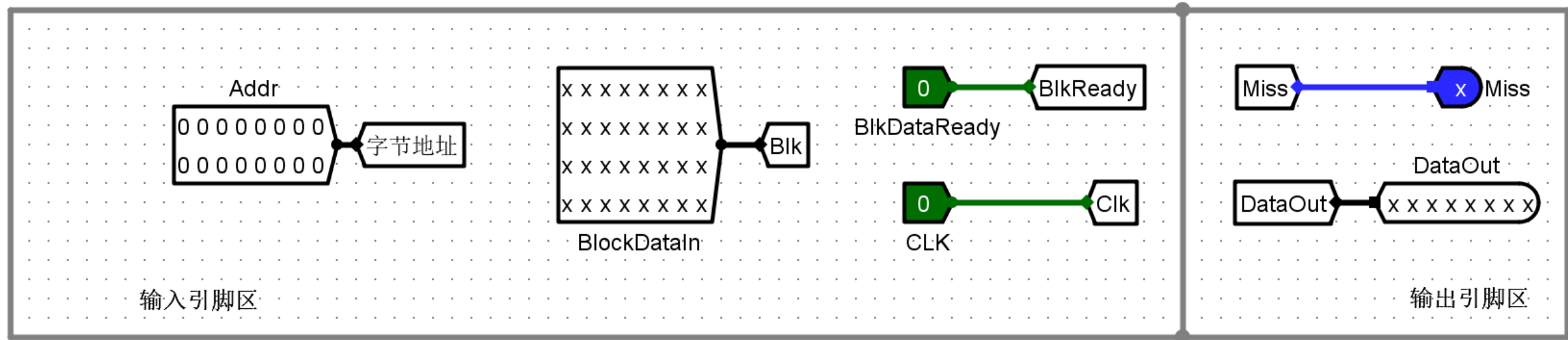
- 主存地址（Addr，字节地址）：16位
- 装入cache的数据块（Blk）：因为cache每行存放4个字节，因此数据块大小=32位
- 数据块准备好信号（BlkReady，1位）
- 时钟信号（CLK）

- **输出：**

- 从cache中得到的数据（DataOut）：1个字节，8位
- 是否命中（Miss）：Miss=1，缺失；Miss=0，命中



2路组相联映射方式 cache 电路的输入和输出（16个cache块）



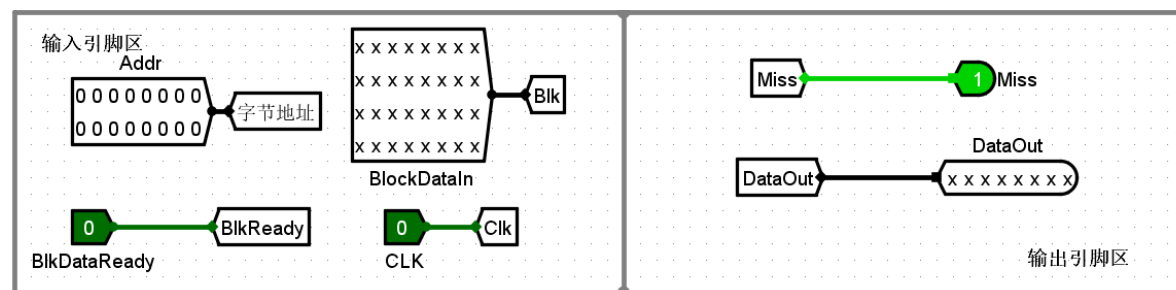
cache（2路组相联映射）电路（4个cache块）

请同学们在此设计电路！

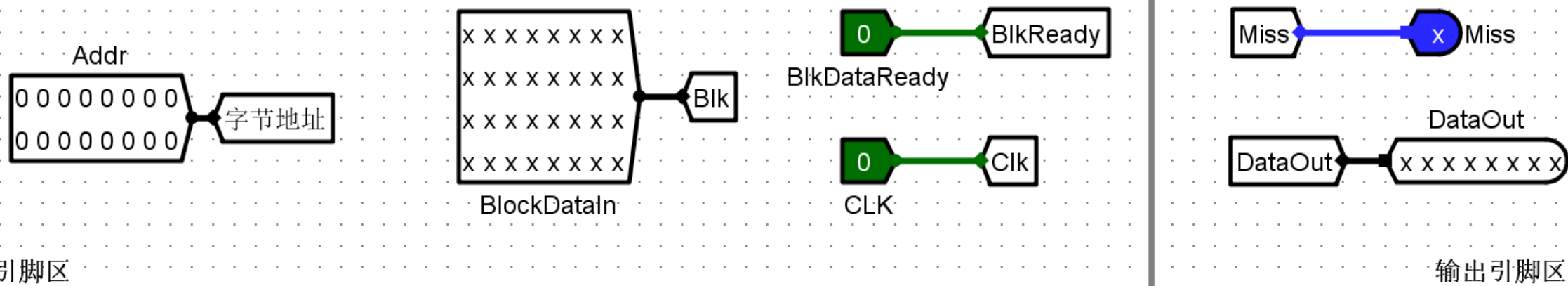
- 6、8路组相联映射方式 cache 电路（16个cache块）（挑战性实验）（前面没有）

- 4路组相联映射方式：主存地址 = 标记（Tag）+ 组索引（Index）+ 块内偏移（Offset）。
- 假设主存地址=16位；其中，标记=13位，组索引=1位，块内偏移=2位；即cache有2组（ $2^1=2$ ）、每组8行（块）。
- cache共有16行（块）（分为2个组、每组8行（块）），每行存放4个字节（ $2^2=4$ ）。
- 2路组相联映射方式 cache 电路的输入和输出：
 - 输入：
 - 主存地址（Addr，字节地址）：16位
 - 装入cache的数据块（Blk）：因为cache每行存放4个字节，因此数据块大小=32位
 - 数据块准备好信号（BlkReady，1位）
 - 时钟信号（CLK）

- 输出：
 - 从cache中得到的数据（DataOut）：1个字节，8位
 - 是否命中（Miss）：Miss=1，缺失；Miss=0，命中



8路组相联映射方式 cache 电路的输入和输出（16个cache块）



cache（8路组相联映射）电路（16个cache块）

请同学们在此设计电路！

需要同学们完成的设计实验的内容

- ① 存储器扩展电路（设计实验）——文件名：存储器扩展设计实验.circ
- ② 寄存器文件电路（4个寄存器）（设计实验）——文件名：寄存器文件设计实验.circ
- ③ 寄存器文件电路（4个寄存器，R0=0）（设计实验）——文件名：寄存器文件设计实验.circ
- ④ cache 设计实验（挑战性实验，从4组中任选1组）——文件名：cache 设计实验.circ

- 完成挑战性实验，本次实验成绩加分参照前述PPT

-

实验报告文件提交要求

- 1、请按照FTP上的实验报告模板（**厦门大学计算机组成原理实验报告样本（第3次实验）.docx**），撰写实验报告（Word版本），实验报告命名为：**学号+姓名+第3次实验报告**
- 2、请将实验报告和3个设计文件打包为1个压缩文件，压缩文件命名为：**学号+姓名+第3次实验.zip**
- 3、将压缩文件上传到**数字化教学平台**上，第3次实验报告提交**截止时间**（2周内）